



CentraleSupélec



Big Data Management & Analytics Master

FROM SINGLE- TO DOUBLE-PAIR: IMPROVING WI-FI
HUMAN ACTIVITY RECOGNITION ON ESP32
DEVICES FOR ELDERLY CARE

Stefanos KYPRITIDIS

[Code Link](#)

Advisor: Piotr ANTONIK - CentraleSupélec & ZoeCare CTO piotr.antonik@centralesupelec.fr

Acknowledgments

I would first like to express my sincere gratitude to my startup, ZoeCare, and especially to my advisor and CTO, Piotr Antonik, for providing me with the opportunity to carry out my thesis within the company. I also extend my thanks to CEO Thomas Saphir and the entire data science team for their continuous guidance through this new topic and for patiently answering all of my questions. Special recognition goes to the hardware team, particularly Livio Dadone, for supplying the precise ESP32-S3 hardware required and for tweaking it specifically for my double-pair configuration needs.

Academically, I am deeply thankful to my supervisor for this semester, Nacera Seghouani, whose guidance and recommendations were crucial, particularly in preparing my final presentation. I also wish to thank Professor Esteban Zimányi, the program coordinator, for organizing and executing such a wonderful master's program. My appreciation extends to all the professors from the universities of ULB, UPC, and CentraleSupélec who have been part of this journey in the Big Data Management & Analytics Master program.

A very practical and crucial thank you goes to the Fondation of CentraleSupélec, and specifically to Soaniaina Andriamahefa. When my laptop unexpectedly stopped working in the middle of my thesis, their swift coordination in lending me a replacement ensured I lost no time and could keep my research on track.

Finally, I want to thank my family for their continuous support throughout my thesis.

Declaration on the Use of Generative AI Tools

In accordance with the UPC guidelines on the use of generative artificial intelligence, I declare the following:

Tools and versions.

- **Claude** - Anthropic, Opus 4.8, accessed June 2026.
- **Claude** - Anthropic, Opus 5, accessed July and August 2026.

Purpose of use. The tools were used to improve the quality of the text (spelling, grammar, punctuation, phrasing and clarity).

Sections concerned. The tools were used solely on the written text and tables of the report, for language editing throughout the manuscript. No figures or images were generated with AI tools.

Review and editing. Every suggestion was reviewed, verified and, where necessary, updated by me. No output was incorporated without checking its correctness against primary sources and my own results.

Statement of responsibility. These tools are not authors or co-authors of this thesis. The work is an accurate representation of my own work and intellectual contribution, and I assume full responsibility for the accuracy, integrity, and correctness of all material it contains.

Contents

1	Introduction	1
1.1	Context of the thesis	1
1.2	Problem and challenges	2
1.3	Objective of this thesis	3
1.4	Contributions	4
1.5	Outline of the report	5
2	Related Work	7
2.1	HAR technology categories	7
2.2	The Evolution of Device-Free Wi-Fi Sensing	8
2.3	Wi-Fi Sensing for Elderly Care	11
2.4	HAR challenges	13
2.4.1	Data	13
2.4.2	Model Generalization	14
2.4.3	Data Leakage	14
2.4.4	Edge deployment	15
2.4.5	Detection Efficiency and Real-Time Segmentation	15
2.4.6	Long-Term Model Adaptation	15
2.4.7	Multiple TX/RX Links	16
2.5	Self-Supervised Learning for HAR	17
2.6	Transfer Learning for HAR	17
2.7	Summary and Problem Identification	19
3	Background	21
3.1	Clinical Context: Daily Activities, Mobility and Functional Decline	21
3.2	Theoretical Foundations of Wi-Fi CSI	22
3.2.1	From RSSI to Channel State Information	22
3.2.2	Amplitude and Phase as Complementary Signals	23
3.2.3	Encoding Motion: Static and Dynamic Multipath	24
3.2.4	Spatial Selectivity and the Fresnel-Zone Blind-Spot Problem	24
3.2.5	Orientation Dependence and the Case for Orthogonal Links	25
3.2.6	Sensing Topologies and Resolution Limits	26
3.3	Specifications of the ESP32-S3	27
3.4	Denoising Filters	27
3.4.1	Outlier Removal: Median and Hampel Filters	28
3.4.2	Frequency-Selective Smoothing: The Butterworth Low-Pass Filter	30
3.4.3	Wavelet Transform	30

4	Our Methodology and Approach	33
4.1	System Requirements & Hardware Analysis	33
4.2	Proposed Architecture: Double TX-RX Setup	33
4.3	Data Collection Procedure	36
4.4	Preprocessing pipeline	38
4.5	Windowing	38
4.6	Evaluation methodology	40
4.7	Learning Models	41
4.7.1	Classical feature-based pipeline	42
4.7.2	Deep temporal convolutional models	42
4.7.3	Per-pair processing: siamese trunk and fusion	43
4.7.4	Per-window normalisation	43
4.7.5	Augmentation and regularisation	44
4.7.6	Test-time adaptation: Adaptive Batch Normalisation	44
4.7.7	Training procedure	44
4.8	Cross-Environment Experiment Data Collection	45
4.8.1	Sensing setup	45
4.8.2	Activity set	46
4.8.3	Multi-environment collection	48
4.8.4	Cross-environment evaluation protocol	49
5	Experiments and Evaluation	51
5.1	Reproducing Cross-Environment Benchmark	51
5.2	Preliminary Experiment: Double-Pair Cross-Day	53
5.3	Preliminary Experiment: Fine-Grained Kitchen Activity	54
5.4	Cross-Environment Experiment Generalization	56
5.4.1	Stage 1: broad pipeline grid search	56
5.4.2	Self-supervised pretraining and the siamese shared trunk	57
5.4.3	Diagnosing the residual confusions: what amplitude cannot encode	58
5.4.4	Recipe refinement and component ablation	59
5.4.5	Timestamp-grid resampling	60
5.4.6	Single-pair versus double-pair deployment	61
5.4.7	Extending the label set with a <code>no_action</code> class	64
5.4.8	Real-time evaluation: live deployment on cued sessions	65
5.4.9	Continuous recordings: evaluation without segment boundaries	66
6	Conclusion and Perspectives	71
6.1	Summary of the work and of the results	71
6.2	Limitations	73
6.3	Perspectives	74
6.4	Working in a team	75
A	Supplementary Material	77
A.1	Work Timeline	78
A.2	Reproduction of published ESP32 benchmarks: protocol and results	80
A.2.1	Preprocessing, models and protocols	80

A.2.2	ESP-Fi HAR: cross-subject and same-environment recognition . . .	81
A.2.3	Subject-disjoint cross-environment control	82
A.3	Preliminary cross-day double-pair experiment: applied workflow	83
A.3.1	Data collection setup	83
A.3.2	Cross-day protocol	84
A.3.3	Classical feature pipeline and grid search	85
A.3.4	Where the errors live: the static-state quartet	86
A.3.5	Deep-learning sweep	87
A.3.6	Merged three-class confirmation	87
A.4	Fine-grained kitchen experiment: data collection and workflow	88
A.4.1	Data collection	88
A.4.2	Phase 1-classical pipeline and feature bank	93
A.4.3	Phase 2-deep-learning architecture sweep	94
A.4.4	The AGC correction and the deployed pipeline	95
A.4.5	Per-recording results on the continuous routines	98
A.4.6	Pipeline selection: Composite Validation Score and grid search . .	102
A.4.7	Exploratory and failed segmentation	104
A.5	Cross-environment experiment: applied workflow	105
A.5.1	Stage-1 grid search: shared harness and preprocessing glossary . .	105
A.5.2	Stage-1 grid search: classical feature grid	105
A.5.3	Stage-1 grid search: deep-learning grid	107
A.5.4	Stage-1 grid search: single-pair companion grids	109
A.5.5	Self-supervised pretraining details	110
A.5.6	Final training recipe: full specification	111
A.5.7	Timestamp-grid resampling variants	111
A.5.8	The <code>no_action</code> class: construction and full results	111
A.5.9	Real-time deployment: rooms, adaptation and confusion matrices .	112
A.5.10	Continuous recordings: mode specification and remaining timelines	115

Introduction

1.1 Context of the thesis

The number of older adults is growing fast. There were 703 million people aged 65 and over in 2019, and there will be around 1.5 billion by 2050 [1]. About 90% of them would rather stay in their own home than move to a nursing home [2], and many already live alone: more than a third of older people in Europe, and close to 29% of the 46 million community-dwelling older adults in the United States. Someone living alone who falls or has an accident may have no way of calling for help, and that is the situation an always-on, non-intrusive sensor is supposed to cover [3].

What matters clinically, though, is rarely a single action. It is the resident’s functional status: whether they can still carry out the activities that independent living requires. Chung et al. [4] sort the behaviours worth observing into activities of daily living, instrumental activities such as preparing a meal, and mobility behaviours such as moving between rooms or sitting and lying down. Functional decline in these behaviours begins years before the symptoms of dementia appear, which makes them an early warning signal rather than a record written after the fact [5]. What a deployed system is expected to produce is accordingly the daily count and duration of each behaviour [4], and what makes such numbers informative is that activity levels are otherwise fairly stable over weeks and months, so a decrease stands out [3]. The useful output is therefore a pattern over days, not a perfect label on every second.

The difficulty is measuring any of this in a real home. Each of the obvious sensors has a drawback for this particular population. Wearables are accurate, but older adults may simply forget to put them on [6], and they usually depend on a smartphone application that a large share of this group does not own or does not know how to use [4]. Cameras see a lot, but they cannot see through walls, they need decent lighting, raise obvious privacy concerns, and are not acceptable in a bathroom where falls can happen [6, 7]. Motion sensors are cheap and unobtrusive, but they only report that something moved, never what was being done [4].

Device-free Wi-Fi sensing avoids all three objections at once. A person moving through a room changes the way radio waves travel between a transmitter and a receiver, and those changes can be read straight from the wireless channel: nothing is worn, nothing is carried, no camera is involved, and the signal goes through walls and furniture [8, 4]. The early systems used the Received Signal Strength Indicator, a single power value per packet, which turned out to be too coarse and too unstable over time [9]. Channel State Information replaced it: it reports amplitude and phase on every OFDM subcarrier, so it describes the channel in much finer detail [10]. CARM [11] is the work that established the direction, linking the frequency of the CSI variations to the speed of the moving body

and reaching 96% accuracy on eight activities, of which more than 80% survived in rooms it had never seen.

For years, reading CSI meant a host computer with a modified Intel 5300 network card [12]. This is impractical for sensing since a host computer is bulky, power hungry, and not something one installs in a room or apartment. Microcontrollers changed that: the ESP32 family exposes CSI on its own [13], and an ESP32-C3 node costs roughly 12× less and draws over 95% less power than an Intel 5300 setup [14]. A sensor becomes a small board plugged into a wall socket, which is what makes a home deployment realistic at all. ESP32 is the hardware this thesis works with.

1.2 Problem and challenges

Reported accuracies on ESP32 hardware look excellent. Everyday activities and falls are classified at 93% with an LSTM running on a Raspberry Pi [15], seven classes at 93.5% with a quantised GRU [16], and long-range presence and just two-class activity detection reaches around 97% in line of sight [17]. None of this means the problem is solved, because these numbers are not measured under realistic continuous monitoring deployment conditions. Four difficulties stand in the way, and they are the ones this thesis works on.

The signal describes the room, not only the person. CSI contains every reflection in the space: the walls, the furniture, where the devices happen to be. A model trained in one room therefore learns the room together with the activity, and it degrades once it is moved. The standard fix in prior work is to collect new data in the new environment and retrain [18], which is not something one can ask of a family installing a monitor at home. The same drift happens with people. In detail, subject height, weight and even clothing change how the signal is attenuated [19]. Cross-environment and cross-user generalisation are, unsurprisingly, listed among the central open problems of the field [7, 19].

Labelled data is expensive, and there is little of it. Recording and annotating CSI is slow manual work, so Wi-Fi datasets stay small next to those of computer vision and thus models overfit them easily [19, 20]. Self-supervised pretraining and transfer learning are the two standard answers [21, 20], but their reliance on a large unlabelled pretraining pool is itself a challenge for Wi-Fi, where such data is far harder to amass than in vision [21].

The way results are measured hides real failures. Two habits inflate published scores. The first is splitting training and test data without separating the variable that is evaluated, for instance, subjects involved. When the same volunteers appear on both sides, the model learns the person rather than the activity, and repairing the split on one widely used benchmark cost between 26 and 32 accuracy points [22]. The second is less discussed and more damaging. Benchmarks score pre-segmented clips, in which every window holds exactly one activity that starts and ends cleanly. A deployed monitor gets no such help: activities follow one another, most of the stream is an empty room or an idle

period [23], and the sliding window regularly straddles two activities at once. A model can therefore score well on clips yet collapse once those boundaries are removed.

A single link has blind spots. A Wi-Fi link responds strongly to motion that crosses it and much more weakly to motion that runs along it, so the same activity leaves a strong or a faint signature depending largely on where the person is and how their path is oriented relative to the link. Despite this, single-link deployments dominate the WiFi sensing literature, and leveraging several diversely placed links is named as an open challenge [23]. Where several links do appear, they are usually spread over different rooms to extend coverage [4, 3] or kept in direct line of sight in the same room [24, 25]. And adding links is not automatically good: feeding a noisy or obstructed link into the model can drag down the good ones, an effect known as dilution [26].

1.3 Objective of this thesis

This thesis was carried out at ZoeCare, a startup working on Wi-Fi sensing for elderly care. In particular, the thesis studies one specific configuration: two ESP32-S3 transmitter-receiver pairs in the same room, the two transmitters co-located and the two receivers placed so that the two direct line-of-sight paths are mutually orthogonal (Section 4.2). The reasoning is simple. Motion that is nearly invisible to one link is, by construction, close to perpendicular to the other, so the two links cover each other’s blind spots. As far as we know, two co-deployed ESP32 pairs used this way in a single room and in non-direct line-of-sight (NLOS) conditions, had not been studied before.

The main question is the one a deployment has to answer: can two low-cost ESP32-S3 pairs recognise the typical daily activities of a person in an unseen environment under NLOS conditions, and how much does the second pair really contribute? It splits into four sub-questions, one per experiment in Chapter 5:

1. Where does the field stand? How well does a single ESP32 pair generalise to an unseen room, on published data and with published models (Section 5.1)?
2. Which activities can a double pair separate at all, when the same room is recorded on different days and both links are in NLOS (Section 5.2)?
3. How fine can the activities get, and does a score measured on controlled clips still hold on continuous, naturally performed routines (Section 5.3)?
4. Can a model trained in two rooms work in a third one it has never seen, what makes that possible, and is the second pair worth its cost (Section 5.4)?

Three constraints frame every answer below. All in-house recordings were made by a single person (the author), who was the experimenter and the subject at the same time, so our results are cross-room but same-subject. Only CSI amplitude is used, because the phase these single-antenna, unsynchronised boards report is unstable. And the models are kept deliberately small, since the longer-term target is inference on the edge device rather than on a laptop.

1.4 Contributions

The main contributions of this work are the following.

1. **A double-pair NLOS setup and a multi-room dataset.** Four ESP32-S3 boards form two orthogonal links, and the same six daily activities of walking, preparing a meal, sitting transition, standing up from the chair transition, lying-down transition, and getting up off the floor, were recorded in three different rooms.
2. **A pipeline that transfers to a room it has never seen.** A broad grid search over different model approaches plateaued at 0.58 macro- F_1 and would not move. What unlocked it was the architecture: a siamese trunk that processes each link with shared weights, a targeted augmentation recipe, label-free test-time adaptation and resampling onto a wall-clock timestamp grid together bring leave-one-room-out performance to 0.766 macro- F_1 on six classes and 0.918 on a four-class grouping.
3. **A measured answer on the value of the second pair.** Averaging the two links' probabilities beats the best single-link deployment by 4.2 points of macro- F_1 on six classes and 3.0 on four classes, and beats a single-link system trained conventionally on its own link by 8 to 16 points. We also find that where a link is placed matters about as much as how many links there are, and that the second link's main benefit is doubling the data collected per session rather than using both links at inference. This gives a practical recipe: collect with two pairs, deploy with one.
4. **Two negative results that are worth stating.** Held static postures (sitting, standing, lying still, and the empty room) cannot be told apart across days with this hardware, because a motionless body leaves no signature in the motion band. Also, the errors that remain in the final model sit almost entirely in `sit↔lying` and `standing up from the chair↔ getting up from the floor`: the preprocessing used that makes the pipeline transfer between rooms is also what removes the near-static information telling apart a chair from the floor.
5. **Evidence that clip-level scores overestimate deployability.** On fine-grained kitchen gestures, the same pipeline reaches 0.91 macro- F_1 on controlled clips and 0.45 on continuous recordings of real coffee- and tea-making routines, with nothing changed but the way the data is presented to it. We take this as an argument that continuous, naturally executed routines belong in the evaluation process.
6. **A system that was tested and actually runs.** The final model was trained on all three rooms, implemented as a live inference program, and ran live in rooms that were not part of the training data. It achieved performance slightly above the offline estimate. Also, calibration to a new room for the live program takes under a minute of unlabelled cued windows, with no annotation and no retraining. In addition, an idle `no_action` class was added at almost no cost to the other classes, and on continuous recordings the predictions follow the performed sequence, with the errors mostly concentrated at the boundaries-transitions between activities.

1.5 Outline of the report

The rest of the report is organised as follows.

Chapter 2 reviews the related work. In detail, that includes the three main families of activity recognition, how device-free Wi-Fi sensing moved from RSSI to CSI and then to low-cost ESP32 boards, what has been done specifically for elderly care, and the challenges that still keep the field in the laboratory. It closes with the gap this thesis addresses.

Chapter 3 introduces the necessary concepts and notations. In particular, it presents the clinical meaning of daily activities, the theory of Wi-Fi CSI, the reason a single link is only sensitive in certain directions, the specifications of the ESP32-S3, and the denoising filters used later on.

Chapter 4 describes the methodology: the hardware and the orthogonal double-pair placement, the preprocessing chain, the windowing stage, the different evaluation protocols, the two model families with the model pipeline components, and finally how the data was collected for the main experiment.

Chapter 5 presents the four experiments and their results, in the order they were carried out. In detail, the experiments were the reproduction of a published cross-environment benchmark, the cross-day preliminary experiment, the preliminary fine-grained kitchen experiment, and the main cross-environment study, followed by the real-time deployment and the continuous recordings.

Chapter 6 gathers the results, states the limitations, proposes the next step that follows from each of them, and ends with a note on working inside the team.

Appendix A contains the full protocols, grid searches, result tables, confusion matrices and implementation details of every experiment.

Related Work

This chapter reviews the research this thesis builds on. We first present the three main ways of performing Human Activity Recognition (HAR) and explain why Wi-Fi sensing is attractive for continuous monitoring at home. We then follow how device-free Wi-Fi sensing evolved, from the early works based on signal strength (RSSI) to fine-grained Channel State Information (CSI), and finally to the low-cost ESP32 devices used in this work. Then, we take a close look at elderly care, which is the application that motivates this thesis. Next, we discuss the challenges that still keep Wi-Fi HAR inside the laboratory, such as data scarcity, poor generalization across environments, users and days, the limits of edge hardware and the use of multiple transmitter-receiver links, together with self-supervised and transfer learning as two ways of reducing the need for labelled data. The chapter closes with the gaps we identified and the directions we chose to follow, whose technical concepts are introduced in the next chapter.

2.1 HAR technology categories

HAR translates raw environmental or physiological data into meaningful, context-aware insights about user actions and vitality. To capture these behaviors effectively, researchers have explored a variety of data acquisition methods. Consequently, HAR systems are generally classified into three main modalities: sensor-based, computer-vision-based, and Wi-Fi-signal-based [7], while some papers also include multimodal HAR [27, 28, 29].

Sensor-based HAR leverages dedicated hardware to measure physiological markers, such as pulse rates, body temperature, and electrocardiograms, enabling the classification of user states and physical activities [7]. Wearable sensors typically utilize miniaturized, wireless devices-such as adhesive patches, smart rings, and electronic textiles-to continuously monitor biological and kinematic signals [30]. Nevertheless, this modality faces significant limitations, primarily concerning the physical burden of continuous wear, signal degradation due to incorrect sensor placement, and the inability of certain user demographics to manage the equipment independently [7]. Moreover, the widespread deployment of these wearable frameworks is severely hindered by their high procurement costs and the inherent inconvenience they impose on the user [28].

Employing computer vision for HAR presents a highly intuitive approach, where visual data captured by cameras is translated into recognised actions through computational analysis [7]. Despite advancements in computer vision, identifying human actions continues to be a central, yet highly complex, focus within the field of computer vision. The integration of RGB, depth, and skeletal data has facilitated the deployment of activity recognition frameworks across diverse fields, including medical monitoring, military defense, and interactive computing [28]. However, vision-based approaches can not identify activities behind objects like walls [8], in conditions of darkness and presence of smoke [31]

and introduce privacy concerns [32, 31], especially for smart homes and elderly monitoring [7].

To overcome the privacy intrusions of vision-based systems and the physical burdens of wearable sensors, device-free HAR utilizing Radio Frequency (RF) signals-particularly Wi-Fi-has emerged as a highly promising modality [33, 8]. By analyzing the reflections and fluctuations of ubiquitous Wi-Fi signals propagating through an environment, these systems can infer human actions without requiring the user to wear or interact with any equipment [8]. While early approaches relied on coarse-grained Received Signal Strength Indicators (RSSI), which are heavily susceptible to environmental noise and multipath distortions, recent advancements focus on fine-grained CSI. CSI provides a robust, multi-dimensional representation of the wireless channel-capturing amplitude, phase shifts, and signal-to-noise ratios [7]. This makes CSI exceptionally well-suited for automated feature extraction, even when deployed on lightweight edge devices [15]. Consequently, Wi-Fi CSI has demonstrated significant efficacy in continuous, privacy-preserving applications such as non-intrusive elderly monitoring and vital sign detection [15, 33], though ensuring robust generalization against environmental variability remains an active area of research [15].

The primary objective of multimodal HAR is to develop architectures capable of fusing and interpreting data streams from heterogeneous sources [34]. Drawing inspiration from human perception, multimodal human activity recognition mitigates the inherent limitations of individual sensors by integrating diverse data streams, such as vision and wireless signals. This integration is typically achieved through two primary mechanisms: co-learning and fusion. Co-learning enhances overall system robustness by transferring knowledge from resource-rich modalities to supplement resource-poor ones. In contrast, fusion methods actively combine information across modalities during training and inference. Depending on the network architecture, this occurs at the data level by merging raw inputs, the feature level by integrating extracted representations, or the decision level by aggregating independent classification outcomes. A major drawback of this architecture is its inability to support end-to-end training. Instead, it requires optimizing three distinct models, which significantly drives up both computational overhead and memory demands[27].

2.2 The Evolution of Device-Free Wi-Fi Sensing

The conceptual foundation of using wireless communication networks for passive environmental sensing can be traced back to early experiments with RSSI of a Wi-Fi network. In the context of Wi-Fi, RSSI quantifies the total combined signal power received from all propagation paths, capturing both the direct line-of-sight connection and the various multipath reflections caused by structural elements and people within the space [35]. Foundational research around 2007 demonstrated that the mere presence of a human body could predictably attenuate the RSSI of a standard Wi-Fi network [36]. Early systems operated on the fundamental physical premise that the human body acts as an effective RF absorber. Specifically, biological tissue contains roughly 70% water, and the resonance frequency of water aligns with the 2.4 GHz band used by typical wireless data networks. As electromagnetic waves interact with the human form, they cause localized signal shad-

owing and multipath alterations that can be analyzed to detect multi-occupation of an environment using patterns of motion[37].

While HAR started by using RSSI, it suffers from severe multipath fading, temporal instability [9], and a complete lack of phase information. CSI on the other hand outperforms RSSI primarily because it provides fine-grained, subcarrier-level measurements rather than a coarse, packet-based average. While RSSI merely outputs the average value of the received signals, CSI measures both amplitude and phase information across individual Orthogonal Frequency-Division Multiplexing (OFDM) subcarriers. This frequency diversity allows CSI to accurately capture the varying multipath reflections induced by moving objects [10], leading to much higher precision, such as sub-meter localization accuracy [9] and the ability to detect fine movements ranging from walking [11] to breath monitoring [38].

A pivotal step in this transition from coarse RSSI to fine-grained sensing was the work that introduced the CSI-based human Activity Recognition and Monitoring system (CARM)[11, 39]. Unlike earlier CSI-based approaches that treated the relationship between signal dynamics and human motion as a black box solved through purely data-driven machine learning, CARM proposed two complementary analytical models that explicitly quantify the underlying physics of the sensing process. The first, a CSI-speed model, relates the frequencies of CSI power variations to the speeds of human body movements through a multipath formulation of the Channel Frequency Response. The second, a CSI-activity model, links the characteristic speed profiles of different body parts to specific activities using a Hidden Markov Model. Built around these models, the system combined Principal Component Analysis for denoising high amplitude impulse noise, Discrete Wavelet Transform for feature extraction of both movement duration and speeds of the body, and multi-link data fusion for robustness under different environments. Evaluated on a database of 1,400 samples spanning eight activities collected from 25 volunteers using commercial Intel 5300 hardware, CARM achieved an average recognition accuracy of 96% and retained over 80% accuracy in previously unseen environments. By providing a principled, model-driven framework rather than a purely empirical pipeline, this work became a foundational reference in the field and shaped much of the subsequent literature on CSI-based HAR.

Building on this model-driven foundation, the field’s progression toward data-driven and deep learning approaches highlighted a different bottleneck: the absence of standardized, publicly available datasets to enable reproducibility and fair comparison across methods. The WiAR paper [35] addressed this gap by releasing a public dataset, one of the first dedicated to WiFi-based human activity recognition. WiAR comprises sixteen activities, partitioned into upper-body (e.g., hand waves, drinking, phone use), lower-body (forward and side kicks), and whole-body categories (walking, squatting, sitting down, bending), performed by ten volunteers of varying sex, height, weight, and exercise background, with thirty samples collected per activity per volunteer across three indoor environments (an empty room, a meeting room, and an office). The acquisition setup follows the configuration of an Intel 5300 NIC operating at 5 GHz with 20 MHz bandwidth, exposing thirty subcarriers across a 1×3 antenna link, and provides both raw RSSI and CSI streams alongside extracted statistical features. The authors benchmark the dataset using five classical classifiers (K -Nearest Neighbors, Naive Bayes, Random Forest, Decision Tree, and

Support Vector Machine) and two deep architectures (CNN, LSTM), reporting average accuracies above 80% for the former and above 90% for the latter. Beyond the numerical results, WiAR reflects a broader shift within the community toward establishing common benchmarks that allow different HAR approaches to be evaluated and compared on equal footing, while simultaneously providing an openly accessible dataset for researchers.

Historically, accessing this fine-grained CSI data required specialized PC-based hardware running modified firmware, most notably the Intel 5300 Network Interface Controller (NIC) [12] or specific Atheros chipsets, as demonstrated by work such as [40]. While these setups pioneered the field of Wi-Fi sensing, they were inherently bulky, power-hungry, and impractical for ubiquitous, unobtrusive deployment in environments such as elderly care homes. The recent evolution of device-free sensing has shifted toward lightweight, low-power Internet of Things (IoT) microcontrollers. Modern embedded platforms, such as the ESP32 series, now expose physical-layer CSI extraction capabilities as standalone device [13]. Variants of the ESP32 have emerged as low-cost, easy-to-deploy CSI-capture platforms, lowering the hardware barrier to WiFi sensing relative to discontinued NICs such as the Intel 5300 [17] while introducing strict new limitations on available memory and computational overhead [41].

Building on this hardware shift, a growing body of work has begun to explicitly characterize and benchmark CSI HAR on ESP32-class platforms. The work [42] released one of the first publicly available ESP32-based CSI datasets, gathering 10 daily activities from 8 volunteers across three indoor floor plans, two line-of-sight (LOS) and one non-line-of-sight (NLOS). The acquisition relies on an ESP32 microcontroller as the receiver and uses the open-source ESP-CSI tool to log one RSSI value and 52 CSI subcarriers per packet into structured CSV files. With 20 trials per activity and 1 600 trials per setup, this dataset showed that the ESP32 can capture activity-discriminative CSI patterns under both LOS and NLOS conditions, while highlighting that its limited subcarrier count and the absence of Multiple-Input Multiple-Output antennas impose constraints not present in Intel 5300-based work. Extending this line of work, the paper [14] introduced the ESP-Fi HAR dataset, collected with the more recent ESP32-C3 in four real-world indoor environments (corridor, office, meeting room, and laboratory) for seven activities (walking, running, falling, jumping, squatting, arm waving, and turning) performed by 8 participants of varied height and weight. The authors quantify the hardware advantage explicitly, reporting that ESP32-C3 nodes make nodes roughly $12\times$ cheaper and reduce active power consumption by more than 95% compared to NIC setups like Intel-5300. As a result, this makes continuous, mains- or battery-powered CSI sensing realistic for ad-hoc IoT deployments. Their benchmarks of classical machine learning baselines (SVM, Random Forest, Logistic Regression) and deep models (LSTM, GRU, CNN, ResNet18, EfficientNet-Lite, Transformer, MobileNetV3) under a leave-one-subject-out protocol show that, even on ESP32-class hardware, ResNet18 reaches 58–70% accuracy across different environments while traditional ML models remain near 20–25%, confirming both the feasibility and the difficulty of HAR generalization on esp-32 devices.

Complementing these dataset contributions, a growing body of work has begun to deploy actual HAR pipelines on ESP32 hardware across a widening range of applications. Strohmayer and Kampel [17] explored long-range through-wall human presence and activity detection using directional antennas on the ESP32-S3, reporting classifica-

tion accuracies of 97% in LOS and 88% in NLOS. Another work [43] targeted industrial safety, proposing a spatio-temporal feature distribution network for device-free recognition of power inspection activities, with reported accuracies around 88% under controlled substation conditions. Moghaddam [8] focused on domestic activities of daily living in kitchen environments, recognizing cooking-related coarse-grained tasks such as storing food and washing pots with 80% accuracy and fine-grained activities such as chopping and slicing with average accuracy of 37%. Another work [15] integrated ESP32 acquisition with edge inference on a Raspberry Pi 4, classifying everyday activities and fall events through an LSTM pipeline with a reported 93% 5-fold cross-validation accuracy. Liu [16] placed particular emphasis on energy efficiency, coupling an ESP32-S3 acquisition module with a lightweight GRU network and INT8-quantised inference for low-power embedded deployment, while still recognising seven activity classes at 93.5% accuracy. Collectively, these works span smart-home monitoring, elderly care, industrial inspection, and through-wall sensing, and they consistently report high in-domain accuracies, suggesting that ESP32-class hardware has matured to the point where credible HAR systems can be built end-to-end on commodity, low-power devices.

2.3 Wi-Fi Sensing for Elderly Care

Of the application domains that motivate device-free sensing, unobtrusive in-home monitoring of older adults is the one that recurs most consistently across the CSI literature, and it is the domain this thesis targets. The demographic pressure behind it is substantial: the work [1] reports that the global population aged 65 and over stood at 703 million in 2019 and is projected to reach 1.5 billion by 2050. Also, roughly 90% of that group would prefer to remain in their own home rather than move to a nursing home[2], and that a large fraction already lives alone. In detail, more than a third of older people in Europe and close to 29% of the 46 million community-dwelling older adults in the United States would prefer that. An older person living alone who has an accident at home may have no means of alerting anyone, which is precisely the situation an always-on, non-intrusive sensor is meant to address [3].

The same case is made from the technology side, and it is made against both of the established alternatives. Li et al. [6] note that a camera cannot see through walls, requires good lighting conditions, and is unsuitable in a bathroom. A bathroom is a room in which activity monitoring is nonetheless particularly valuable, since it is a potential site of a fall. They judge wearable systems equally inappropriate for this population, on the grounds that older adults may simply forget to carry or wear the sensor. Chung et al. [4] sharpen the objection to wearables. Such devices are not easy to use for people with mild cognitive impairment, and they typically depend on a smartphone application for data synchronisation, yet a significant proportion of older adults facing health disparities do not own a smartphone, lack proficiency with smartphone applications when they do, or have limited IT literacy. Wi-Fi sensing sidesteps both objections at once. It is device-free, so the location and activity of a resident can be inferred without equipping them with anything. And because the signal penetrates walls and furniture, it continues to work in poor lighting or where a camera’s view would be obstructed [4].

Existing work divides into three lines. The first pursues coarse, long-horizon be-

havioural monitoring without naming activities. WiMonitor [3] instruments a smart home of four subspaces (i.e. living room, bedroom, kitchen and toilet) with one transmitter and five Intel 5300 receivers. In particular, it uses at least one receiver per area, and reports room-level area detection with precision and recall both above 96% over a continuous 22-day deployment in which four volunteers each lived alone in the apartment in rotation. Alongside the occupied area, the system outputs a binary still or non-still motion state and a continuous activity-intensity metric. Aggregated across days, these three signals reveal a resident's daily routine and expose abnormalities within it. The authors observe that a volunteer's accumulated body-movement time during sleep was much longer on two of his fourteen nights than on the other twelve, and, on consulting the ground-truth video, find that he had suffered from insomnia on those nights. However, what the system does not do is label activity types. The authors argue that for elderly daily-life monitoring the essential information is the area the resident occupies, whether they are moving and how vigorously.

The second line performs activity identification but binds it to location. E-eyes [44] separates in-place from walking activities by thresholding the variance of CSI amplitude, then matches each segment against a library of per-location activity profiles. It was evaluated in two apartments with one access point and three connected Wi-Fi devices, and over a four-month period in which profiles were built on one day and tested on subsequent ones. It reports average in-place accuracies of 97% and 97.38% and average walking accuracies of 97% and 94%. These results rest on the observation that most activities meaningful for home monitoring occur at a few fixed locations, so that a small number of profiles per location suffices. The cost is that the system assumes a single occupant and a broadly stable environment, and that whether a profile built at one location can recognise the same activity elsewhere is left open (the only remedy offered is to add further profiles).

The third line targets elderly activity classification directly. Li et al. [6] extract both amplitude and phase from a single commodity link with an Intel 5300 receiver, apply PCA across the CSI streams to reduce dimensionality and expose the motion-induced trend. Then, they classify seven activities (i.e. walk, sit down, lie, stand up, squat down, fall and crawl) with a Random Forest, reporting 95.43% average accuracy in a line-of-sight configuration and 91.2% when the access point is placed in a different room from both the target and the receiver. However, some caveats bound the result. Within each group the users had to perform the activities at a fixed place, because CSI values differ from one location to another. Also, the reported accuracies come from five-fold cross-validation over a pool of samples contributed jointly by all five users (without the fold partitioning being specified), so it cannot be established from the paper whether the evaluation is subject-independent.

Only one of these studies has been carried into the homes of the target population. Chung et al. [4] deploy a low-cost, ESP32-based CSI system for a week in the apartments of older adults in low-income senior housing. In their preliminary controlled experiment, three TX-RX pairs positioned so that every activity fell close to the line of sight of at least one pair achieved 93.8% accuracy on kitchen activities, 100% on dining-room and bathroom activities, but only 80.8% in the living room. In the last case, the error is almost entirely confusion between writing at a desk and sitting on a sofa, which are two activities with limited body movement occurring close together. Their proposed remedy is to place

additional TX-RX pairs nearer to the affected locations. In the subsequent pilot with two residents, an initial model recognised six activities with 68.8% accuracy.

Reading those studies together, they expose three limitations that shape the present work. First, evaluation remains largely laboratory-bound. Chung et al. [4] report that Wi-Fi sensing for healthcare has predominantly been assessed in controlled environments with prompted activities over brief timeframes, and that they are aware of no prior study assessing everyday functioning in a natural setting with older adults over time. Second, with the exception of Chung et al. [4], every system reviewed above relies on the Intel 5300 NIC, which requires a host laptop or desktop to capture CSI. Hernandez and Bulut [23] identify this cost and bulk as the principal obstacle to scalable deployment, and propose the ESP32 as the low-cost alternative.

Third, and most directly relevant here, the multi-link deployments in this literature distribute their links across rooms so that every area is covered, rather than concentrating them on a single space. Chung et al. [4] assign one pair per zone and position each so that its activities fall close to its line of sight. WiMonitor [3] spreads five receivers over four areas of a home. E-eyes [44] places three devices around an apartment, fusing them but weighting each by how strongly it senses the activity, which favours whichever device is nearest. Redundancy is therefore obtained by adding coverage, and correct recognition still depends on some link having a clear view of the activity. Hernandez and Bulut [23] note that most Wi-Fi sensing work still assumes a single TX-RX pair and list the use of multiple, diversely positioned links as an open challenge.

2.4 HAR challenges

Transitioning Wi-Fi-based HAR from controlled environments to scalable, real-world deployments exposes critical difficulties across the data, algorithmic, and hardware pipelines.

2.4.1 Data

The primary challenge for HAR using Wi-Fi is a severe data-scarcity bottleneck that prevents the field from replicating the large-scale success seen in other domains like computer vision and natural language processing. Since collecting and annotating real-world Wi-Fi CSI is highly labor-intensive, current Wi-Fi datasets still lack the massive scale, diverse action categories, and demographic variance required to train robust, generalizable foundation models. While researchers are attempting to overcome this limitation by generating synthetic data through 3D modeling and simulated signal propagation (such as ray-tracing), this introduces a complex simulation to reality domain gap. The core current obstacle is accurately simulating how radio signals physically propagate through human bodies and dynamic environments, ensuring that models trained on synthesized CSI data can seamlessly generalize and perform reliably when deployed in real-world physical settings [19].

Similarly, the study [7] highlights that the major limitation of current Wi-Fi sensing research is the reliance on datasets gathered in tightly controlled environments. These datasets artificially remove the natural interference found in actual deployment settings (such as physical barriers, furniture, ambient noise, and multi-person movement), and

therefore fail to represent true Wi-Fi signal dynamics. To develop algorithms that are robust and reliable enough for practical use, data collection must embrace the complexities of real-world environments. Incorporating realistic floor settings, varying hardware, and everyday signal interferences is essential for training models that can successfully generalize outside the lab.

2.4.2 Model Generalization

Model generalization is a challenge across diverse real-world applications, including image classification, speech recognition, and wifi sensing. In detail, model generalization is the capacity of a machine learning or deep learning framework to maintain high performance when exposed to out-of-sample data. This trait is essential for practical deployment, ensuring the system does not simply overfit to its training set, but can reliably process new inputs [7]. Wi-Fi sensing is significantly influenced by environmental factors, human body factors, and device heterogeneity. Environmental variables, such as room layouts, wall materials, furniture placement, and device positions, dictate the signal propagation paths and multipath profiles. Furthermore, human physiological factors (i.e., height, weight, body composition, and even clothing materials) can modify the transmission and attenuation of Wi-Fi signals. Finally, device heterogeneity, particularly protocol versions, operating channels, bandwidth, antenna design, physical orientation, and chipset configurations, generate considerable signal fluctuation[19]. Consequently, these compounded signal fluctuations manifest as different categories of generalization challenges in HAR: cross environment, cross location, cross orientation, cross user [45], cross day, and cross device [46].

2.4.3 Data Leakage

Beyond architectural and hardware considerations, the reliability of CSI-based HAR benchmarks is also conditioned by the methodological soundness of their evaluation protocols, and in particular by how training and test sets are partitioned. The paper [22] highlighted a critical data leakage issue affecting a widely used WiFi CSI benchmark [47], in which CSI samples were randomly split into training (75%) and test (25%) sets without separating individuals across the two partitions. Because the same volunteers appeared in both sets, the models inadvertently learned person-specific signatures (such as body geometry and gait idiosyncrasies that modulate the multipath profile) rather than generalizable, activity-specific patterns, yielding overly optimistic accuracy estimates. To quantify this effect, the work retrained the four original architectures (2D-CNN, 1D-CNN, LSTM, and attention-based BLSTM) under a subject-independent split, where the data of two volunteers were used for training and the third was held out for testing. Under this corrected protocol, accuracy dropped substantially across all models: from 92.2% to 66.4% for the 2D-CNN, from 86.9% to 55.0% for the 1D-CNN, from 89.3% to 61.8% for the LSTM, and from 93.6% to 62.2% for the BLSTM, retaining only 63-72% of the originally reported performance. Confusion matrices further revealed that some activities, such as falling, remained well classified, whereas more subtle actions like *lie down* suffered the largest degradation. These findings underscore that subject-independent partitioning is a prerequisite for any meaningful assessment of cross-user generalization in Wi-Fi HAR [22].

2.4.4 Edge deployment

Transitioning Wi-Fi sensing from theoretical models to real-world edge deployment relies on a hierarchical cloud-to-edge pipeline, which introduces several critical engineering hurdles [19]. Because edge hardware inherently suffers from restrictive processing capabilities, limited storage, and finite power reserves, models must undergo rigorous compression to function effectively [7]. In detail, pre-trained cloud models must be heavily compressed using different techniques (like quantization, structure-aware pruning, and knowledge distillation) to achieve efficient inference on resource-constrained commercial chips (such as the ESP32) without losing critical sensitivity to human motion. Finally, practical edge deployment must carefully balance the high bandwidth consumption of continuous CSI streaming to avoid disrupting standard Wi-Fi communications. This should happen while simultaneously ensuring the system remains resilient against physical-layer interference and adversarial attacks [19]. Consequently, model selection requires carefully balancing computational overhead, power efficiency, and environmental robustness to guarantee reliable performance on constrained hardware [7].

2.4.5 Detection Efficiency and Real-Time Segmentation

Deploying continuous Wi-Fi sensing on resource-constrained edge devices introduces the tightly coupled challenges of real-time temporal segmentation and detection efficiency. In practical, always-on deployments (such as continuous HAR monitoring for the elderly) the vast majority of streaming CSI captures an empty room or inactive periods, like in the middle of the night [23]. Running continuous machine learning inference on this raw, unfiltered stream creates severe computational bottlenecks and unnecessary power drain on lightweight hardware [7]. Therefore, systems must employ rapid, low-overhead segmentation to distinguish active human movements from ambient noise, triggering the complex HAR prediction model only when a relevant action actually occurs [23]. If this temporal segmentation lacks efficiency, the resulting data processing delays compromise the system’s real-time responsiveness to critical health events. Simultaneously, it exhausts the computational load and battery life of the IoT device. Consequently, having detection-efficient models is a foundational requirement for viable real-world operation [7].

2.4.6 Long-Term Model Adaptation

Beyond strict hardware limits, edge devices face the constant challenge of environmental variability. Since even slight furniture changes alter signal multipath profiles, systems require lightweight, on-device fine-tuning to adapt locally without relying on massive data transfer or impractical manual data labeling from users [19]. Similarly, the paper [23] mentions that while current WiFi sensing models perform well in the short term, collecting CSI data over extended periods (such as months or years) exposes systems to signal variations that significantly degrade accuracy. Two primary adaptation strategies are presented. The first is detrending streaming data signals, though its application remains scarce and is largely limited to specialized tasks like sleep monitoring, walking speed tracking, and respiratory analysis. The second strategy bypasses detrending in favor of continuous on-device learning through online stream sampling [48]. Given the limited scope of these

existing solutions, long-term model adaptation to both temporal CSI drift and physical alterations in the surrounding multipath environment is still an open challenge[23].

2.4.7 Multiple TX/RX Links

Typical Wi-Fi sensing experiments assume a simplified single transmitter (TX) and single receiver (RX) architecture. However, real-world environments (such as smart homes or assisted living facilities) are inherently rich with interconnected devices, including routers, smart TVs, and IoT devices. Relying on a single TX-RX pair often restricts sensing coverage, as a single link naturally suffers from physical blind spots or performs poorly for specific activities due to occlusion. Consequently, leveraging multiple TX-RX links dispersed throughout an environment is a major frontier for improving HAR performance and spatial coverage [23].

Recent literature demonstrates both the necessity and potential of multi-link systems. Recognizing the limitations of single-link setups, researchers are increasingly experimenting with high-dimensional, multi-transceiver CSI datasets to capture diverse spatial perspectives and enhance foundational HAR accuracy [24]. Extending this multi-transceiver paradigm, the work [25] utilized the previously mentioned dataset to fuse front- and side-view CSI streams in a lightweight 2D CNN, reaching around 84.5% accuracy, substantially outperforming pre-trained backbones such as DenseNet121 and EfficientNetV2M on the same task. Single TX-double RX are also being utilized to track complex, multidimensional movements, such as fine-grained speed estimation during bodyweight exercises [49] and on-desk gestures tracking [50]. Crucially, spatial diversity dictates that different pair TX-RX links naturally excel at classifying different activities based on their positioning relationship to the user. For example, one pair link might fail to detect a user washing dishes, while a second link might capture the micro-movements perfectly [23]. Real-world daily activity tracking deployments in low-income senior housing environments further emphasize this need, demonstrating that single-link systems struggle to maintain reliable coverage across multiple rooms and daily routines [51]. If a system can dynamically identify and select the most qualified link at any given moment, overall prediction accuracy improves significantly compared to relying on a static, single-pair connection [23]. In addition, a comparable benefit is visible in the related task of device-free localization, where three ESP32 links were deployed around a single room and observed median error fell monotonically as links were added (from roughly 1.0 m with one link, to 0.65 m with two, and 0.45 m with three). The authors attribute this to multi-link fusion increasing fingerprint distinctiveness and reducing spatial ambiguity [52].

Despite this potential, multi-link sensing introduces complex algorithmic and system-level challenges. Blindly aggregating data from all available antennas does not guarantee better performance. Multi-link systems often suffer from the dilution effect. In these scenarios, feeding poor-quality, noisy, or obstructed links into the model actively degrades the predictive power of the high-quality links, demonstrating that less is more when dealing with uninformative streams [26].

2.5 Self-Supervised Learning for HAR

When annotated datasets are limited, Self-Supervised Learning (SSL) serves as a powerful alternative to traditional supervised methods. Already proven effective in fields like computer vision and natural language processing, SSL excels at reducing the need for manual labeling while offering advantages in generalization. As a branch of unsupervised learning, it generates its own training signals from raw data by solving engineered pretext tasks. These tasks encourage the model to extract meaningful representations. The resulting representations can then be fine-tuned for specific downstream applications. Furthermore, the widespread availability of WiFi infrastructure provides an ideal, non-intrusive medium for gathering the large volumes of unlabeled data required to drive these models [21].

Building upon this passive data collection, Wi-Fi CSI has proven particularly valuable for HAR. By analyzing the subtle disruptions that human movements cause to Wi-Fi signals, systems can distinguish between states such as an empty room, stationary postures like sitting, or dynamic activities like walking, all without relying on intrusive cameras. However, scaling these systems for complex, fine-grained recognition is heavily bottlenecked by the need for manually annotated CSI datasets. In this context, SSL utilizes both contrastive and non-contrastive learning architectures to autonomously extract robust feature representations from raw signal variations. This highly efficient feature extraction is critical for condensing complex spatial-temporal patterns into lightweight representations. While real-time edge training is still a challenge, on-device inference and on-cloud training are possible [29].

Evaluating the generalizability and robustness of various SSL algorithms for WiFi-based HAR, the Masked Autoencoder (MAE) is consistently highlighted as the superior method. Through linear and non-linear classifier evaluations, MAE demonstrated an exceptional capacity to learn high-quality features, significantly outperforming contrastive learning methods on complex datasets featuring diverse environmental conditions and numerous activity classes. Rigorous domain shift experiments (testing cross-user, cross-room, and cross-receiver scenarios) revealed that while receiver displacement causes the most substantial performance degradation, unsupervised pre-training can successfully mitigate human labeling errors and often surpass supervised baselines. Finally, when confronted with severe labeled data scarcity, MAE maintained its dominant efficiency in both source-to-source and source-to-target settings. This proves MAE’s robust viability for real-world smart home deployments despite a noted sensitivity to the size of pre-training datasets in certain transfer configurations [21].

2.6 Transfer Learning for HAR

Similar to SSL, Transfer Learning (TL) addresses the data-scarcity bottleneck that constrains Wi-Fi CSI-based HAR [20]. While SSL generates training signals from unlabeled in-domain data [21], TL instead transfers knowledge from a source domain (typically a model pre-trained on a large, well-curated dataset) to a target domain with limited labeled samples [20, 53]. This paradigm is particularly attractive for Wi-Fi sensing because collecting and annotating CSI data across environments, users, and devices is labor-intensive [8], and training deep networks from scratch on small CSI datasets often leads to overfit-

ting and poor generalization [20]. By initializing models with weights learned from rich source domains, TL reduces both the volume of required training data and the number of training epochs needed to reach competitive accuracy [53].

A central technique in this line of work is to recast the Wi-Fi sensing problem as an image classification task, enabling the reuse of CNNs pre-trained on large-scale natural image datasets such as ImageNet [18, 53]. In work [18], CSI amplitude streams from three antenna pairs are stacked into three-channel image matrices and classified with VGG16 and VGG19 backbones pre-trained on ImageNet. Two transfer strategies are compared. First, is using the pre-trained CNN as a fixed feature extractor coupled with an SVM classifier (CNN-SVM), and second, fine-tuning the final convolutional and fully connected layers on CSI images (FT-CNN). Across twelve gestures collected in two indoor environments, fine-tuning consistently outperformed the frozen-feature variant, with FT-VGG19 reaching 98% accuracy compared to 97% for VGG19-SVM, indicating that adapting intermediate representations to the CSI domain is beneficial despite the strong general-purpose features inherited from ImageNet. A similar transformation strategy is adopted in [53], where CSI magnitude and phase matrices are converted into images via packet-versus-subcarrier mappings and fed into an Inception V3 backbone fine-tuned for multi-person activity recognition (i-CNN). Against a fifteen-layer CNN trained from scratch (d-CNN), the transfer-learned i-CNN reached 96.7% accuracy using a single transmitter-receiver link and 99.1% when aggregating CSI across multiple links. Also, the transfer-learned i-CNN requires substantially fewer training epochs, exceeding 90% test accuracy within only 50 epochs compared to roughly 200 epochs for the scratch-trained d-CNN baseline.

Beyond raw CSI-to-image conversion, time-frequency representations have been combined with deeper residual backbones to address the cross-scene generalization problem. The Wi-AR system in [20] applies a Butterworth low-pass filter to denoise CSI, segments activity windows using a threshold-based sliding window over the variance of principal components, and converts each segment into a Short-Time Fourier Transform (STFT) spectrogram. The spectrograms are classified by a ResNet18 pre-trained on ImageNet and fine-tuned with a low learning rate to avoid forgetting the source-domain weights. With only 480 training samples spread across three room layouts, the system achieved an average accuracy of 94.2% on four activities, outperforming AlexNet, VGG variants, and deeper ResNet models while requiring less training time. The same study also reported generalization to an external dataset collected with a different sampling rate and antenna configuration, suggesting that the combination of time-frequency features and transfer learning produces representations that transfer across environmental and hardware variation [20].

Despite these encouraging results, transfer learning for Wi-Fi HAR remains constrained by the mismatch between the source and target domains. Pre-trained ImageNet features only partially transfer to CSI pseudo-images. This is why fine-tuning consistently outperforms frozen backbones [18, 53]. Cross-environment evaluation also remains a weakness: models trained solely in one environment and tested in another suffer substantial accuracy degradation because the captured CSI encodes not only the activity but also the surrounding multipath structure (such as furniture and walls). However, transfer-learning methods such as [20] partially mitigate this. Nonetheless, generalizing to a new environment typically requires collecting new CSI data from that environment and retraining

the model [18]. Furthermore, the transfer-learning HAR pipelines discussed above rely on heavy backbones such as VGG, Inception V3, and ResNet, which are impractical for direct deployment on resource-constrained microcontrollers given the strict memory and compute limits of edge hardware [7, 19].

2.7 Summary and Problem Identification

The literature reviewed in this chapter shows that Wi-Fi CSI-based HAR has matured along several distinct axes. Foundational work established the physical and algorithmic basis for fine-grained sensing on PC-class hardware such as the Intel 5300 NIC. More recent contributions have demonstrated that ESP32-class microcontrollers can capture activity-discriminative CSI at a fraction of the cost and power budget [42, 14]. In parallel, deep learning, self-supervised pre-training, and transfer learning have each been proposed to address the data-scarcity and generalization bottlenecks that constrain the field [21, 20]. Nevertheless, several limitations recur across this body of work.

First, most of ESP32 HAR pipelines and datasets are evaluated single-environment conditions, where the training and test data share the same temporal and spatial context [15, 43]. The few studies that contain cross-environment samples, either do not evaluate cross-environment generalization [42] or do so hiding the exact details [16] or using non-realistic settings like a 10-second window length [14]. Second, fine-grained activity recognition on ESP32 hardware remains largely unsolved, with reported accuracies for subtle actions such as chopping and slicing falling well below those of coarse-grained tasks [8]. Third, while multi-link configurations have been shown to improve coverage and robustness by mitigating blind spots, occlusions, and link-specific failure modes, almost all ESP32 HAR work relies on a single TX–RX (single-pair) setup, and the few multi-link studies only use a direct LOS setup [24, 25].

To the best of our knowledge, the combined use of two co-deployed ESP32-S3 TX–RX pairs in a single room to jointly sense human activity (and in particular its behaviour under cross-day, cross-environment and NLOS conditions) remains largely unexplored. This is even though the low unit cost and power footprint of the ESP32 make such deployments far more realistic than with Intel 5300 hardware. This thesis addresses this gap through a progression of experiments on the ESP32-S3 sensors. We first (i) replicate the established cross-environment ESP-Fi HAR benchmark [14] to validate the current state of cross-room HAR generalization. Next, we (ii) investigate a double-pair TX–RX setup under cross-day, NLOS conditions, examining whether the additional spatial diversity improves robustness and to what extent the static postures (*sit*, *stand*, *lie*) and *empty room* remain difficult to separate. We further (iii) extend the double-pair setup to fine-grained kitchen activity detection under same-day, same-room and LOS conditions, and deploy the resulting recogniser on continuous, naturally executed routines (rather than pre-segmented clips, which are the usual evaluation regime of the mentioned benchmarks). Finally, we (iv) evaluate double-pair cross-environment generalization across three rooms under a single fixed placement configuration. Together, these experiments characterise when and how a low-cost double-pair ESP32 deployment improves over the single-pair baseline that dominates the existing literature.

Background

This chapter introduces the concepts and notations needed to follow the rest of this thesis. We begin with the clinical context of elderly monitoring, defining the different types of activities that a system is expected to recognise, and explaining why their long-term evolution is what matters for detecting functional decline. We then present the theoretical foundations of Wi-Fi CSI. In detail, we present how it differs from RSSI, how its amplitude and phase are defined, how the movement of a person is written into the channel through the dynamic multipath component, and why a single TX-RX link is sensitive only in a limited region of a room and only to certain directions of motion. We continue with the hardware, describing the ESP32-S3 board used for our data collection, its processing and memory resources, and the Wi-Fi subsystem that gives access to CSI. Finally, we review the denoising filters commonly applied to raw CSI amplitude, since some of them are the building blocks of the preprocessing pipeline described in the next chapter.

3.1 Clinical Context: Daily Activities, Mobility and Functional Decline

The quantity of clinical interest in an elderly-care monitoring system is not an activity in isolation but the resident's functional status. More precisely, it is their capacity to carry out the activities that independent living requires. Chung et al. [4] organise the observable behaviours into three categories. Activities of daily living (ADLs) cover basic self-maintenance, such as eating at the dining table. Instrumental activities of daily living (IADLs) cover the more complex tasks that sustain independence, for instance meal preparation and the use of electronic devices such as microwaves, computers and telephones. Mobility behaviours are life-space movements, like entering and exiting the apartment, entering and exiting a private area such as a bedroom or bathroom, and sitting or lying on a sofa. Each is defined operationally by a start and an end time and the metrics extracted from a deployed system are the daily count and duration of each behaviour.

These categories matter because they degrade before a diagnosis does. It is reported that roughly 15% of individuals with mild cognitive impairment (MCI) progress towards Alzheimer disease or a related dementia within two years. Older adults with MCI are also at greater risk of limitations in ADLs, IADLs and mobility behaviours, as cognitive impairment affects the ability to perform and complete daily activities. People with MCI tend to be less physically active and spend more time in sedentary routines. Crucially, functional decline is reported to begin years before the clinical symptoms of dementia appear, which makes the everyday activities above an early-warning channel rather than a retrospective record. In addition, delayed recognition of early cognitive impairment often leads to functional decline, higher risk of falls, premature institutionalization, diminished quality of life, and increased mortality [4].

It follows that the clinically useful signal is a temporal pattern rather than any single correct classification. Since physical activity patterns typically remain steady over weeks or months, early detection of a declining trend can serve as a critical warning sign to prompt timely intervention [3]. Similarly, E-eyes [44] applies this concept to aging-in-place contexts. Specifically, tracking whether daily routines are maintained is essential, as deviations or missing activities can directly signal a fall or an emergency. A monitoring system is therefore judged on whether it can run continuously and unattended for long enough that a drift in behaviour becomes distinguishable from its own error.

This requirement is what rules out the conventional alternatives. Self-report is subject to recall bias and memory impairment, which is precisely what smart home technologies were introduced to bypass. Motion and presence sensors supply objective, continuous, low-intrusion data but cannot label specific activity types [4]. For instance, a drop in kitchen presence cannot by itself reveal whether a resident has stopped preparing meals or is simply spending less time there. Wearables deliver objective, high-frequency, remote monitoring but are difficult to use for people with MCI, typically require a smartphone application for data synchronisation, and are frequently forgotten or resisted [4, 3]. Cameras are constrained by line of sight and by lighting, raise severe privacy concerns [3], and are least acceptable in the bathroom, where falls are possible [6]. Device-free Wi-Fi sensing is attractive here not because it outperforms these modalities in the laboratory but because it is the only one of them that is simultaneously label-capable, wearable-free, more privacy-preserving and able to sense through walls and furniture [4].

Three design requirements follow, and they set the agenda for the remainder of this thesis. The system must label specific activity types rather than only presence and intensity. Next, it must operate in ordinary furnished homes over different days and rooms rather than in a cleared laboratory over minutes. And last, it must remain reliable when the resident's position and orientation relative to the hardware are uncontrolled.

3.2 Theoretical Foundations of Wi-Fi CSI

Device-free Wi-Fi sensing rests on a simple physical premise that the electromagnetic field that carries a communication signal from TX to a RX is reshaped by everything it encounters along the way. A person moving through the propagation environment perturbs this field, and those perturbations are recorded by the RX as part of its ordinary channel-estimation procedure [31]. The remainder of this section develops the metric that captures these perturbations (CSI). Starting from the metric it replaced, the discussion examines CSI's amplitude and phase components, models how motion is encoded into it, and finally explains the spatial and resolution limits that motivate the multi-link architecture investigated in this thesis.

3.2.1 From RSSI to Channel State Information

For much of the history of commodity Wi-Fi, the only channel descriptor exposed to higher layers was the RSSI, a single scalar reporting the total power collected at the RX. Because it aggregates the contributions of all propagation paths and all frequencies into one number, RSSI is coarse. In other words, two links can share an almost identical RSSI

yet behave very differently, and the metric reveals little about how the channel varies with movement [12, 54].

The IEEE 802.11n standard introduced a finer-grained alternative. Originally defined so that a TX could improve a link through transmit beamforming, the standardised CSI feedback mechanism was repurposed into a measurement tool that records, for every correctly received packet, signal strength and phase information at the granularity of individual OFDM subcarriers and for each TX-RX antenna pair [12]. Where RSSI collapses the channel to one value, CSI exposes its frequency structure.

Formally, in an OFDM system the channel between a transmit and a receive antenna is decomposed into S subcarriers. Denoting by x_i and y_i the transmitted and received signal vectors on subcarrier $i \in \{1, \dots, S\}$, the relation of transmitted and received signals is the following:

$$y_i = H_i x_i + n_i, \quad (3.1)$$

where n_i is the additive noise vector and H_i is the CSI associated with the i th subcarrier [49]. The collection $\mathbf{H} = [H_1, H_2, \dots, H_S]$ is precisely this vector that CSI reports. Each CSI coefficient can be written in the form

$$H(f_k, t) = |H(f_k)| e^{j\angle H(f_k)}, \quad (3.2)$$

where f_k is the carrier frequency of the k -th subcarrier, t is the timestep and $|H(f_k)|$ and $\angle H(f_k)$ are respectively the magnitude and phase response of that subcarrier. A CSI measurement therefore yields a vector of complex numbers per antenna pair, rather than the single power value provided by RSSI [55].

The practical consequence is the ability to observe frequency-selective fading. Because multipath components combine constructively on some subcarriers and destructively on others, the amplitude profile across subcarriers is uneven, and deeply faded subcarriers require disproportionately more power to deliver the same performance [12]. It is exactly this per-subcarrier, per-antenna structure (invisible to RSSI) that makes CSI a rich signal for sensing. The number of reported subcarriers depends on the platform. The foundational Intel 5300 tool exposes 30 subcarrier groups [12], while the ESP32-class hardware used in this work exposes 52 non-zero subcarriers [13, 49].

3.2.2 Amplitude and Phase as Complementary Signals

The two components of Equation (3.2) carry complementary and unequally reliable information about motion. Amplitude $|H(f_k)|$ is comparatively stable and is the component most commonly exploited in early CSI-based systems. Phase $\angle H(f_k)$ is more sensitive to small displacements of reflectors. Therefore, it is potentially more discriminative for fine-grained motion, but also far more corrupted by hardware-induced artefacts when extracted from commodity devices [11, 54].

The measured phase of subcarrier i can be modelled as

$$\hat{\phi}_i = \phi_i - 2\pi \frac{k_i}{N} \delta + \beta + Z, \quad (3.3)$$

where ϕ_i is the true phase, δ is the receiver timing offset that introduces a phase error through the term $2\pi(k_i/N)\delta$, β is an unknown constant phase offset, N is the Fast Fourier

Transform (FFT) size, k_i is the subcarrier index, and Z is measurement noise. Because of the unknowns δ and β , isolating the true phase shifts is practically impossible with commercial Wi-Fi devices [54].

A standard remedy is a linear transformation across the frequency band that removes the offset terms. By estimating the slope and offset of the phase across subcarriers and subtracting them, one recovers a sanitised, linearly-combined phase from which the random offsets have been eliminated, with the small measurement noise term Z neglected [54]. In practice, however, many CSI-based HAR systems built on commodity hardware (and on the ESP32 in particular) forgo phase entirely and operate on amplitude alone [17, 15, 16]. Because phase is more susceptible to hardware noise and distortion on commodity CSI, these systems typically adopt the amplitude representation instead [16].

3.2.3 Encoding Motion: Static and Dynamic Multipath

To understand how movement is written into the CSI, the received signal is separated into contributions that are stationary and contributions that change because of motion. The CSI is represented as

$$H(f, t) = H_s(f) + H_d(f, t) + n(t), \quad H_d(f, t) = a(f, t) e^{-j2\pi d(t)/\lambda}, \quad (3.4)$$

where $H_s(f)$ aggregates the signals arriving over static paths (like walls, and furniture), $H_d(f, t)$ is the dynamic component reflected from a moving body, $n(t)$ is environmental and hardware noise, $a(f, t)$ is the complex amplitude and initial phase offset of the dynamic path, $d(t)$ is the length of that reflected path, and λ is the carrier wavelength [49]. Only $H_d(f, t)$ is informative about movement.

In the idealised case without multipath clutter or noise, the motion state of the target is contained in the phase of the dynamic component, and its rate of change gives the signal frequency shift induced by motion,

$$f_d = \frac{1}{2\pi} \frac{\partial \angle H_d}{\partial t} = -\frac{d'(t)}{\lambda}, \quad (3.5)$$

where $d'(t)$ is the rate of change of the dynamic path length. Equation (3.5) is the bridge between the abstract channel coefficient and a physically meaningful quantity. The speed at which a reflecting surface (a limb, a torso) changes the length of its propagation path is mapped onto an observable frequency shift in the CSI. In realistic indoor settings, extracting H_d is non-trivial because the measured CSI mixes many multipath components and noise, and the relationship between the frequency shift and the true motion speed is further complicated by the geometry of the deployment [49].

3.2.4 Spatial Selectivity and the Fresnel-Zone Blind-Spot Problem

A single TX-RX link is not uniformly sensitive throughout a room. Its sensitivity is governed by Fresnel-zone geometry. The n -th Fresnel zone is an ellipsoid, with the TX and RX at its foci, whose boundary marks where a reflected path arrives $n\pi$ out of phase with the direct path [26].

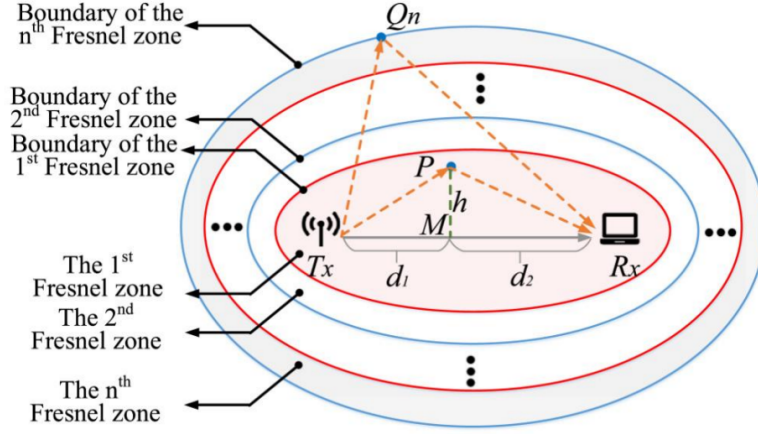


Figure 3.1: Geometry of the Fresnel zones. Reproduced from [56]

Figure 3.1 illustrates this geometry. The transmitter T_x and receiver R_x sit at the two foci of a family of confocal ellipses. Each ellipse is the locus of points Q_n for which the reflected path $|T_x Q_n| + |Q_n R_x|$ exceeds the direct LOS path $|T_x R_x|$ by an integer multiple of the half-wavelength,

$$|T_x Q_n| + |Q_n R_x| - |T_x R_x| = n \frac{\lambda}{2}, \quad (3.6)$$

The n -th Fresnel zone is the elliptical annulus between the $(n-1)$ -th and n -th boundaries. The innermost ellipse delimits the first zone, through which more than 70% of the transmitted energy propagates [56].

At a point lying at distances d_1 and d_2 from the two endpoints, the radius of the n -th zone is

$$r_n = \sqrt{\frac{n \lambda d_1 d_2}{d_1 + d_2}}, \quad (3.7)$$

with $\lambda \approx 0.125$ m at 2.4 GHz. For a representative 4 metre link, evaluating Eq. (3.7) at the midpoint ($d_1 = d_2 = 2$ m) gives a first-zone radius of about 0.35 m, and human motion outside this ellipsoid produces minimal variation in the CSI [26].

This geometry can be cast as an information-theoretic bound. Letting $O \in \{0, 1\}$ denote the occupancy state and $\mathbf{H}_\ell$ the CSI of link ℓ with Fresnel zone $\mathcal{F}_\ell$, the mutual information between the channel and the state satisfies

$$I(\mathbf{H}_\ell; O) \propto P(\text{Human} \in \mathcal{F}_\ell \mid O = 1), \quad (3.8)$$

so that if the region in which activity occurs does not intersect $\mathcal{F}_\ell$, then $I(\mathbf{H}_\ell; O) \approx 0$. A consequence is that the informativeness of any single link depends on whether its Fresnel zone intersects the region in which motion occurs: links whose Fresnel zones traverse only static space carry essentially no mutual information with human presence, a bound that no amount of algorithmic sophistication can overcome [26].

3.2.5 Orientation Dependence and the Case for Orthogonal Links

Beyond whether motion intersects a Fresnel zone, the angle of that motion relative to the link strongly modulates how much it perturbs the CSI. A body movement can be modelled

as a movement at speed v along a direction defined by an azimuth angle (direction in the horizontal plane) measured from the straight line connecting the TX and the RX (together with an elevation angle for the vertical component of the motion). As the moving object moves along this trajectory, the dynamic path length (the sum of the distances from the moving object to the TX and to the RX) changes over time, and it is the rate of this change, $d'(t)$, that sets the observed frequency shift through Equation (3.5) [49].

The key empirical and theoretical observation is that this shift is strongly orientation-dependent. Motion oriented perpendicular to the straight line connecting the TX and the RX produces a much larger frequency shift (and thus a much stronger, more discriminative signal) than the same motion performed parallel to the line (which produces much weaker, less discriminative CSI fluctuations). Crucially, this dependence is non-linear, since shifts for orientations close to perpendicular remain nearly indistinguishable, while the difference grows rapidly as the motion turns toward parallel [49]. Consequently, a single link is largely insensitive to motion along certain directions, so that a person's body orientation alone can substantially alter the reflected CSI pattern for an otherwise identical activity.

This directional selectivity is the second pillar motivating two TX-RX pairs. By arranging two links to view the sensing area from different angles, the configuration captures complementary spatial perspectives, so motion poorly observed by one link tends to be well observed by the other.

3.2.6 Sensing Topologies and Resolution Limits

Generally, configurations are classified as monostatic (co-located TX and RX), bistatic (distinct TX and RX), and multistatic (multiple TX and/or RX), with the double pair TX-RX arrangement studied in this thesis falling into the latter category. In a device-free bistatic topology, the moving body does not carry any radio. It is detected purely as a diffractor, reflector, or scatterer of signals exchanged between a separate TX and RX. This is also the arrangement adopted in this thesis, in which an edge device acts as the receiving endpoint of each pair link [31].

A fundamental constraint on any such system is its sensing resolution, which is inversely proportional to the operating bandwidth B as:

$$\frac{c}{2B} \quad (\text{monostatic}), \quad \frac{c}{B} \quad (\text{bistatic}), \quad (3.9)$$

where c is the speed of light [31]. Because commodity Wi-Fi in the 2.4 GHz and 5 GHz band offers only limited bandwidth, the achievable resolution is on the order of metres. Centimetre-scale resolution becomes attainable only at the far larger bandwidths of the mmWave spectrum. This bandwidth-imposed, meter-level resolution is an inherent limitation of the 2.4 GHz hardware adopted in this work, and it natively constrains the granularity of activity that can be distinguished. The paper further notes that sensing accuracy improves with additional RX and a larger number of TX [31]. This also motivated us to move from a single TX-RX pair to a dual-pair configuration. Whether this theoretical promise of spatial diversity and resolution translates into measurable gains for machine-learning-based activity recognition is the empirical question this thesis sets out to answer.

3.3 Specifications of the ESP32-S3

To facilitate HAR using Wi-Fi CSI in a lightweight, unobtrusive deployment, the Seeed Studio XIAO ESP32-S3 was selected as the edge computing platform. The XIAO ESP32-S3 is an ultra-compact development board built around the Espressif ESP32-S3R8 System-on-Chip (SoC), combining significant processing power with a minimal physical footprint (21×17.8 mm) [57].

The core of the system is an Xtensa® 32-bit LX7 dual-core processor operating at up to 240 MHz. This architecture provides the necessary computational throughput to capture, buffer, and process raw CSI packets in real time. Given the high data rate required to accurately detect fine-grained physical activities (ranging from macro-movements like walking to micro-movements like extending the arm), efficient memory management is critical.

To accommodate the heavy memory footprint of CSI time-series buffering and the execution of the subsequent HAR inference models, the board is equipped with 8 MB of in-package octal PSRAM. Furthermore, it includes 8 MB of Flash memory [57]. While the physical Flash capacity is 8 MB, not all of those MB are typically available for the application firmware and model weights.

Because the proposed methodology relies on analyzing perturbations in Wi-Fi signals to predict human activities, the board’s fully integrated 2.4 GHz Wi-Fi subsystem is its most vital component. The device supports 802.11 b/g/n protocols and provides direct access to the physical layer state, enabling the extraction of detailed CSI matrices [57]. The complete hardware specifications are summarized in Table 3.1.

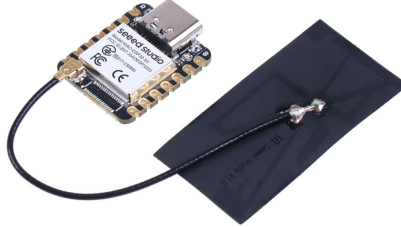


Figure 3.2: Image of ESP32-S3 with antenna

3.4 Denoising Filters

The raw CSI delivered by commodity hardware is never a clean observation of the channel. As established in Section 3.2.3, the measured response mixes the informative dynamic component $H_d(f, t)$ with static reflections, environmental noise, and hardware-induced artefacts. On low-cost single-antenna platforms such as the ESP32 these corruptions are particularly pronounced. Specifically, an unstable power amplifier in the RF chain adds

Table 3.1: Hardware Specifications of the Seeed Studio XIAO ESP32-S3 [57]

Component	Specification
Product	XIAO ESP32-S3
Processor	ESP32-S3R8 (Xtensa® LX7 dual-core, 32-bit up to 240 MHz)
External Memory	8 MB PSRAM, 8 MB Flash
Wireless	2.4 GHz Wi-Fi (802.11 b/g/n), Bluetooth LE 5.0
Dimensions	21 × 17.8 mm
Operating Voltage	3.8 V (battery)/ 5 V (Type-C)
Interfaces	1 × UART, 1 × IIC, 1 × SPI, 11 × GPIO, 9 × ADC

sudden spikes and bursts of noise to the amplitude, and interference from nearby Wi-Fi devices and household electronics lowers the signal quality further [58]. At the same time, thermal noise together with fluctuations from transmission-rate and power adaptation introduce isolated spikes that bear no relation to human motion [59]. Because the activity-recognition pipeline of this thesis operates on amplitude (Section 3.2.2), denoising is the preprocessing stage that conditions the signal before any feature extraction or classification.

It is useful to separate the corruptions into two regimes. The first is impulsive noise. This consists of isolated samples, upward spikes or downward dips, that depart sharply from the surrounding values while the rest of the signal remains well-behaved. The second is low-level Gaussian noise. In detail, this is small, broadly distributed fluctuation spread across the whole record. Order-statistic filters such as the median and Hampel filters are highly resistant to impulsive noise but are comparatively poorly suited to smoothing low-level noise [60], which instead motivates low-pass filtering (for instance, Butterworth) [61] and signal-decomposition methods such as the wavelet transform [59]. Some works pair a member of each family together [16, 61].

3.4.1 Outlier Removal: Median and Hampel Filters

The median filter is the classic approach for suppressing impulsive noise. It operates on the amplitude time series $\{x_k\}$ through a moving data window of half-width K ,

$$\mathbf{W}_k^K = \{x_{k-K}, \dots, x_k, \dots, x_{k+K}\}, \quad (3.10)$$

replacing each sample with the window median,

$$m_k = \text{median}\{x_{k-K}, \dots, x_k, \dots, x_{k+K}\}. \quad (3.11)$$

Its single tuning parameter, the window half-width K , governs the trade-off between outlier suppression and detail preservation. The filter is highly resistant to impulsive noise, but it is applied unconditionally to every sample and therefore tends to distort the portions of the signal worth keeping [60]. An example of the filter can be seen in Figure 3.3.

The Hampel filter removes this indiscriminate behaviour by acting only on samples that are statistically anomalous. Following the work [60], it is a decision filter that retains

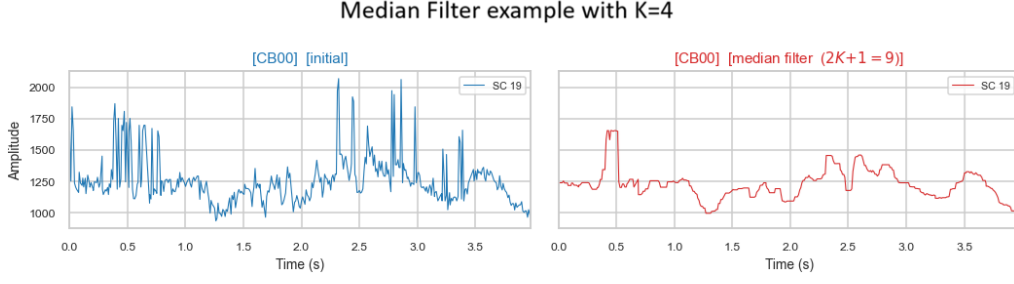


Figure 3.3: CSI amplitude preprocessing on subcarrier 19 during a recorded activity. **Left:** Raw CSI amplitude showing high-frequency burst noise and isolated outliers. **Right:** Denoised signal using a 1D median filter with a sliding window of size $2K + 1 = 9$ packets.

the original sample unless it lies too far from the local median m_k , in which case the median is kept:

$$y_k = \begin{cases} x_k, & |x_k - m_k| \leq t S_k, \\ m_k, & |x_k - m_k| > t S_k, \end{cases} \quad (3.12)$$

where the local scale is estimated from the Median Absolute Deviation (MAD),

$$S_k = 1.4826 \times \text{median}_{j \in [-K, K]} |x_{k-j} - m_k|. \quad (3.13)$$

The constant 1.4826 scales the MAD so that it matches the standard deviation under Gaussian data. As a result, the threshold t is expressed in robust standard-deviation units. Setting $t = 0$ recovers the median filter of Equation (3.11), which is why the Hampel filter is best understood as a tunable generalization of it. Increasing t makes the filter progressively less aggressive, approaching an identity map for large t [60]. An example is shown in Figure 3.4. Some works chain median and Hampel filtering into a multi-stage outlier-removal step for CSI amplitude [58], and Hampel filtering is often paired with a subsequent low-pass stage that preserves the slow, motion-induced signal variation [61].

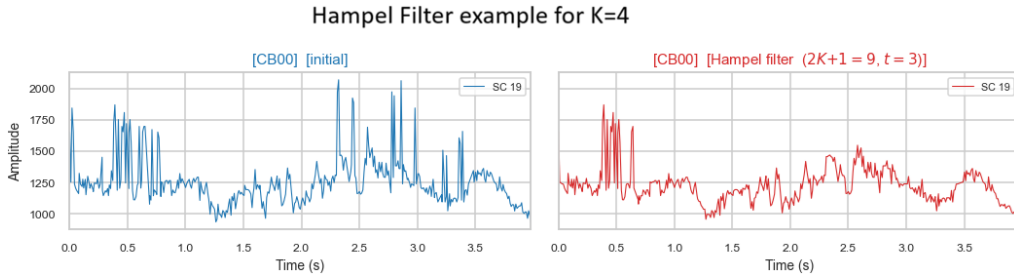


Figure 3.4: Outlier rejection via Hampel filter (window $2K + 1 = 9$, threshold $t = 3$) on raw CSI amplitude. Unlike a standard median filter, this approach selectively targets extreme deviations. This effectively suppresses isolated hardware spikes while strictly preserving the remaining high-frequency signal dynamics.

3.4.2 Frequency-Selective Smoothing: The Butterworth Low-Pass Filter

Once outliers are removed, the low-level Gaussian noise must be suppressed. Because human activity perturbs the channel at comparatively low frequencies, a low-pass filter preserves the motion signature while discarding faster fluctuation. The Butterworth filter is a common choice owing to its flatness of frequency response in the passband. For an N -th order Butterworth filter, the amplitude squared response is

$$|H(j\omega)|^2 = \frac{1}{1 + \left(\frac{\omega}{\omega_c}\right)^{2N}}, \quad (3.14)$$

where ω is the angular frequency, ω_c is the cut-off frequency separating passband from stopband and the order N sets the steepness of the attenuation outside the passband [16]. Figure 3.5 shows an example of applying this kind of filter. In practice, the filter order is chosen to balance roll-off steepness against computational cost. For example, the following works deploy an eighth-order low-pass stage in an embedded edge deployment with ESP32-S3 acquisition [16], or a fifth-order stage following Hampel filtering [61].

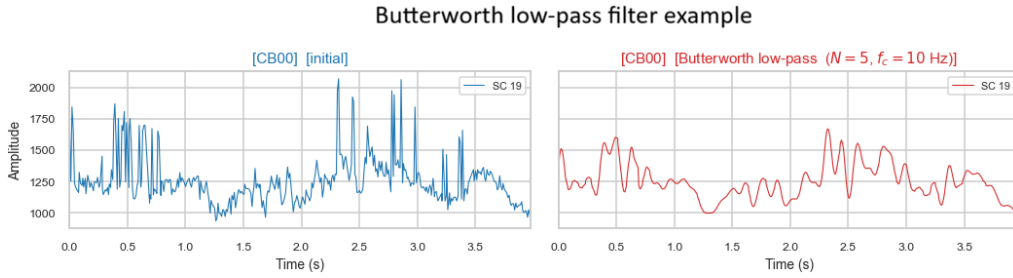


Figure 3.5: Smoothing CSI amplitude using a Butterworth low-pass filter on subcarrier 19 applied directly to the raw signal. **Left:** Raw CSI signal containing fast, high-frequency fluctuations. **Right:** Filtered signal using a 5th-order ($N = 5$) Butterworth filter with a 10.0 Hz cutoff frequency. By attenuating frequencies above the cutoff, this filter effectively suppresses low-level noise while preserving the low-frequency motion signature of the activity.

3.4.3 Wavelet Transform

The Discrete Wavelet Transform targets the same low-level noise regime as the low-pass filter of the previous subsection, but through a basis-driven rather than a fixed-cutoff mechanism. Instead of attenuating everything above a frequency cutoff, the signal is projected onto a wavelet basis and separated by coefficient magnitude. Large-magnitude coefficients are retained, while small-magnitude coefficients, which predominantly encode noise, are removed, and the inverse transform reconstructs a cleaned signal [59]. Wavelet-based denoising is also used to suppress environmental noise after phase calibration in dual-link speed-estimation systems [49].

The wavelet transform works by repeatedly splitting the signal into a smooth, low-frequency part (the approximation) and a fast-changing, high-frequency part (the detail)

over several levels, giving a representation of the signal at different scales that is useful for non-stationary signals such as CSI amplitude. To remove noise, the detail coefficients that fall below a noise-based threshold are set to zero (or shrunk in soft thresholding), and the signal is then rebuilt with the inverse transform. Unlike a low-pass filter, which simply discards everything above a single cutoff frequency, this cleans the noise inside each frequency band, so the sharp high-frequency features are preserved [62]. An example is shown in Figure 3.6.

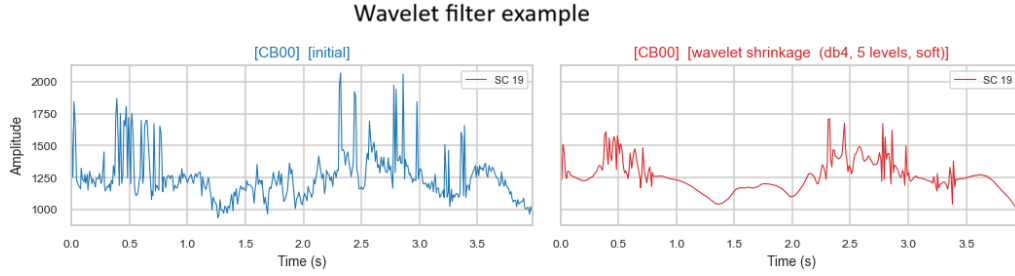


Figure 3.6: Denoising CSI amplitude using the Discrete Wavelet Transform on subcarrier 19. **Left:** Raw CSI signal. **Right:** Reconstructed signal using a **db4** wavelet (5 levels) and soft thresholding. By setting small-magnitude detail coefficients to zero across multiple scales, this method removes low-level noise while preserving sharp transients (impulsive outliers survive shrinkage, since they produce large-magnitude coefficients).

Our Methodology and Approach

This chapter describes how our system is built and how it is evaluated. The complete overview is shown in Figure 4.1. We start with the hardware, showing the TX and RX devices used to record CSI, and then present the central design choice of this thesis, which is the two TX-RX pairs placed so that their direct line-of-sight paths are mutually orthogonal. We then describe how the data was collected, presenting the audio-cued solo acquisition protocol shared by all our recordings. We then follow the data through the processing chain, first the preprocessing pipeline that cleans the raw amplitude, then the windowing stage that turns a recording into fixed-length inputs, explaining how the window duration, the timestamp-based cutting and the sliding stride are chosen. Next we define the evaluation protocols used throughout the thesis, followed by the two model families we compare. Specifically, the two model families consist of a classical pipeline of hand-crafted features and compact one-dimensional convolutional networks, together with the fusion, normalisation, augmentation, test-time adaptation, and training options built around them. Finally, we describe the multi-room collection of the cross-environment experiment, with its activity set and its leave-one-room-out protocol. The experiments that combine these components, and their results, are presented in the next chapter.

4.1 System Requirements & Hardware Analysis

The hardware of the whole system consists of four identical Seeed Studio XIAO ESP32-S3 boards, whose specifications were already given in Section 3.3. Two act as transmitters and two as receivers, forming the two TX-RX pairs described in the next section. Each board is fitted with a small external antenna and draws its power directly from a socket, so a device only needs a free power outlet to operate, as shown in Figure 4.2 for a TX and in Figure 4.3 for an RX. TX and RX are therefore the same device with the same specifications, and differ only in their role and in one addition. The RX carries a microSD card mounted on top of the board, on which it logs the captured CSI locally during a session. This is what keeps the deployment fully wireless, since no board is ever cabled to a laptop for power or for data transfer, and the recordings are retrieved over the network afterwards (Section 4.3). The result is a compact and unobtrusive setup. Specifically, each unit is about the size of the wall adapter it is plugged into, which makes it easy to place at the desired positions in a room without modifying the environment.

4.2 Proposed Architecture: Double TX-RX Setup

The double TX-RX configuration is motivated by the direction-selectivity limitation of single-link Wi-Fi sensing developed in Section 3.2.5. A single propagation path is intrinsically direction-selective, responding strongly to motion perpendicular to the link while

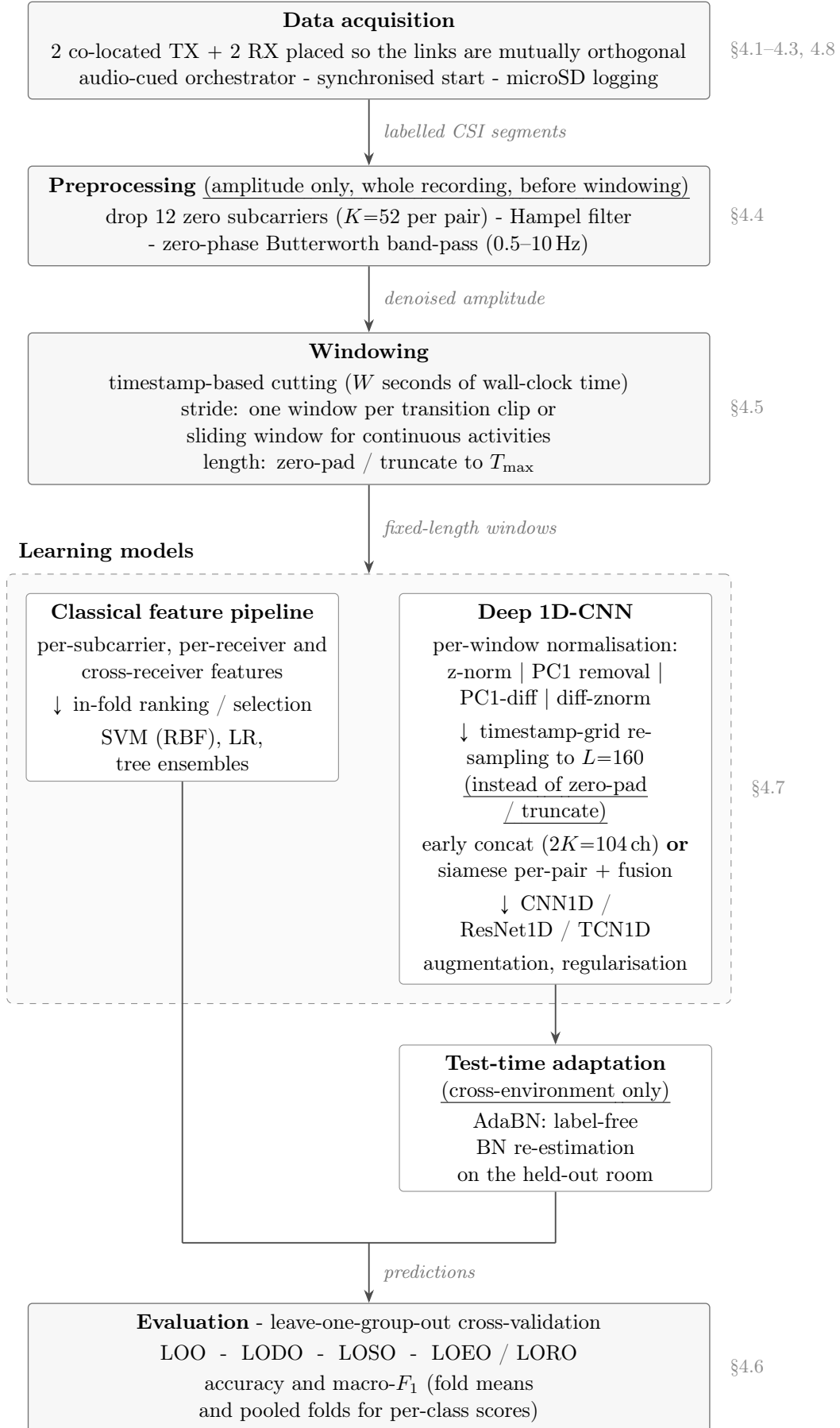


Figure 4.1: Overview of the methodology. Section numbers on the right indicate where each component is detailed.

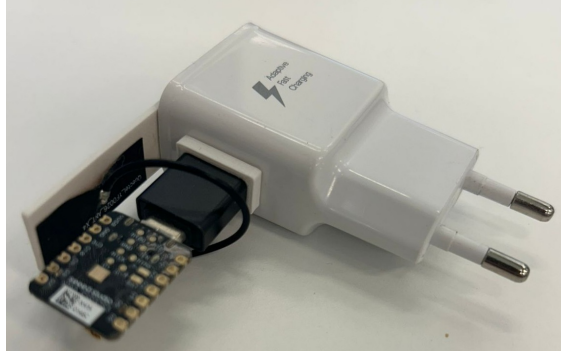


Figure 4.2: Image of TX device

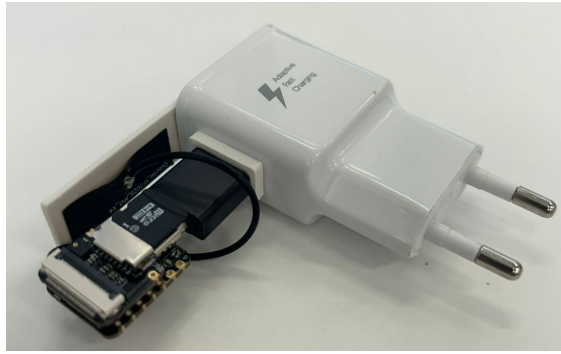


Figure 4.3: Image of RX device. Additionally it carries a microSD card mounted on top of the ESP32-S3

remaining largely insensitive to motion parallel to it. The same activity can therefore produce a strong or a weak CSI signature depending only on the subject's orientation relative to the link. To mitigate this, the two TX-RX links use separate TX that are co-located in one corner of the room, while the two RX are positioned so that the two TX-RX direct LOS paths are mutually orthogonal, as seen in the figure 4.4. This orthogonal geometry makes the two links observe the activity from complementary spatial perspectives. In other words, motion that is poorly captured by one link, because it runs near-parallel to that LOS path, is by construction close to perpendicular to the other, so at least one link retains strong sensitivity regardless of the subject's orientation within the sensing area. Orthogonality here is a deliberate placement choice rather than a mere increase in link count. As noted in Section 3.2.6, naively aggregating redundant links can dilute rather than improve performance [26]. Therefore, the second link is positioned to maximise the independent spatial information it contributes.

This placement is inspired by SpeedFi [49], one of the few Wi-Fi sensing systems to adopt a dual-receiver setup with deliberately orthogonal links and to sense the target off the direct TX-RX LOS line via reflected paths rather than on the line itself. Although SpeedFi targets fine-grained exercise-speed estimation rather than HAR, its design rationale transfers directly. It empirically shows that an orthogonal (90°) inter-link geometry yields markedly lower estimation error than 30° or 60° separations, since near-parallel links capture highly correlated CSI and contribute little independent spatial information. The

present work adopts the same orthogonality principle but extends SpeedFi’s shared-TX, dual-RX design to a fully separated two-pair multistatic topology. This is consistent with the multistatic argument of Section 3.2.6, in which sensing accuracy improves with both additional RX and a larger number of TX [31], and it matches the natural one-TX-to-one-RX operation of each ESP32 bistatic link. The resulting configuration is evaluated under both LOS conditions (the fine-grained kitchen experiment) and non-LOS conditions, testing whether this spatial diversity translates into measurable robustness gains for activity recognition.

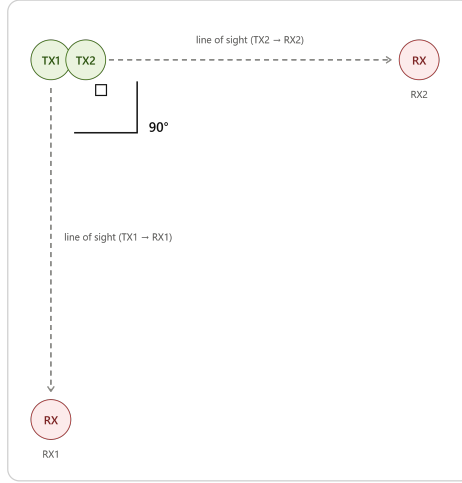


Figure 4.4: Floorplan of the double pair placement

4.3 Data Collection Procedure

All in-house data acquisitions of this thesis share the same collection procedure. Every recording was performed, supervised, and labelled by the author, acting simultaneously as experimenter and subject. Reliable labels under this constraint are only achievable if the data collection procedure is partly automated. The subject cannot simultaneously perform an activity and mark its boundaries. To this end, every session was driven by a custom Python acquisition orchestrator script, developed by a member of the hardware team and configured by us for each experiment’s protocol. This script coordinates the two RX boards, guides the subject through a scripted protocol with audio cues, and produces automatically segmented, labelled recordings without any manual annotation. For all the experiments, we used the fully wireless version described here, in which each receiver logs CSI autonomously to a local microSD card and is coordinated over the network and downloaded to our laptop.

Session protocol. A recording session is a fixed, pre-declared sequence of segments. After a short warm-up period (discarded from the data), the protocol alternates action segments and pause segments. During an action segment, lasting a configurable duration

d_a , the subject performs exactly one scripted activity. During the subsequent pause segment, lasting d_p , the subject repositions for the next action. Sessions begin with buffer phases that absorb the initial settling of the channel and of the subject.

Each segment boundary is announced by a distinct sound. In detail, one cue signals the exact moment an action must start, a different cue signals the start of a pause, and a closing jingle marks the end of the session. This audio-cued design is what makes solo collection feasible, as the subject never needs to look at a screen or operate a device mid-session. Moreover, because the same cue that prompts the subject also triggers the log entry, the recorded segment boundaries match the moments the actions are actually performed, up to human reaction time. Cue timing follows an absolute schedule anchored to the session start. Each cue fires at its exact pre-computed offset from the beginning of the session, rather than being timed relative to the end of the previous segment, so small software delays cannot accumulate over the course of a session. The orchestrator logs the resulting timeline as an ordered list of (`start timestamp`, `end timestamp`, `action name`, `label type`) tuples, which constitutes the ground-truth segmentation of the session. In other words, no post-hoc manual labelling is required for the controlled clips.

Double-pair synchronisation. Because the two pairs are recorded by two independent, unattached RXs, their streams must be aligned in time. Each RX periodically announces its presence to the orchestrator script. When a session is created, the selected boards acknowledge it and the orchestrator assigns a single synchronised start instant, expressed in absolute wall-clock time and scheduled far enough in the future to exceed the worst-case network round-trip. Both RX therefore begin recording within a few tens of milliseconds, and the offset between the recording start and the first audio cue (the preamble) is measured and stored, so that the protocol timeline can be mapped exactly onto the packet timestamps of both CSI streams. This wall-clock alignment is also what makes the timestamp-based windowing of Section 4.5 exact.

Quality control. Different safeguards protect against silently corrupted sessions. First, before a session is allowed to start, each RX measures its incoming CSI rate over a ten-second window. If any board falls below 90% of the nominal 100 Hz rate, or exceeds the loss/gap limits, the session is aborted and needs to be restarted. This can happen due to interference or device issues. Second, no telemetry is exchanged during the actual recording. The ESP32-S3 has a single radio, and concurrent network traffic would itself cause CSI packet loss. Recording integrity is instead assessed after the recording, from the hardware timestamps of the logged packets, as per-segment statistics (effective sampling rate, maximum inter-packet gap). Lastly, an RX aborts the session itself and flags the error if it stops capturing CSI packets altogether, so a total mid-session failure is reported rather than silently truncating the recording, and the run is repeated.

Output. During recording, each receiver logs the raw CSI stream to its local microSD card. At the end of a session, the logs are uploaded from the RX to the startup's cloud storage bucket, from which the orchestrator laptop downloads them. Once the transfer is verified, the files are deleted from both the microSD cards and the server. The downloaded logs are then converted into per-segment records. Specifically, one labelled CSI segment per

action or pause phase, with the warm-up stripped and the per-segment quality statistics attached. This retrieval path makes the deployment fully wireless. In other words, the laptop is never cabled to any TX or RX, and each TX and RX only needs a power socket. These segmented recordings are the direct input to the preprocessing pipeline of Section 4.4.

4.4 Preprocessing pipeline

Before any windowing or learning, every segmented recording of Section 4.3 passes through a fixed sequence of signal-conditioning steps. The pipeline operates on CSI amplitude only (Section 3.2.2) and begins with a structural clean-up. Of the 64 subcarriers reported by the ESP32-S3, the 12 subcarriers that are structurally zero are removed, leaving $K = 52$ informative channels per pair.

Then the amplitude is denoised following the two-regime view of Section 3.4. A Hampel filter (window of 9 packets, 3σ MAD threshold) suppresses the impulsive spikes characteristic of the ESP32 RF chain while leaving genuine motion-induced variation untouched, and a zero-phase, second-order Butterworth filter removes the remaining broadband noise. For the band limits, a band-pass of 0.5-10 Hz proved consistently superior to any other alternative tested in the main experiment. The upper cut-off discards fluctuation faster than human motion, while the lower cut-off additionally removes the slowly drifting static-multipath baseline that differs between rooms, days and sessions (a property that matters most under the cross-environment protocol).

Filtering is applied at the level of the whole segment, before windowing, and not to each window separately. Running the filter in both directions distorts the samples at both the start and the end of the signal it is given. Filtering window by window would therefore put that distortion at the start and end of every window, whereas filtering the segment itself confines it to the segment’s own first and last samples.

Different alternatives were evaluated and set aside. A very important insight was that configurations without the band-pass stage were consistently dominated in the deep-learning pipeline sweeps of Chapter 5. The output of this stage (despiked, band-limited per-pair amplitude streams with their packet timestamps) is the input to the windowing stage of Section 4.5, where the remaining per-window operations are defined.

4.5 Windowing

The ML pipeline does not consume a whole recording at once. Each recording is first sliced into fixed-duration analysis windows, and every window becomes one input tensor.

Windowing is therefore the stage that turns a variable-length CSI amplitude stream into the uniform fixed-length inputs the network expects (up to $2K = 104$ channels when the two pairs of Section 4.2 are consumed jointly). Two decisions govern this stage: how long each window is, and how windows are positioned within a recording.

Window duration and packet loss. A natural first instinct is to set the window equal to the nominal duration of the recorded action. In practice this fails, because the ESP32 does not deliver a perfectly regular packet stream: packets are occasionally lost

and the effective rate fluctuates around its nominal $f_s \approx 100$ Hz. A clip nominally lasting x seconds therefore contains slightly fewer than $f_s x$ usable packets, and the duration it actually covers is a little shorter than x . To guarantee that every recording contains at least one full window, the window length W is set strictly below the nominal clip duration (e.g. for a 4s action clip a safe choice is roughly $W \approx 3.5$ s rather than 4.0s) leaving a margin that absorbs the missing packets and the start/stop jitter of the audio-cued protocol (Section 4.3). Cutting windows too close to the nominal length would silently drop the shortest recordings of each class, reducing an already limited training pool.

Timestamp-grid resampling. The cross-environment experiment replaces the zero-pad/truncate step with resampling: after per-window z-normalisation, each channel is linearly interpolated from its true packet timestamps onto a uniform wall-clock grid of $L = 160$ points spanning the window. This removes the packet-rate jitter of the ESP32 stream (the effective rate wobbles around the nominal 100 Hz) and, because both receivers are interpolated onto the same wall-clock grid, exactly time-aligns the two pairs' windows (a property the synchronised-start acquisition of Section 4.3 makes possible).

Timestamp-based cutting and fixed tensor length. For the same reason, windows are cut on wall-clock time rather than on a fixed packet count. Each window spans exactly W seconds read from the packet timestamp column, so it covers the same physical duration regardless of how many packets happened to arrive in that interval. Because the resulting packet count still varies slightly from window to window, every window is then zero-padded or truncated to a common length $T_{\max}$, chosen as a high percentile of the observed per-window packet counts (the 95th percentile, e.g. $T_{\max} = 289$ packets for a 2.9s window). This yields identically shaped tensors without distorting the large majority of windows.

Stride: continuous versus atomic activities. The number of windows a recording yields depends on the nature of the activity. Brief, atomic events like the posture transitions (`sit`, `lying`, ...) and the short kitchen gestures, are recorded as one action per clip and yield a single window. Overlapping windows here would merely re-sample the same isolated event. Continuous, state-like activities like `walk`, `prepare_meal`, instead persist for many seconds, so a single recording is cut into several windows using a sliding stride S . The window is advanced by S seconds (e.g. $S = 2.5$ s) between successive cuts. For a recording covering D seconds this produces

$$n_w = \left\lfloor \frac{D - W}{S} \right\rfloor + 1 \quad (4.1)$$

windows. Striding serves two purposes. First, it makes data collection more efficient: rather than recording each state-like activity many times, a single longer recording is cut into several labelled windows, so fewer physical recordings are needed to obtain the same number of samples. Second, it provides a lever for class balancing. The number of short transition clips actually collected varies per class. This depends on how many were captured and how many were lost to recording failures. As a result, the stride length can be tuned to make the continuous classes yield a comparable number of windows, rather than either swamping the transition clips or being swamped by them.

Window duration as a hyper-parameter. In some experiments, the window length W is not fixed a priori but treated as an experimental factor, with W swept across configurations. Longer windows give the classifier more temporal context per decision but reduce the number of windows via Equation (4.1). Shorter windows retain more recordings and produce more windows, at the cost of less context per decision. One important consequence of striding carries directly into the next section. Since windows cut from the same recording overlap and share static-multipath context, the recording (not the individual window) must be the atomic unit that any evaluation split holds out, or else temporal leakage occurs.

4.6 Evaluation methodology

A single accuracy figure is uninterpretable unless it is clear what kind of generalisation it reflects. A model can score highly because it genuinely recognises an activity, or only because the test data shares some incidental factor (e.g. the same person, room, day, or even the same recording) with the training data. To separate these cases, every experiment in this thesis uses a leave-one-group-out cross-validation scheme, in which the “group” is precisely the factor whose influence is being measured. In each fold one group is withheld entirely for testing while the model trains on all the others, rotating until every group has served once as the test set. Performance is reported as the mean over folds, using classification accuracy together with macro-averaged F_1 . The latter is insensitive to class imbalance, and where every class is collected with the same number of trials the two track closely. The choice of grouping factor defines four protocols, each isolating a different generalisation axis.

Leave-one-clip-out (LOO). This is the finest partition having one held-out recording per fold. It measures only whether the controlled clips are internally separable, not whether the model transfers, and is the in-distribution protocol used for the same-day controlled experiment (Section 5.3). LOO is valid only when every clip is an independently recorded, single-activity segment. It must never be applied to windows obtained by striding over a longer recording (Section 4.5). Overlapping windows from one recording share packets and static-multipath context, so placing some in training and others in test leaks information across the split and inflates the score. Whenever a sliding stride is used, the whole recording (not the clip) is the atomic unit a fold holds out.

Leave-one-subject-out (LOSO, cross-subject). Folds are split by the person performing the activity, so no window from the held-out subject appears in training. This measures whether the model has learned activity-specific patterns rather than person-specific signatures (e.g. body geometry, gait, height) and directly addresses the identity-leakage failure mode documented in Section 2.4.3, where mixing subjects across the split produced optimistic but misleading accuracy.

Leave-one-environment-out (LOEO, cross-environment). Folds are split by room, so no recording from the test environment is seen during training. Because CSI

encodes the surrounding multipath structure (walls, furniture etc.), as well as the activity, this is among the hardest and most deployment-relevant settings. It asks whether a model trained in some rooms still works in a room it has never observed, which is the realistic condition of a deployed monitor installed in an unseen home environment. It is used for the ESP-Fi cross-scene reproduction (Section 5.1) and for the double-pair cross-room daily-activity study (Section 4.8 and 5.4), there under the room-specific name leave-one-room-out (LORO).

Leave-one-day-out (LODO, cross-day). Folds are split by recording day, so no sample collected on the test day appears in training. This isolates temporal drift. Even in a fixed room with a fixed subject, the channel changes between sessions as device placement shifts and as ambient conditions (including co-channel interference from other Wi-Fi devices and electrical appliances) vary from day to day. As a result, a model that has implicitly memorised one day’s static background degrades on another. It is the protocol behind the cross-day double-pair experiment (Section 5.2).

Combining and comparing protocols. These four axes are not mutually exclusive and can be stacked to rule out residual leakage. For instance, a subject-disjoint LOEO variant withholds a subject and an environment simultaneously, testing whether apparent cross-environment gains are in fact memorised per-subject cues (Section A.2.3). Generally, the protocols increase in difficulty and deployment realism from LOO, to LODO, LOSO, and LOEO, each of which withholds a factor that a real system would face. All of the above score pre-segmented windows. Continuous deployment is even harder. The stream is unsegmented, activity boundaries are unknown, and a sliding window may straddle two activities at once. We identify and evaluate this challenge in the Section 5.3

Fold means and pooled predictions. The results in this thesis summarise the same held-out predictions in two ways, so it is worth stating the difference once. Headline scores are always the mean over the folds, because the withheld group (the room, the day, the subject) is what we want to generalise to, and each fold should count once. Per-class scores and confusion matrices are instead computed on the predictions of all folds pooled together. This is not a second evaluation, as nothing is retrained and no split changes, but only a different way of aggregating the same predictions. The reason is F_1 may move by several points from fold to fold, while pooling the three folds makes it more stable. Also, a confusion matrix cannot be averaged over folds in a meaningful way. Since F_1 is not linear in the counts and the folds are not equally difficult, the two summaries do not give exactly the same number. For this reason we never mix them together. A pooled number is only compared with another pooled number, and a fold mean with another fold mean.

4.7 Learning Models

All experiments in this thesis draw their classifiers from two model families: a classical pipeline of hand-crafted features with conventional machine-learning classifiers, and a family of compact one-dimensional convolutional networks operating directly on the windowed CSI amplitude. Three constraints shaped both families. First, the training pools

are small (in the cross-environment on the order of a few hundred labelled windows), which rules out high-capacity architectures that only pay off at scale. Second, the input is not an image but a multichannel time series. A window is a matrix of amplitude over time across the $K = 52$ subcarriers of each pair, consumed either concatenated into a single $T \times 2K$ input ($2K = 104$ channels) or per pair, and the discriminative structure is temporal (motion-induced fluctuation) rather than spatial. Third, the long-term deployment target is ESP32-class edge hardware, which favours models with small parameter counts and cheap inference.

4.7.1 Classical feature-based pipeline

The classical pipeline condenses each window into a fixed-length vector of statistical and spectral descriptors computed per subcarrier, per receiver, and across receivers: distributional statistics (mean, dispersion, skewness, kurtosis), spectral-shape measures (spectral entropy, dominant frequency, band powers), autocorrelation profiles, and complexity measures (Hjorth parameters, permutation entropy), together with receiver-level summaries and cross-receiver correlation features that explicitly encode the double-pair structure. The full feature bank is specified in Appendix A.4.2. Because the number of features far exceeds the number of training windows, feature selection is mandatory. Features are ranked by an ℓ_1 -penalised one-vs-rest logistic regression, or, for the tree ensembles, by their own feature importances. In some experiments the number of retained features is treated as a hyperparameter, with the ranking recomputed inside every cross-validation fold to avoid selection leakage. The selected vectors are classified with standard learners (support vector machines with RBF kernel, logistic regression, and tree ensembles). This family serves two main roles. First, a strong, interpretable baseline against which the deep models must justify their complexity, and also a diagnostic tool, since the ranked features indicate which signal properties carry class information.

4.7.2 Deep temporal convolutional models

The deep family treats the window as a multichannel one-dimensional signal ($2K$ channels under early concatenation, K per pair under the siamese arrangement of Section 4.7.3) and applies convolutions along the time axis only. This choice reflects the physics of Section 3.2.3: motion is encoded as temporal fluctuation of each subcarrier’s amplitude, whereas the subcarrier axis has no comparable translation structure for a convolution to exploit. In the main cross-room experiment, image-style two-dimensional convolutions over the time-subcarrier plane were evaluated but did not beat a one-dimensional backbone. One-dimensional temporal convolutions therefore match the structure of the data while keeping the parameter count orders of magnitude below image-style 2D backbones. Three architectures of increasing sophistication are used:

- **Shallow 1D CNN (CNN1D)** Three temporal convolution blocks (kernel sizes 9, 5 and 3; 64, 128 and 128 channels, each followed by batch normalisation and a ReLU, the first two by a max-pooling and a dropout stage), closed by global average pooling and a linear head. The shallowest and fastest model, used as the deep baseline.

- **Residual 1D CNN (ResNet1D)** A compact residual network ($\approx 143k$ parameters; full specification in Table A.19) whose skip connections stabilise training on small datasets. A variant, **ResNet1DStats**, replaces the global average pooling with concatenated mean-and-standard-deviation pooling, so that near-static classes are not flattened away by averaging.
- **Temporal Convolutional Network (TCN1D)** A stem convolution followed by four residual blocks of dilated temporal convolutions (dilations 1, 2, 4 and 8), closed by the mean-and-standard-deviation pooling described above. The exponentially growing dilation gives a receptive field spanning the majority of the window at constant depth and parameter count, capturing both the fast fluctuation of gestures and the slow envelope of whole-body transitions. This is the final backbone of the cross-environment experiment (Section 5.4).

Recurrent (LSTM, BiLSTM) and attention-based (Transformer) sequence models were evaluated in early sweeps but consistently underperformed the convolutional family on our data volumes and thus were dropped. Also, the mentioned architectures mirrored their behaviour on the public benchmark of Section A.2.2. Data-hungry architectures are a poor match for the limited sampled datasets of this thesis experiments.

4.7.3 Per-pair processing: siamese trunk and fusion

The models of the previous subsection can consume the double-pair input in two ways. The simplest is early concatenation. The two pairs' subcarriers are stacked into a single $2K$ -channel window and processed jointly, as in the kitchen experiment. The cross-environment experiment instead processes each pair separately. Every link's $T \times K$ window is passed through the same trunk (a siamese arrangement in which the two links share all convolutional weights) and the two resulting descriptors are combined only afterwards. Weight sharing halves the number of parameters the small training pool must support and forces the trunk to learn link-agnostic motion features rather than binding features to one receiver's geometry.

Two ways of combining the two links are considered, both applying a single shared set of trunk weights to each link. Feature-level fusion concatenates the two links' pooled descriptors into a single vector before the classification head, letting the head weigh the two views jointly. Late fusion instead trains a single-link network on the pooled windows of both links (each recorded window enters training twice, once per link, carrying the same label) and averages the two links' predicted class probabilities at inference time. Late fusion has a practical advantage. The same trained model can be deployed on one pair alone or on both, so a single training run covers both the single-pair and double-pair deployment scenarios.

4.7.4 Per-window normalisation

Between windowing and the network input, the principal normalisation modes are applied per window and per channel. One is z-normalisation, which standardises each channel over time and removes static amplitude offsets while preserving the fluctuation structure. Second is PC1 removal, which subtracts the projection onto the window's first principal

component (the dominant static multipath background shared across channels) before z-normalising the residual. Third is PC1-diff, which additionally differences consecutive frames to cancel any residual static component. Fourth, differencing may also be applied on its own, without the PC1 step (diff-znorm). These modes trade robustness against information loss. Specifically, removing more of the static background improves invariance to the environment at the risk of discarding class-relevant amplitude structure. The mode is treated as a pipeline hyper-parameter and selected per experiment.

4.7.5 Augmentation and regularisation

Because the controlled clips are few and were collected in a single configuration, the deep models are trained with aggressive on-the-fly augmentation drawn from a common pool of operations. Additive Gaussian noise (global or per-channel), random amplitude scaling, temporal shifting, and time masking act on the whole window, while subcarrier masking is drawn independently for each receiver (randomly selected subcarriers in the cross-environment recipe). The operation specific to the double-pair architecture, device dropout, zeroes one entire receiver’s 52 channels with a fixed probability, forcing the network to remain predictive from either pair alone and directly guarding against any silent-pair failure. Training-level regularisation like mixup (optionally restricted to same-class pairs, which produces plausible intra-class variation without blending labels across activities), label smoothing, focal loss, and exponential-moving-average weights are drawn from the same pool. The exact combination used by each experiment is specified in the corresponding experiment.

4.7.6 Test-time adaptation: Adaptive Batch Normalisation

Under the cross-environment protocol, the batch-normalisation layers pose a subtle problem. Their running mean and variance are estimated on the training rooms, so at test time every activation is normalised by statistics that reflect the training rooms’ channel distribution rather than the unseen room’s. Adaptive Batch Normalisation (AdaBN) [63] corrects exactly this. After training, the BN running statistics are discarded and re-estimated by passing the held-out room’s windows through the frozen network. Crucially, this uses no labels (only the unlabelled test windows, which any deployed system observes for free) so it introduces no leakage into the leave-one-room-out evaluation. All network weights remain untouched. In the double-pair setting the statistics are re-estimated separately per test link, since the two RX exhibit different amplitude distributions. AdaBN is a one-off, inference-time procedure costing two forward passes over the test windows, making it well suited to the install-and-forget deployment model. That means a newly installed monitor can self-calibrate to its room from a short cued sweep of unlabelled windows covering each class, in under a minute.

4.7.7 Training procedure

Unless stated otherwise, all deep models are trained with the Adam or AdamW optimiser (the exact variant is stated per experiment) under a cosine-annealing learning-rate schedule and a cross-entropy loss with label smoothing, for a fixed epoch budget with

mini-batches of 32. To reduce the variance of individual training runs on small data, the cross-environment experiment additionally trains each configuration from three random initialisations and averages the predicted class probabilities across the three networks (a seed ensemble). No test window ever enters a gradient update, and no test label is used at any point. The only test-set interaction is the label-free statistics re-estimation AdaBN of Section 4.7.6. Moreover, pipeline configurations were ranked by their mean score over the held-out folds rather than on an inner validation split due to the limited data samples issue described.

4.8 Cross-Environment Experiment Data Collection

The cross-environment experiment asks a different and, for deployment, more demanding question compared to the preliminary experiments. Does a model trained with the same double-pair architecture and NLOS conditions still recognise daily-activities in a room it has never seen? Because no public CSI dataset captures the same activity set across several rooms with the identical dual-link hardware, this experiment relies on the in-house acquisition built around the double-pair architecture introduced in Section 4.2. The experiment consists of a cross-room, same-subject, non-LOS daily-activity recognition. This section describes the sensing geometry, the activity set, the multi-room collection, and the leave-one-room-out protocol used to measure cross-environment generalisation.

4.8.1 Sensing setup

The deployment reuses the two ESP32-S3 TX-RX pairs of the double pair configuration, where each RX acts as the receiving endpoint of a bistatic link and identified throughout by its device identifier, AC84 and CB00. Here, actions are performed in the non-direct LOS of any of the two pairs. Both observe the same subject performing whole-body daily activities from two different vantage points. The two TX are co-located and share a single multiplug, whereas the two RX occupy complementary positions in the room. CB00 is mounted along one wall and AC84 along another wall, each at a different stand-off distance from the activity area. This arrangement makes the two propagation paths mutually orthogonal. As a result, it is possible to get two simultaneous, spatially decorrelated bistatic views of every activity, the same complementary-view argument motivated by the Fresnel-zone blind-spot and orientation-dependence analysis of Sections 3.2.4 and 3.2.5. A posture change that is near-parallel to one link (and therefore weakly modulated on that path) is generally transverse to the other, so the two RX cover each other’s blind spots.

Each RX reports CSI amplitude over 64 subcarriers at roughly 100 Hz. Phase information is discarded, consistent with the amplitude-only choice justified in Section 3.2.2. Of the 64 subcarriers, 12 are null and thus are removed, leaving $K = 52$ usable subcarriers per pair. In the initial tries, the two pairs are concatenated along the subcarrier axis, producing an input of $2K = 104$ channels per packet. In later tries, the two pairs are recorded as two synchronised K -channel streams. In that case, they are not concatenated into a single input, each pair’s window is processed separately by the shared trunk of Section 4.7.3, and the two views are fused at the feature or probability level.

Every recording folder additionally begins with a short `empty` segment (the static

room, no person present). Its per-channel statistics are retained as an optional empty-room calibration reference.



Figure 4.5: The double-pair setup in the room MC208. The two TX are co-located. The red circles contain the RXs and the green ones the TXs.

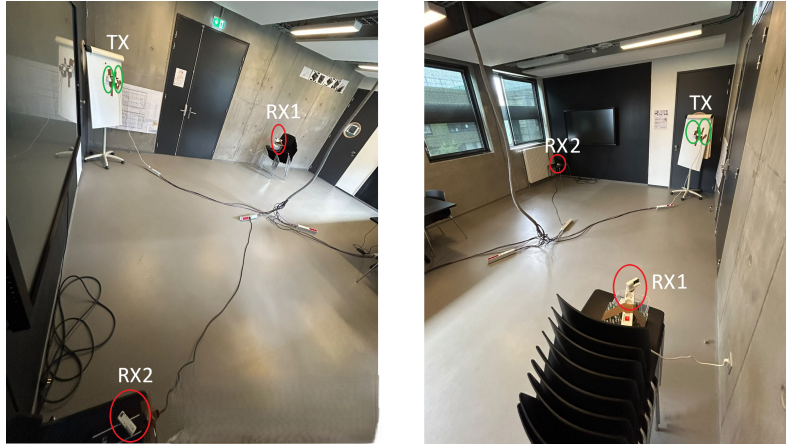


Figure 4.6: The double-pair setup in the room MF207.

Table 4.1: Physical configuration of the double-pair setup. Both TX share one multiplug and are mounted at the same height (≈ 140 cm).

Link	RX naming	TX height (cm)	RX height (cm)	RX–TX distance (m)
AC84	RX1	≈ 140	95	3.8
CB00	RX2	≈ 140	55	3.5

4.8.2 Activity set

The cross-environment set targets coarser, whole-body activities of daily living (ADLs) that the elderly would perform. The choice of these six classes is driven by the clinical

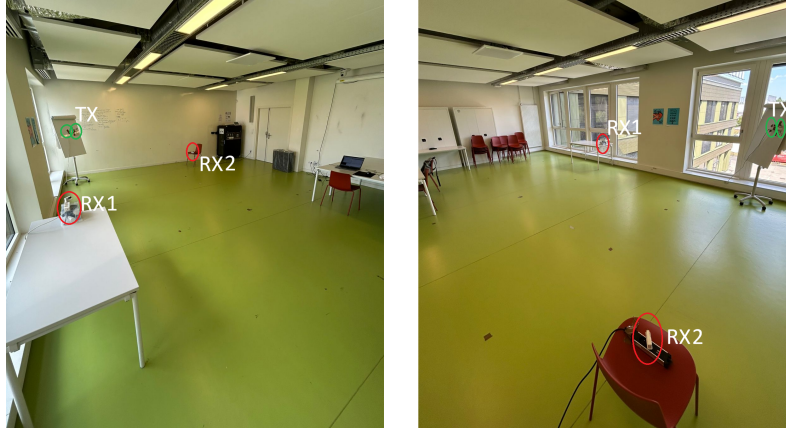


Figure 4.7: The double-pair setup in the room SB207.

target of the experiment. More precisely, the target is unobtrusive monitoring of older adults, for whom the ability to independently perform daily activities is a direct marker of functional status. Functional decline in ADLs, instrumental ADLs and mobility behaviour is known to begin years before the clinical symptoms of cognitive impairment and dementia, which makes a low-cost, device-free sensor that can log these everyday activities in the home a valuable early-warning tool [4]. Crucially, a device-free Wi-Fi sensing approach requires the user to wear or carry no sensor and to operate no app, which is particularly valuable for elderly people.

The specific six classes mirror the activity vocabulary that recent elderly-care Wi-Fi sensing protocols set out to capture. The work [4], deploying a device-free Wi-Fi sensing system in the homes of low-income older adults, suggests monitoring precisely this mix of meal-related instrumental activities, seated behaviour (sitting at the dining table or on the sofa), horizontal rest (lying on the sofa) and walking between rooms. Our set condenses that vocabulary into six atomic, room-agnostic units, listed in Table 4.2. Two of which are continuous activities (**walk** and **prepare_meal**, standing and performing meal-preparation arm movements) and four are brief posture transitions (**sit**, **sit_up**, **lying** and **lying_up**). The two paired transitions are deliberately kept distinct (i.e sitting down versus rising from the chair, lying down versus getting up) because sit-to-stand and lie-to-stand transfers are exactly the mobility events whose slowing or increasing difficulty signals functional decline. Those six classes stress the cross-environment task precisely where a deployed elderly-care monitor would most need it to remain reliable.

One extra label present in the raw recordings is deliberately excluded from the classifier. The **empty** label is reserved for the per-room calibration reference described above and is not a prediction target in the main experiment but only in the section 5.4.7. As a result, the six action classes are described in the Table 4.2, over which every model is trained and scored.

Table 4.2: The six cross-environment activity classes and how each was physically modelled during collection.

Class	Type	# Samples	Physical realisation
walk	continuous (≈ 10 s)	24–26 per environment	Walking around the room
prepare_meal	continuous (≈ 10 s)	24–26 per environment	Standing and performing meal-preparation arm movements
sit	transition (≈ 4 s)	70 per environment	Sitting down onto a chair
sit_up	transition (≈ 4 s)	70 per environment	Rising from the chair to standing
lying	transition (≈ 4 s)	70 per environment	Lying down on a mattress on the floor
lying_up	transition (≈ 4 s)	70 per environment	Getting up from lying to standing

4.8.3 Multi-environment collection

The distinguishing feature of this acquisition is that the same activity set was recorded in three different rooms, so that “environment” becomes an explicit, controlled factor. The three rooms (referred to by their room codes **mc208**, **sb207** and **mf207**) differ in size, furniture layout and static multipath, and each was recorded on separate days over June and early July 2026, as summarised in Table 4.3. All recordings were performed and captured by the same single subject (myself) using the audio-cued solo-collection protocol of Section 4.3. Each activity is performed on cue as a short, controlled, single-activity clip and segmented automatically, which keeps labelling reliable but bounds how much data can realistically be gathered per room. After balancing, each room contributes roughly 70 segments for each of the four transition classes and a comparable number of windows for the two continuous classes, giving on the order of 420 labelled windows per room.

Table 4.3: The three data-collection rooms, the days on which each was recorded, and its role as one cross-validation fold.

Fold	Room	Recording days
A	mc208	15, 16 June, 1 July
B	sb207	18, 25 June
C	mf207	29, 30 June

Each labelled segment is converted to fixed-duration analysis windows cut from the packet timestamp column rather than from a fixed packet count, so that every window spans exactly the chosen number of seconds of wall-clock time regardless of the per-recording packet sampling rate. The window duration is treated as an experimental factor (2.5, 3.0 or 3.5 s). The four short transition recordings yield exactly one window each (one sample, no overlap), while the two longer continuous recordings are cut with a 2.5 s

sliding stride for the 6-class grouping (and 0.5s sliding stride for the 4-class grouping), chosen so that `walk` and `prepare_meal` contribute a comparable number of windows to the transition recordings rather than swamping them.

4.8.4 Cross-environment evaluation protocol

The purpose of the multi-room collection is to measure generalisation to an unseen environment, so the models are evaluated under a strict leave-one-room-out (LORO) cross-validation, with one fold per room (Table 4.3). In each of the three folds the model is trained on the two remaining rooms and tested on the held-out room, rotating so that every room serves as the test environment exactly once. Since the split is by room, no labelled data from the test room is used during training. Performance is reported as the mean macro-F1 (with accuracy alongside) across the three held-out folds, so that a configuration is rewarded only if it transfers across rooms rather than exploiting room-specific static multipath. This same-subject, cross-environment protocol isolates environment shift as the factor under study and is deliberately harder than the evaluation of the preliminary experiments of Section 5.2.

Experiments and Evaluation

This chapter presents the four experiments of this thesis and their results. We begin by reproducing a published cross-environment benchmark on the ESP-Fi dataset, which gives a reference point for what a single TX-RX pair on ESP32 hardware can achieve in a room the model has never seen. We then report two preliminary experiments with our own double-pair setup. The first records coarse activities in NLOS conditions and in one room over several days and finds that held static postures cannot be told apart across days. The second targets fine-grained kitchen gestures in LOS and shows that the same model loses a large part of its accuracy once it is scored on continuous real kitchen routines. The main experiment then attacks the change of environment on six daily activities recorded in three rooms under leave-one-room-out cross-validation. We follow the order in which the pipeline was actually built, from the initial grid-search plateau to the Siamese shared trunk, the augmentation and test-time adaptation recipe, the timestamp-grid resampling, and we also measure how much the second pair is really worth against every single-pair deployment option. Lastly, we extend the label set with a `no_action` class for idle periods and test the model in longer continuous recordings.

5.1 Reproducing Cross-Environment Benchmark

Before this thesis introduces its double-pair contribution, it needs a reference point. This reference point is how well device-free human activity recognition currently performs on ESP32-class hardware with a single TX-RX pair, measured on published data with published models. The aim here is not to validate a preprocessing or postprocessing pipeline of our own but to measure the baseline that the published models already achieve on their own data, and in particular to settle the question the rest of this thesis is about: whether it is possible for a model trained in some rooms to recognise activities in a room it has never seen.

We use the ESP-Fi HAR dataset [14], the most directly comparable prior work. Specifically, it ships not only raw CSI but also a complete model benchmark suite and a published results table, making a like-for-like reproduction possible. Eight participants perform seven daily activities (run, walk, jump, squat, arm-wave, turn, fall) in four indoor environments (corridor, office, meeting room, laboratory), with ten trials per activity and participant, resulting in $8 \times 7 \times 10 = 560$ samples per environment and 2,240 in total. Each sample is a single-antenna CSI amplitude matrix of $950 \text{ packets} \times 52 \text{ subcarriers}$, i.e. a window of roughly 9.5 s at the 100 Hz sampling rate. The preprocessing, and the seven deep models follow the ESP-Fi HAR reference code, and also the three classical baselines were reimplemented to match the paper’s published table. All details are specified in Appendix A.2. In fact, none of the denoising or other preprocessing steps of Chapter 4 are applied, so the measured accuracies reflect the published baseline rather than this thesis’s own pipeline. Performance is reported as classification accuracy and macro-averaged F_1 , which track

closely since every class is collected with the same number of trials. All files and results can be found in the repository.¹

Cross-environment recognition. The experiment that matters for this thesis is the paper’s cross-scene protocol, leave-one-environment-out (LOEO). In that experiment, the model is trained on three environments and tested on the fourth, unseen one. Table 5.1 reports that. ResNet18 leads throughout, at 73.0/74.8/79.1/77.3% on the held-out corridor/office/meeting room/laboratory (76.1% mean accuracy, 75.6% macro- F_1), comparable to or above the paper’s cross-scene ResNet18 (≈ 61 –75%). The only notable divergence from the original study is that EfficientNet-Lite, its cross-scene champion, is here surpassed by ResNet18 by some eight points. The split between model families is sharper across rooms than within one. The convolutional models transfer whereas the recurrent models, the Transformer and the three classical baselines sit between 15 and 35% (some are barely above the 14.3% seven-class chance level). Recognising activities in an unseen room is therefore not a matter of the protocol alone but of the representation the backbone learns.

Table 5.1: LOEO (Leave-One-Environment-Out) results on the ESP-Fi HAR dataset. Train on 3 environments, test on the held-out environment (all 8 subjects). Metrics: Accuracy % / Macro-F1 %. Chance level is 14.3%.

Type	Model	Corridor	Office	Meeting Room	Laboratory
DL	ResNet18	73.04 / 72.28	74.82 / 74.07	79.11 / 78.67	77.32 / 77.28
	EfficientNetLite	67.50 / 67.00	63.39 / 61.26	73.57 / 72.89	68.57 / 68.41
	MobileNetV3	65.71 / 65.41	67.14 / 66.21	69.82 / 69.14	68.57 / 68.69
	CNN	51.07 / 49.40	53.75 / 51.85	60.54 / 58.04	58.21 / 57.68
	LSTM	21.25 / 20.17	25.36 / 22.35	35.18 / 30.01	32.86 / 31.73
	GRU	17.50 / 16.47	25.54 / 23.60	26.25 / 22.62	30.54 / 27.32
	Transformer	18.57 / 16.52	21.79 / 20.54	25.89 / 20.70	26.61 / 25.47
ML	RF	22.86 / 22.07	24.64 / 23.23	28.93 / 26.22	26.96 / 27.12
	SVM	20.54 / 19.42	23.75 / 22.69	25.71 / 20.14	23.57 / 21.16
	LR	20.54 / 20.19	17.50 / 16.37	18.93 / 17.60	15.36 / 13.31

Cross-environment recognition on ESP32-class hardware with a single TX-RX pair is therefore achievable at around 75% accuracy on a seven-class task inside a room the model has never seen, and the figure survives withholding the test subject from training as well as the test room (Appendix A.2), so it is not the subject-identity leakage discussed in Section 2.4.3. What buys it, however, is the large dataset. The in-house acquisition of this thesis is significantly different. This thesis data collection contains three rooms instead of four, a single subject instead of eight, roughly 420 labelled windows per room instead of 560, and windows of 3.5 s (Section 4.8). The question the remaining experiments answer is therefore not whether cross-environment recognition is possible in principle, but what has to take the place of the large dataset. The following preliminary experiments evaluate the

¹<https://github.com/stef4k/ESP-Fi-HAR-experiments>

Table 5.2: Preliminary double-pair experiment: cross-day macro- F_1 of the best model per family, six-class versus the merged three-class task (`sit+stand+lying+empty` $\rightarrow$ `no_action`). Means over the three held-out days; full sweeps and per-class breakdowns in Appendix A.3.

Family	6-class		3-class (merged)	
	Acc	F_1	Acc	F_1
Classical (features + LogReg)	0.558	0.558	0.911	0.911
Deep learning (best CNN)	0.463	0.451	0.956	0.956
Chance level	0.167		0.333	

capabilities of a double pair TX-RX dataset. We investigate which activities two TX-RX pairs can successfully distinguish across different days (Section 5.2) and assess the limits of recognition within a single room (Section 5.3), before addressing cross-environment generalization in Section 5.4.

5.2 Preliminary Experiment: Double-Pair Cross-Day

The first in-house experiment with the double TX-RX architecture of Section 4.2 asked the question of which activities can two co-deployed pairs recognise at all when the same room is observed across days. Six classes (`walk`, `prepare_meal`, and the four static states `sit`, `stand`, `lying` and `empty` which is the unoccupied room) were recorded in a single room over four days, with two orthogonal pairs placed along the walls and every activity performed in NLOS conditions of both links. Crucially, the static classes were recorded as held states (e.g. a person sitting for the whole segment), not as transition movements. Evaluation follows the leave-one-day-out protocol of Section 4.6 with three folds. The first day, which contributed noticeably more recordings than the others, was used exclusively for training in every fold (the fairest use of the surplus, since holding it out would have made one test fold larger than the rest). The full data-collection setup, both model sweeps and all result tables are given in Appendix A.3.

The finding is a clean dichotomy. Despite extensive sweeps in two independent families (a hand-crafted-feature grid of cross-day configurations with logistic regression, forest and boosted-tree, and a deep-learning sweep spanning 1-D/2-D CNNs, recurrent, Transformer and domain-adversarial models) the six-class task plateaus at 0.56 macro- F_1 (classical) and 0.45 (deep). The per-class scores of the best model locate the entire failure. The two motion classes are recognised almost perfectly (`walk` 0.92, `prepare_meal` 0.79 F_1), while the three static states collapse to 0.31-0.41 F_1 (`sit`, `lying`, `empty`) (i.e. essentially chance among themselves) and `stand` reaches only 0.59. Merging only `sit+lying` does not repair it (leading to 0.562 f_1 , but `empty` still fails), but merging all four static states into a single `no_action` label lifts both families to 0.91-0.96 macro- F_1 under the identical protocol (Table 5.2). In other words, in an NLOS double-pair deployment the amplitude stream reliably tells whether someone is moving, but a motionless sitter, stander, liar, and an empty room are mutually indistinguishable across days.

The physical reading is that a static posture leaves no motion-band signature. Its

identity lives only in the static multipath level, which is precisely the component that drifts between days and is removed by any detrending robust enough to survive that drift. This preliminary result directly shaped the cross-environment experiment. Its activity set (Section 4.8.2) records posture transitions (`sit`, `sit_up`, `lying`, `lying_up`) rather than held states, so that every class carries a motion signature. The next section turns to the complementary question of fine-grained recognition under LOS conditions.

5.3 Preliminary Experiment: Fine-Grained Kitchen Activity

The second preliminary experiment retained the double TX-RX architecture of Section 4.2 but inverted almost every other condition of Section 5.2. In detail, the experiment conditions are LOS instead of NLOS, fine-grained hand gestures instead of coarse whole-body states, and a same-room, same-subject, and same-day protocol instead of a cross-day one. In this experiment the six classes consist of two generic activities (`walk`, `stand`) and four fine-grained manipulations performed at a counter (`grab_above`, `grab_counter`, `pour`, `prepare_cafe`). The experiment was recorded in a room repurposed as a mock kitchen and divided into a coffee zone and a tea zone, with one pair viewing each zone in LOS and the orthogonal geometry leaving the other zone off-path. The class set was specified jointly with the LURPA lab partner, whose downstream automaton-based layer consumes the recogniser’s output as an alphabet of atomic movements from which it infers the high-level routine (either prepare coffee or tea). The recogniser is therefore deliberately the front end of a larger stack rather than an end-to-end make-coffee/tea classifier.

The experiment has two data regimes, that showcase an important finding for real-world deployment. Training uses 198 short, controlled, single-activity clips (6×33 segments of ≈ 2.9 s, half performed in each zone). Evaluation uses both these clips, under the leave-one-clip-out (LOO) protocol of Section 4.6, and separately six continuous recordings of real coffee- and tea-making routines, in which the activities are chained together without breaks. All six activities are chained in the coffee routines, while four are in the tea routines (no `stand` and `prepare_cafe`). The continuous routines were filmed with a synchronised camera and manually transcribed second by second into (`start`, `end`, `class`) intervals, then scored with a sliding window (2.9 s length, 0.5 s stride) against that ground truth. The full sensing setup, activity set, both classifier sweeps and the complete deployed-pipeline specification are given in Appendix A.4.

The controlled-to-real collapse. Two independent classifier families were developed: a classical pipeline of hand-crafted statistical and spectral features with an ℓ_1 -ranked feature selection swept over classifiers, and a deep-learning sweep of one-dimensional CNN, residual and recurrent backbones on the $(T, 104)$ windows. The deep learning approach reached a max $\approx 94\%$ LOO accuracy on the controlled clips while the hand-crafted features reached $\approx 92.5\%$. The deployed pipeline (`gain_denoise_bandpass` | `ResNet1D_znorm` | `device_robust` described in Appendix A.4.4) reaches 0.909 accuracy and 0.909 macro- F_1 under LOO, with all six classes reaching above 0.87 F_1 . The deployed pipeline trades a small amount of LOO accuracy and macro- F_1 for robustness. Among the top-ranked configurations on the real continuous recordings it was preferred because its errors are small temporal shifts rather than gross mislabels. Those errors are preferred for a model

Table 5.3: Preliminary kitchen experiment: the same deployed pipeline scored on the controlled clips and on the continuous real routines. All details are in Appendix A.4.

Evaluation regime	Acc	macro- F_1
Controlled single-activity clips (LOO, 198 clips)	0.909	0.909
Continuous real routines (350 windows, 6 sessions)	0.589	0.445
<u>Per-class F_1 on the real routines</u>		
Generic: stand / walk	0.92	0.65
Fine-grained: pour / prepare_cafe	0.53	0.44
Fine-grained: grab_counter / grab_above	0.13	0.00

that is deployed continuously.

None of this survives contact with the continuous recordings (Table 5.3). Over the six continuous routines (350 scored windows) the same pipeline reaches only 58.9% window accuracy and 0.445 macro- F_1 . The drop is carried almost entirely by the fine-grained classes since the two generic activities hold up (**stand** $F_1 = 0.92$, **walk** 0.65). On the other hand, the four counter actions fall to 0.53 (**pour**), 0.44 (**prepare_cafe**), 0.13 (**grab_counter**) and 0.00 (**grab_above**).

Implications for lab-only evaluation. The methodological lesson is the reason this experiment is reported at all. High controlled-data cross-validation is not predictive of real-world behaviour, even under same-day, same-subject, single-room collection. Crucially, this gap is not the cross-environment problem that benchmarks already probe. It is a different and largely unmeasured blind spot. Such benchmarks always evaluate on pre-segmented, isolated single-activity clips, never on a continuous stream. A model can therefore generalise across subjects and rooms on clean isolated windows and still be unusable the moment it must run continuously over chained actions, transitions, ambient periods and the boundary windows that straddle two activities. None of those appear in a segmented benchmark. Reporting only segmented accuracy, in-distribution or cross-environment, thus risks substantially overstating deployability. Therefore, an honest assessment of CSI-HAR for real use requires held-out, naturally executed, continuous recordings of the kind used here, even when they are considerably harder to obtain and label.

Two caveats bound the reading. First, the pipeline was selected with these labelled recordings in the loop (Appendix A.4.6), so the real-routine scores are an optimistic, in-sample estimate. The LOO number is the only figure not selected on the deployment recordings, and the real-routine numbers are treated qualitatively. Second, and precisely because the estimate is optimistic, the collapse of the fine-grained classes is a robust finding. They fail even under favourable conditions. In fact, the four counter actions share almost-identical arm trajectories over the same two LOS regions, so their amplitude signatures overlap heavily. Combining this signature overlap with the meter-level bandwidth resolution limit of Section 3.2.6 and the very few clips per class, two co-deployed pairs are not sufficient to separate those actions in continuous real routines.

Fine-grained gesture recognition is therefore left aside. The remaining experiment targets coarse, whole-body activities that would be typically performed by an elderly

person, while knowing that evaluation should also contain continuous routines. The next section turns to the distribution shift that matters most for deployment: the change of room.

5.4 Cross-Environment Experiment Generalization

This section attacks the distribution shift that matters most for deployment (the room itself) using the in-house multi-room acquisition of Section 4.8. Precisely, six daily activities (`walk`, `prepare_meal`, `sit`, `sit_up`, `lying`, `lying_up`) recorded with the same double-pair, NLOS setup in three rooms (`mc208`, `sb207`, `mf207`), evaluated under strict leave-one-room-out (LORO) cross-validation. Throughout, performance is the mean macro- F_1 over the three held-out rooms. Alongside the six-class task we also report a coarser four-class grouping that merges the two descending transitions (`sit+lying` $\rightarrow$ `sit_lying`) and the two ascending ones (`sit_up+lying_up` $\rightarrow$ `get_up`). For an elderly-care monitor this grouping still captures the clinically relevant transfer events while removing the hardest distinction.

Unlike the kitchen study, which validated a single fixed pipeline, this experiment was an iterative search. The presentation below follows its actual chronology, because each stage’s failure motivated the next design. A broad two-family grid search (Section 5.4.1) plateaued well below a usable level. Then replacing the stacked dual-pair input with a siamese shared trunk produced the first substantial jump (Section 5.4.2). Also, a normalisation study explained which confusions no amplitude cue can fix (Section 5.4.3). A refined training recipe and a component ablation quantified what actually matters (Section 5.4.4). A timestamp-grid resampling added the final increment (Section 5.4.5). And a closing study isolated the value of the second pair against every single-pair deployment option (Section 5.4.6). All implementation details and explanations can be found in the Appendix A.5.1

5.4.1 Stage 1: broad pipeline grid search

The first stage transplanted the two model families of Section 4.7 onto the cross-environment data and swept their pipeline hyper-parameters exhaustively. The classical family reused the hand-crafted feature bank of Appendix A.4.2 (statistical, spectral, auto-correlation and cross-receiver descriptors) followed by per-fold feature selection and standard classifiers (SVM-RBF, logistic regression, random forest). The deep family covered some of the one-dimensional convolutional models of the referenced section (ResNet1D, ResNet1DStats, TCN1D) on the stacked $T \times 104$ dual-pair window. Both grids crossed pre-processing (gain correction, Hampel denoising, several Butterworth band-pass limits, and a raw variant), window duration ($W \in \{2.5, 3.0, 3.5\}$ s) and an optional empty-room calibration reference. The deep grid additionally swept the per-window normalisation (`znorm`, `pc1_znorm`, `diff_znorm`) and augmentation, and the classical grid the packet-rate handling and the standardisation scope of the feature matrix. Because the full cross-product is computationally infeasible on the available CPU-only hardware, each family’s grid harness was executed twice: a screening pass at $W = 3.0$ s that pruned dominated options, followed by a re-configured, focused second stage. Companion single-pair grids repeated

Table 5.4: Stage-1 grid search: best configuration per family under LORO (mean macro- F_1 over the three held-out rooms, single model, no test-time adaptation).

Family	Best configuration	Mean F_1	Acc.	Worst room
Deep (dual)	gain correction + denoising + band-pass 0.3–8 Hz diff-znorm $W=3.5$ s TCN1D	0.579	0.592	0.530 (sb207)
Features (dual)	gain correction per-env std. ℓ_1 top-300 RF	0.558	0.567	0.532 (sb207)
Deep (single, CB00)	gain correction + denoising + band-pass 0.5–10 Hz znorm $W=3.5$ s TCN1D	0.541	0.547	0.507 (sb207)
Features (single, CB00)	gain correction per-env std. ℓ_1 top-300 RF	0.539	0.548	0.534 (sb207)

the sweep on each link alone.

Table 5.4 summarises the best configurations. Three axis-level verdicts were consistent across both families. First, removing the environment-specific signal level is essential. For the classical pipeline, per-environment feature standardisation was the single largest improvement (the room the un-standardised classical pipeline handled worst, mc208, rose from ≈ 0.30 to ≈ 0.53 macro- F_1), and for the deep models the band-pass filter played the analogous role. Second, the longer 3.5 s window dominated 3.0 and 2.5 s almost everywhere, and the raw pipeline (no gain correction, denoising or band-pass) was one of the worst configurations in almost every cell. Third, the empty-room calibration reference underperformed in every configuration and was dropped. Among architectures the dilated TCN1D was consistently strongest.

The outcome, however, was a plateau. The best deep configuration (gain_correction+denoising+bandpass_0.3_8 | diff_znorm | W=3.5 | TCN1D) reached 0.579 mean macro- F_1 (rooms 0.607/0.530/0.600) and the best classical configuration (gain | perenv_l1_top300 | rf) 0.558. This is far above the six-class chance level of 0.167, but far below deployability, and hundreds of additional grid cells failed to move either number. The grid had saturated what stacked-input models could extract. Progress required changing the model structure, not its hyper-parameters.

5.4.2 Self-supervised pretraining and the siamese shared trunk

The first structural change tested was self-supervised pretraining (Section 2.5), motivated by the small labelled pool (≈ 420 windows per room). The ESP-Fi dataset of Section A.2.1 (single-link, 52 subcarriers, ≈ 100 Hz, i.e. format-compatible with one thesis link after removal of the zero subcarriers) served as a large pretraining set, complemented by unlabelled double-pair recordings from earlier experiments. A first iteration pretrained a transformer encoder with three SSL methods (MAE, SimCLR, BYOL) and fine-tuned it under LORO. The result was instructive rather than useful: masked-autoencoder pretraining lifted the transformer by ≈ 18 points over the same architecture trained from scratch (0.495 vs. 0.316 ensemble macro- F_1), confirming that the pretext task learns real signal structure, yet even the pretrained transformer remained below a TCN trained from scratch. The insight is

Table 5.5: Self-supervised pretraining vs. architecture under LORO (macro- F_1 : mean of 3 seeds / 3-seed probability ensemble). All rows share the pretraining study’s own fixed recipe ($W=3.0$ s, band-pass 0.5–10 Hz, **znorm**, resampling to $L=128$, light jitter-and-scale augmentation), deliberately untuned relative to the grid winner of Table 5.4: the rows are comparable to one another, not to the 0.579 of Section 5.4.1.

Model	Mean F_1	Ensemble F_1
TCN, stacked 104-ch input (scratch)	0.532	0.574
Siamese shared trunk (scratch)	0.595	0.633
Siamese shared trunk + masked-recon pretraining	0.603	0.639
Pretrained transformer (MAE fine-tune)	0.485	0.495
Transformer (scratch)	0.303	0.316

that on a few hundred windows, the inductive bias of the backbone outweighs the benefit of pretraining.

The second iteration therefore moved the pretraining onto the TCN trunk itself, using a masked-reconstruction pretext (random temporal spans zeroed, L_2 reconstruction on the masked steps only), and simultaneously replaced the stacked 104-channel input with the siamese shared-trunk architecture of Section 4.7.3. Each link’s $T \times 52$ window is encoded by the same trunk and the two descriptors are fused afterwards. Table 5.5 reports the outcome. The pretrained siamese model reached 0.639 ensemble macro- F_1 , which is the first substantial improvement over the grid plateau, and remarkably uniform across rooms (0.629/0.629/0.659). The decisive comparison, however, is between the two siamese rows: the same architecture trained from scratch scored 0.633, while the stacked-input TCN under the identical training recipe scored 0.574. The siamese shared trunk, not the pretraining, is the real win ($\approx +6$ points). In detail, weight sharing halves the parameters the small training pool must support and forces link-agnostic motion features, whereas the SSL contribution after full fine-tuning is $\approx +1$ point, which is within seed noise on three folds. As a result, we considered pretraining a non-essential component for the pipeline. More details can be found in the Appendix A.5.5.

5.4.3 Diagnosing the residual confusions: what amplitude cannot encode

With the siamese model established, a dedicated study asked where the remaining third of the errors live and whether they are fixable within an amplitude-only pipeline. The error mass is concentrated almost entirely in the posture-transition quartet: the two continuous activities are essentially solved (**walk** and **prepare_meal** around 0.83–0.87 F_1), while the dominant confusions are **sit** $\leftrightarrow$ **lying** and **sit_up** $\leftrightarrow$ **lying_up**. The model reliably detects that a transition occurred and its direction, but not whether its endpoint is a chair or the floor.

The physically plausible cue separating these pairs is magnitude. Specifically, lying onto the floor sweeps a larger vertical extent than sitting onto a chair. Two hypotheses were tested. First, that the per-window z-normalisation (Section 4.7.4) erases this magnitude cue: replacing it with per-room or global input normalisation, however, degraded macro- F_1

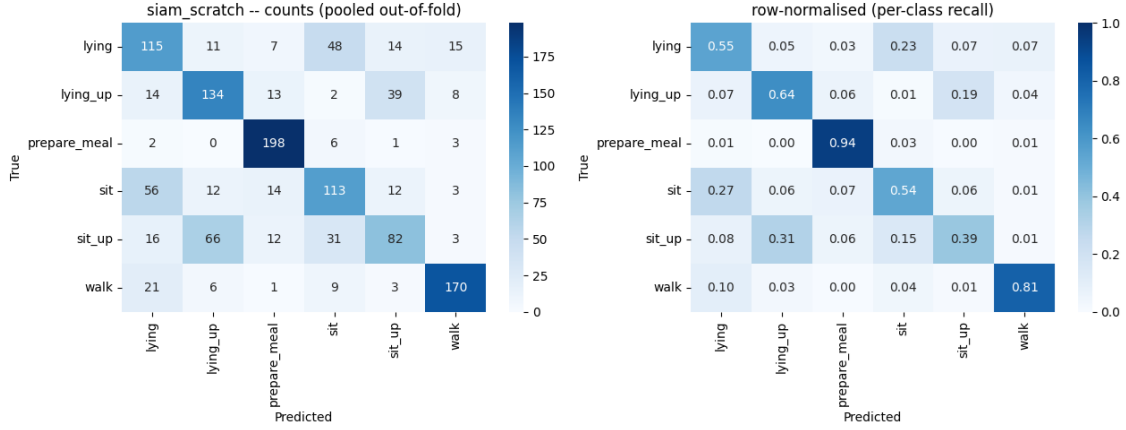


Figure 5.1: Pooled and row normalised leave-one-room-out confusion matrix of the shared-trunk scratch model (six classes). Errors concentrate in the posture-transition pairs **sit**↔**lying** and **sit_up**↔**lying_up**.

up to 5 points. In other words, re-scaling the input perturbs the learned features more than the restored magnitude helps. Second, the cue can be re-injected without touching the trunk input: late-fusing a per-channel log-std (magnitude) descriptor into the classification head left the headline score statistically unchanged (+0.006, within seed noise) but reduced the pooled **sit**↔**lying** confusion by $\approx 21\%$ ($0.248 \rightarrow 0.195$) and raised **lying_up** recall from 0.64 to ≈ 0.74 . Richer descriptor variants (multiple statistics, hierarchical coarse-to-fine heads, low-frequency posture log-ratios extracted before the band-pass) all plateaued at the same level or regressed through overfitting on the small training pool.

The convergent conclusion is a negative result worth stating explicitly: every magnitude or energy descriptor saturates at the same residual confusion, because the 0.5–10 Hz band-pass that makes the pipeline transfer across rooms also deletes the static, near-DC (i.e., quasi-constant, zero-frequency) amplitude level in which endpoint height lives, and what magnitude information survives does not separate the ascending pair at all. Distinguishing chair-height from floor-height transitions from band-passed CSI amplitude alone appears to be information-limited. The natural levers (phase-based Doppler features or additional training rooms) lie outside this amplitude pipeline. This finding directly motivates the four-class grouping used alongside the six-class task in the remainder of the section.

5.4.4 Recipe refinement and component ablation

The next stage optimised the training recipe on top of the siamese trunk, adding several components and tricks. Three additions proved consistently positive. First, an augmentation suite combining SpecAugment-style masking (contiguous subcarrier spans and time spans, applied independently per link) with mixup. In this experiment, mixup is restricted to same-class pairs: a window is blended only with another window of the same activity, producing plausible intra-class variation without the label ambiguity of standard cross-class mixup. Under the siamese trunk, the mixing partner may come from the other training room, blending two rooms’ views of the same class. Second, Adaptive Batch Nor-

Table 5.6: Component ablation of the refined recipe under LORO (AdaBN + 3-seed ensemble macro- F_1 , mean over the three folds). Δ is the change caused by removing the component from the full recipe.

Configuration	6-class F_1	Δ	4-class F_1	Δ
Full recipe (band-pass, 3.5s window, siamese shared trunk, subcarrier/time masks, same-class mixup, SSL pretraining, base augmentation)	0.740	–	0.881	–
– no band-pass	0.599	–14.2	0.817	–6.4
– 3.0 s window (instead of 3.5s)	0.681	–6.0	0.852	–2.9
– no adaptive batch normalization	0.689	–5.1	0.871	–1.1
– no siamese shared trunk	0.702	–3.8	0.868	–1.3
– no subcarrier/time masks	0.716	–2.4	0.866	–1.5
– no same-class mixup	0.731	–0.9	0.878	–0.4
– no SSL pretraining	0.747	+0.7	0.872	–0.9
– no base augmentation	0.755	+1.4	0.879	–0.2

malisation (Section 4.7.6), which was positive for almost every configuration tested and is kept always-on. Third, widening the analysis window to $W = 3.5$ s, the same verdict the stage-1 grid had already reached, worth $\approx +6$ points on this recipe, which is the largest single jump of the refinement phase.

To separate the contributions, a leave-one-component-out ablation retrained the full recipe with each ingredient individually removed (Table 5.6). The hierarchy is unambiguous: the band-pass filter dominates everything (–14.2 points when removed), followed by the 3.5 s window (–6.0), AdaBN (–5.1), the siamese trunk (–3.8) and the masking augmentation (–2.4). Same-class mixup is a small but positive regulariser (–0.9). Two components turned out to be negative: generic jitter-and-scale base augmentation (+1.4 when removed, meaning it is a no-op at best after z-normalisation and batch-norm) and, notably, the SSL pretraining itself (+0.7 when removed). The pretraining that had contributed +18 points to a weak transformer baseline adds nothing to the full recipe: strong augmentation, AdaBN and the siamese prior extract the same invariances from the labelled data directly. Both negative components were dropped, fixing the recipe for the remaining experiments.

5.4.5 Timestamp-grid resampling

With the recipe fixed, the final pipeline stage examined was the temporal sampling of the window, which is the step that converts a jittery ≈ 100 Hz packet stream into a fixed-length tensor (Section 4.5). Table 5.7 sweeps the target length L , the interpolation basis, and two no-resampling controls.

The three bases differ in what they assume about the packet stream. The linear index grid ignores timestamps entirely and linearly interpolates the packets onto L evenly spaced sample indices, implicitly treating the stream as if it had arrived at a perfectly constant rate. The timestamp grid instead builds a uniform wall-clock axis of L points between the first and last packet of the window and linearly interpolates each subcarrier

Table 5.7: Effect of the temporal sampling strategy (recipe otherwise fixed; AdaBN + 3-seed ensemble macro- F_1 , mean over LORO folds).

Sampling	6-class F_1	4-class F_1
Timestamp grid, $L=160$	0.761	0.882
FFT resample, $L=160$	0.749	0.879
Linear index grid, $L=160$	0.744	0.876
Timestamp grid, $L=128$	0.743	0.891
Timestamp grid, $L=96$	0.740	0.882
Linear index grid, $L=128$	0.735	0.887
Linear index grid, $L=96$	0.732	0.878
Band-pass after resampling, $L=160$	0.708	0.865
Linear index grid, $L=320$	0.696	0.857
No resampling (zero-pad)	0.663	0.858
No resampling (per-window band-pass)	0.637	0.858

onto it, so a packet’s true arrival time decides where its amplitude lands. FFT resampling also works on the index axis, but changes the length in the frequency domain rather than by interpolation: it takes the FFT of the window, truncates (or zero-pads) the spectrum to the target length and transforms back, which is equivalent to band-limited sinc interpolation under the assumption that the window is uniformly sampled and periodic. It is therefore smoother than linear interpolation and acts as an implicit anti-aliasing filter when downsampling, but it cannot correct timing jitter and its periodicity assumption introduces ringing at the window edges.

Resampling itself is essential. The zero-pad/truncate variants used elsewhere in this thesis lose 8–11 points here, and $L = 320$ (effectively no downsampling) is the worst re-sampled option. The sweet spot lies at $L \approx 128$ – 160 (≈ 37 – 46 Hz), where the convolutional receptive field spans more of the window at the same depth. Interpolating onto a uniform wall-clock grid built from the true packet timestamps, rather than onto an index grid, gave the best six-class result (0.761 ensemble macro- F_1 with the best single fold recorded up to this point, `mf207` at 0.831). This removes the packet-rate jitter of the ESP32 stream and, because both RX are interpolated onto the same grid, exactly time-aligns the two links (the property that the synchronised-start acquisition makes possible). Finally, filtering placement matters: applying the band-pass per window or after resampling costs 2.5–3.5 points relative to recording-level filtering, confirming that filter edge effects, not the $f_s \approx 100$ Hz approximation, are the failure mode the recording-level design of Section 4.4 avoids. All details can be found in the Appendix A.5.7.

5.4.6 Single-pair versus double-pair deployment

The final study answers the question this thesis set out to ask: with the best pipeline identified, how much is the second pair actually worth, and what is the best way to consume it? The finished recipe (shared single-link trunk, band-pass 0.5–10 Hz, $W=3.5$ s, timestamp-grid $L=160$, masks and same-class mixup, AdaBN, 3-seed ensemble) was evaluated under every deployment configuration regarding the pair links. In detail, those are

Table 5.8: Single-pair vs. double-pair deployment under LORO with the final recipe (Timestamp resampling $L=160$, AdaBN + 3-seed ensemble macro- F_1 , mean over folds). Link A = AC84 (RX at 95 cm), link B = CB00 (RX at 55 cm) as seen in Table 4.1. To ensure class balance for the 4-class classification task, we applied shorter window strides to the `prepare_meal` and `walk` classes to match the sample sizes of the `getup` and `sit_lying` activities.

Training	Test-time input	6-class F_1	4-class F_1
Shared trunk (both links)	Both links, prob. average	0.766	0.918
Feature-concat dual	Both links, concatenated	0.761	0.905
Shared trunk (both links)	Link B only	0.724	0.888
Single link B	Link B only	0.686	0.867
Shared trunk (both links)	Link A only	0.677	0.869
Single link A	Link A only	0.607	0.827

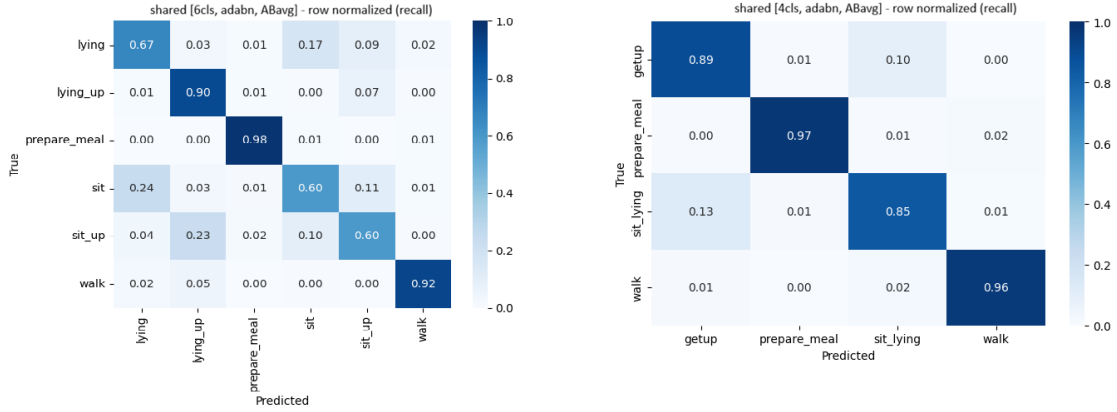


Figure 5.2: Final model row-normalised confusion matrices for 6-class (on left) and 4-class (on right) granularities. Classifications reports can be found in the appendix tables A.33 and A.34.

training and testing on a single link (single-same, per link), training the shared trunk on both links pooled but testing on one link’s probabilities (shared, single-link), averaging the two links’ predicted probabilities at inference (shared, late fusion) and the feature-concatenation dual model as the anchor. Table 5.8 reports the results and Figure 5.2 shows the confusion structure of the winning configuration. These scores, like every cross-environment figure in this chapter, come from configurations ranked on the held-out folds themselves, so they are best-of-search estimates rather than estimates from an untouched test set.

Five findings follow. (i) **Late fusion is the best dual-pair model.** Probability-averaging a single shared trunk (0.766 / 0.918 f_1) beats the feature-concatenation dual on both granularities, winning the four-class task in all three rooms. The dual advantage is therefore “two views of the same activity”, not any learned cross-link feature interaction. The late-fusion model is simpler, having one 52-channel trunk, deployable unchanged on

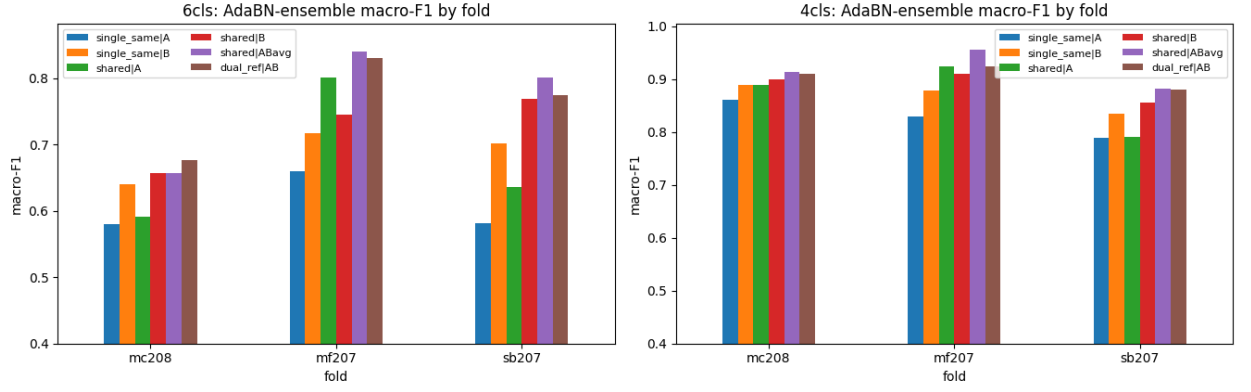


Figure 5.3: Per-fold macro- F_1 (AdaBN + 3-seed ensemble) of the deployment configurations of Table 5.8, for the six-class (left) and four-class (right) tasks.

one pair or two. **(ii) The second pair is worth 4-16 points of f1.** Relative to the best achievable single-pair deployment, adding the second link buys +4.2 points of f1 on six classes and +3.0 on four classes. Relative to a single-pair system trained conventionally on its own link only, the gap widens to +8-+16 points depending on the link on the six class grouping. This is the quantitative answer to the thesis question. The second link’s information is real and measurable. Now whether it is worth a second live device is a deployment question taken up in (v). Also, the gain concentrates exactly where single-link physics predicts: in the fold/link combinations where one geometry is blind (Section 3.2.5). **(iii) Link geometry matters more than link count.** Link B (CB00) outperforms link A by 5–8 points of average f1 in the six-class grouping. A well-placed single pair approaches a badly-placed dual system, echoing the dilution-effect literature of Section 2.4.7. **(iv) Even a single-pair deployment should be trained on both geometries.** Testing link B on the pooled shared trunk beats the same link trained alone by +3.8 points (six-class). The pooled links act as free domain augmentation for one another, so a single training run covers every deployment scenario. **(v) The second pair’s largest dividend is data collection, not inference.** Because every repetition performed in front of the dual setup is captured simultaneously by both links, each session yields two geometrically distinct training windows per activity. The pooled dataset that trains the shared trunk doubles at no extra cost to the participant, who needs only half the repetitions to produce the same amount of training data. In our view this is the decisive argument for the double pair, and it suggests an asymmetric deployment recipe: collecting with both pairs (doubling the data and covering both geometries, per finding iv), but inferring with one well-placed pair. In detail, the +3-+4-point f1 gain of fusing a second link at test time does not justify doubling the hardware, synchronisation, and compute overhead of the live system.

A closing control experiment re-examined the Automatic Gain Control (AGC) gain correction that was essential in the kitchen experiment. Added to this pipeline it gave no consistent benefit: for the deployed late-fusion configuration the LORO-averaged macro- F_1 moves by -0.009 at six classes and $+0.004$ at four, both within the seed-to-seed spread. A likely reason is the per-window z-normalisation, which already cancels any slowly varying

gain and leaves only its in-band component. The correction is therefore not adopted here.

In summary, the cross-environment experiment closes the arc of this thesis: starting from a grid-search plateau of 0.579 macro- F_1 , the combination of a siamese shared trunk, an aggressive but targeted augmentation recipe, label-free test-time adaptation and timestamp-grid resampling reaches 0.766 f1 on six classes and 0.918 f1 on the four-class granularity in rooms the model has never seen (with the residual six-class errors mostly confined to the chair-versus-floor distinction that amplitude-only CSI demonstrably cannot resolve).

5.4.7 Extending the label set with a `no_action` class

Every model reported so far is a forced-choice classifier over activities. If the classifier is presented with a window in which nothing happens, it must still emit one of the six action labels. However, a deployed real-time system needs an additional behaviour. In detail, most of its operating time is idle, and thus reporting an action event that did not occur is a failure mode in elderly care. The final experiment therefore extends the label set with a `no_action` class, turning the six-class task into a seven-class one and the merged four-class grouping into a five-class one.

No additional acquisition was required. Each recording session of the cross-environment experiment already contains a short capture of the empty room with nobody performing an activity (4s and 10s segments, 39 segments and ≈ 246 s in total, distributed over the three rooms), which the earlier stages used only as a calibration reference and then discarded when that reference proved harmful (Section 5.4.1). These segments are relabelled `no_action` rather than `empty`, because what an amplitude-only pipeline can certify is the absence of motion inside the room, not the absence of a person in the room (as seen in the Section 5.2). They pass through the identical pipeline (Hampel denoising, recording-level band-pass 0.5–10 Hz, $W=3.5$ s windows on the timestamp grid with $L=160$) with a single adjustment. Because the empty room recordings are fewer, the sliding stride is reduced to 0.5s (seven-class) and 0.25s (five-class) so that the `no_action` pool exceeds the per-class balance target in every room. The class is then subsampled down to the same count as every other class (≈ 70 and ≈ 140 windows per class respectively), so the class is never up-weighted. As a result, both the training and the held-out test sets stay balanced, and accuracy continues to track macro- F_1 . Models, LORO folds and the deployment configurations of the previous subsection are otherwise unchanged.

Table 5.9 reports the outcome. `no_action` is recognised reliably and is in fact the best-recognised class of the seven-class set in four of the five configurations, and level with `walk` in the fifth. In the best configuration (shared trunk, late fusion) it reaches at least 0.946 per-class F_1 at both granularities, well above the seven-class macro-average, and it never falls below 0.869 in any single-pair deployment. Crucially, the added class costs the original six little. Restricting the macro- F_1 of the seven-class model to the six original labels gives 0.755 (Table A.37) against 0.773 for the six-class model (Figure 5.2), obtained from their pooled confusion matrix. Both are pooled over folds, in line with Section 4.6, and the fold mean of the latter is the 0.766 quoted below. The loss is therefore under two points. Also, the reverse observation deserves the same honesty. The headline seven-class figure (0.777) sits above the six-class one (0.766) only because an easy class has been

Table 5.9: Adding a `no_action` class: LORO results with the final recipe (AdaBN + 3-seed ensemble). The two macro- F_1 columns are means over the three held-out rooms. The last column is the per-class F_1 of `no_action` itself, computed on the predictions pooled over folds, at the seven-/five-class granularity. Link A = AC84, link B = CB00.

Training	Test-time input	Macro- F_1 (mean over folds)		no_action F_1 (pooled)
		7-class	5-class	7-class / 5-class
Shared trunk (both links)	Both links, prob. average	0.777	0.917	0.948 / 0.946
Shared trunk (both links)	Link B only	0.734	0.881	0.905 / 0.910
Single link B	Link B only	0.711	0.862	0.920 / 0.923
Shared trunk (both links)	Link A only	0.692	0.848	0.869 / 0.873
Single link A	Link A only	0.627	0.830	0.891 / 0.901

added to the average, and should not be read as an improvement of the recogniser.

Two insights of Section 5.4.6 survive the extension unchanged: late fusion of the shared trunk remains the best configuration, and link B remains ahead of link A in every LORO mean. The practical conclusion is that the idle state can be added to the label set essentially for free, giving the system the "nothing is happening" output a continuous monitor requires. Per-class results for every deployment configuration, and the pooled confusion matrices of the two best ones, are given in Appendix A.5.8.

5.4.8 Real-time evaluation: live deployment on cued sessions

Every result so far was produced offline, from recordings already captured and segmented. A deployed monitor instead reads a raw CSI stream from boards installed minutes earlier in a room unseen during training, and must adapt to that room before its first prediction. The recipe of Section 5.4.6 and extension 5.4.7 was therefore implemented as a stand-alone acquisition-and-inference program (Appendix A.5.9) and run in two new rooms: a university room in which the double-pair geometry of Table 4.1 could be reproduced, and a furnished living room in which only the single-pair link B (with shared trunk) geometry is used. The live room being the held-out domain, the model is trained on all three recorded rooms pooled. The rest of the pipeline of Table A.31 stays unchanged.

AdaBN at install time. The test-time adaptation of Section 4.7.6 becomes an install-time step. The operator performs a short cued sweep, which is one beep-cued 3.5 s window per class. As a result, each network's batch-norm statistics are re-estimated per link and per seed, while weights stay frozen and no label is consulted. It takes less than one minute, and what matters is the pool's composition (at least one sample of each class) rather than its size or duration. Experiments using free-form calibration recordings of longer durations for instance 60 to 180 sec straight, consistently gave worse results for the 7-class problem granularity.

Live results. Table 5.10 reports both sessions, scored over the operator-labelled live windows. The double-pair session recognises seven classes at 0.8 accuracy / 0.801 macro- F_1 , while the single-pair session at 0.783 accuracy and 0.779 macro- F_1 . Both sit a few

Table 5.10: Live cued sessions with the final recipe, over the operator-labelled windows of one session each. The 5-class rows collapse the same predictions onto the coarse granularity post hoc. The last column is the offline LORO mean of Table 5.9 for comparison.

Session	Deployment	Task	n	Acc	macro- F_1	LORO F_1
10 August, room e212	dual, prob. average	7-class	35	0.8	0.801	0.777
same windows, merged	dual, prob. average	5-class	35	0.943	0.948	0.917
23 July, living room	single, shared trunk with link B	7-class	60	0.783	0.779	0.734
same windows, merged	single, shared trunk with link B	5-class	60	0.883	0.899	0.881

points above the offline LORO estimate for the corresponding test-time configuration, but the comparison is loose in two ways: the live model is trained on all three rooms rather than two, and with 35 and 60 labelled windows the sessions cannot resolve differences of a few points. The defensible reading is therefore that the live scores are consistent with the offline estimate, so moving to a raw stream costs little beyond what LORO already measured.

The two live evaluation sessions are not comparable. Reading across its two blocks, Table 5.10 appears to settle the single- against double-pair question: the double-pair session appears to score slightly above the single-pair one at both granularities. That is not a conclusion this section can support. The two sessions differ in room, in day, in class balance and in size (35 against 60 windows), so their scores cannot be set against one another. That comparison was made under controlled conditions offline, in Section 5.4.6. Each session may only be read against its own offline LORO estimate in the last column, and both sit a few points above it. This result is expected. The model in this section was trained on all three rooms, whereas each LORO model was trained on only two. What the pair of sessions does establish is narrower and, for a deployment, more useful: neither test-time configuration collapses when it has to adapt to an unseen room from a raw stream, and the install-time AdaBN sweep is what makes that possible. Appendix A.5.9 quantifies how far each session falls when the training batch-norm statistics are kept instead.

Errors and scope. The live errors are the offline ones: six of the seven double-pair errors fall inside the posture quartet of Section 5.4.3, and only two survive the merge to five classes. The single-pair session repeats the pattern, adding a new one: a delayed-start artefact of four `walk` windows read as a lying transition. Both confusion matrices are given in the Figures A.17 and A.18. Continuous recordings are in any case still untested, since the beep sounds tell the model when each activity begins and ends, and each 3.5 s window captures exactly one action-class. The next section explores longer continuous recordings evaluation.

5.4.9 Continuous recordings: evaluation without segment boundaries

The live sessions of Section 5.4.8 still granted the model one simplification: every classified window contained exactly one activity, exactly as every training window does. A deployed monitor receives no such cue. Specifically, activities follow one another, idle periods are of

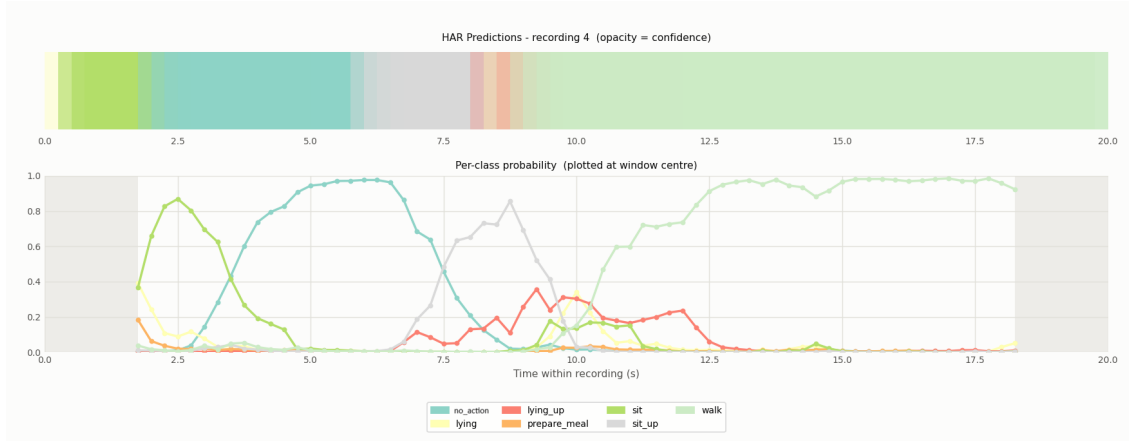


Figure 5.4: Continuous recording 4 (`lying` → `no_action` → `lying_up` → `walk`): predicted labels with confidence as opacity, over the per-class probabilities plotted at each window centre. Lying is slightly confused with sitting in the beginning and `lying_up` is confused with `sit_up` after the idle state. The `no_action` and `walk` activities are identified perfectly.

unknown length, and the sliding window regularly straddles two activities at once. This is the regime in which the kitchen activity recogniser of Section 5.3 collapsed, from 0.909 macro- F_1 on pre-segmented clips to 0.445 on continuous routines. The final experiment of this chapter therefore subjects the cross-environment model to the same test, as a stress test of the finished recogniser rather than as a new measurement.

Setup and what is measured. Nine recordings of 20s of freely performed activity were captured with an adapted continuous version of the script of Section 5.4.8, and classified afterwards by sliding the $W = 3.5$ s window at a 0.25s stride. The deployment is the double pair with late probability fusion over the seven classes of Section 5.4.7, preceded by the same install-time cued AdaBN. The pairs follow the configuration of Table 4.1, and everything else in Table A.31 stays unchanged. This section is not a scored evaluation but a test that the pipeline does not collapse in continuous real-time recordings. Appendix A.5.10 specifies the mode, lists the nine sequences in Table A.39 and gives the remaining seven continuous recordings.

The model does not collapse, and duration is what makes a class easy. Nothing resembling the kitchen collapse occurs. Across the nine recordings the prediction stream follows the performed sequence in order and spends most of its time on the correct class rather than defaulting to one dominant label. The clearest structural pattern is that confidence tracks persistence. Sustained activities (like `walk`, `prepare_meal` and the `no_action` state) settle into long, near-saturated plateaus and stay settled. Brief posture transitions are inherently harder. For example, a 4s sit-down seen through a 3.5s window occupies a handful of consecutive windows, so one wrong window is a large share of that activity’s evidence, whereas the same window is negligible inside a ten-second walk activity. The offline scores hide this, because there every window is judged on its own and counts the same.

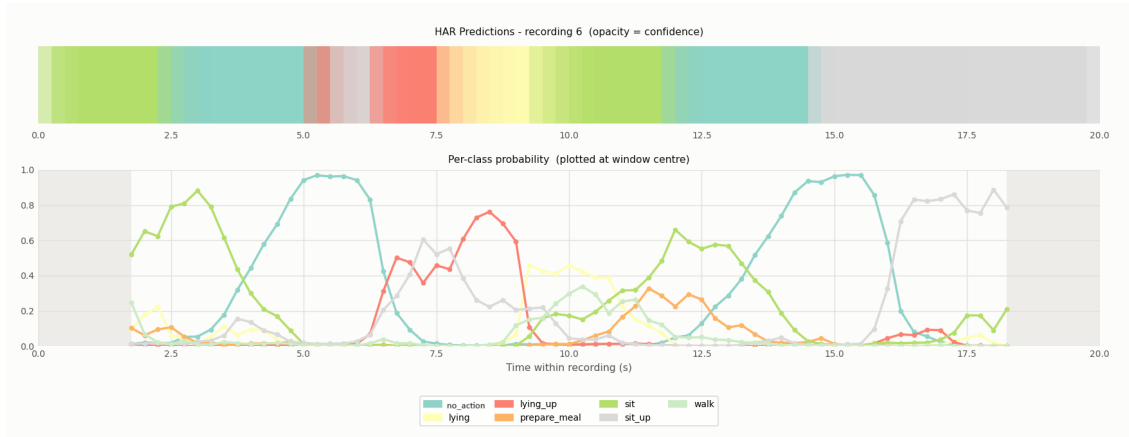


Figure 5.5: Continuous recording 6 (lying $\rightarrow$ no_action $\rightarrow$ lying_up $\rightarrow$ sit $\rightarrow$ no_action $\rightarrow$ sit_up): the hardest of the nine, chaining six phases and passing through the idle state twice. The stream still follows the sequence, except for lying which is misclassified as sit.

Which classes hold, and which confusion survives. prepare_meal, walk, sit and lying_up are recognised most reliably in the stream, and the no_action idle state is recognised whenever it is held. This confirms, in continuous operation, what Section 5.4.7 measured on windows, that no_action behaves as a motion detector rather than an occupancy detector. The one substantive error inherited unchanged is the sit-lying confusion. Descents onto the floor are still liable to be read as descents onto the chair. This is consistent with the diagnosis of Section 5.4.3 so continuity neither worsens nor repairs it.

Boundary windows are the new failure mode. The genuinely new errors belong to the boundaries. A window straddling two activities, or catching only the onset or tail of one, is frequently labelled with a third class (most often sit, lying or sit_up) so that a correct plateau is separated from the next by a short run of a label that was never performed. Transitions into and out of the idle state are the most systematic instance as seen in Figure 5.4 and 5.5. The cause is the training dataset rather than the model. Every training window lies wholly inside one labelled segment, and the no_action windows in particular were cut from static empty-room captures, so no window shows an activity decaying into stillness and none shows two activities meeting. The model has never been given the input it is being asked to classify, and answers with the nearest class it knows. Because these runs are short and comparatively unconfident while the correct plateaus are long and saturated, a persistence rule over the stream, for instance emitting an event only after k consecutive agreeing windows above a confidence threshold, would suppress most of them at almost no cost. More details on this are developed in Appendix A.5.10.

Closing the chapter. Together with the cued sessions of Section 5.4.8, these experiments trace a path from a published benchmark to a system that actually runs in real-time. The ESP-Fi HAR reproduction pinned down what a single ESP32 pair can do under subject- and environment-disjoint protocols. The two preliminary double-pair experiments

were less encouraging. Held static postures turned out to be indistinguishable across days, and the fine-grained kitchen gestures did not survive continuous deployment. The cross-environment experiment is where the double-pair setup paid off. Its recogniser reaches 0.766 macro- F_1 on six classes and 0.918 on four, in rooms it has never seen. Adding an idle class costs almost nothing, calibration to a new room needs under a minute of unlabelled audio cued clips, and performance does not collapse even once the segment boundaries are removed.

Some limits remain, and the experiments pin down both. Amplitude alone cannot separate sitting down onto a chair from lying on the floor. And a model trained only on segmented data never encounters the transitions between activities, so it cannot learn to handle them. The next chapter gathers these results and returns to both as directions for future work.

Conclusion and Perspectives

6.1 Summary of the work and of the results

This thesis compared how one pair and two pairs of cheap ESP32-S3 boards can recognise daily activities well enough to be useful for unobtrusive elderly-care monitoring, and in particular whether such a system still works in a room it has never seen during training. We answered this with four experiments: a reproduction of a published benchmark, two preliminary in-house studies, and a main cross-environment study, followed by a real-time deployment of the final model.

The work began with a reproduction of the ESP-Fi HAR benchmark, run on its public dataset with its published models. The aim was not to validate a preprocessing pipeline of our own, but to establish a reference point. That reference point is how well device-free HAR currently performs on ESP32-class hardware with a single TX-RX pair, measured on published data with published models. That reference settled the question the rest of this thesis is built on. A model trained in some rooms did recognise activities in a room it had never seen, so cross-environment recognition on ESP32 hardware is possible in principle. What remained open, and what the in-house experiments then attacked, was whether it survives our own harder conditions: clear NLOS geometry, short windows, a specific class set for elderly-care, and a few hundred windows per room collected by a single person.

The first preliminary experiment (Section 5.2) gave a negative result that shaped everything that came after it. With the double-pair setup and both links in NLOS, we tried to recognise six classes across different days, four of which were held static states (sitting, standing, lying and the empty room). Two completely independent model families were swept, and both plateaued at 0.56 (classical) and 0.45 (deep) macro- F_1 . The per-class scores showed exactly where the problem was. The two motion classes were satisfactory (**walk** was near-perfect and **prepare_meal** clearly above the static classes), while the three of the four static states dropped to 0.31–0.41 F_1 , which is essentially chance among themselves (only standing was partly separable at 0.59 F_1). Merging all four into a single **no_action** label immediately lifted both families to 0.91–0.96 macro- F_1 under the same protocol. The conclusion is that a double pair in NLOS reliably tells us whether someone is moving, but can not distinguish between a motionless person sitting, a motionless person standing, a person lying still and an empty room. A static posture produces no motion-band signature at all. In detail, its identity lives only in the static multipath level, which is the very component that drifts from day to day. In other words, static presence and absence detection is out of reach for this hardware and this geometry, and every action-class must contain some human movement.

The second preliminary experiment (Section 5.3) exposed the other big practical problem, one that is rarely evaluated in the literature. Scores obtained on cued clips do not predict what happens on continuous recordings. On short, controlled, single-activity

kitchen clips in LOS, both a classical feature pipeline and a compact 1D CNN reached over 90% leave-one-out accuracy and macro- F_1 . The same models were then run on six continuous recordings of real coffee- and tea-making routines, manually labelled for the occasion, and the macro- F_1 (that averages only over the classes that actually occur in that) fell to 0.23–0.51 depending on the recording. Nothing about the room, the subject or the day had changed, just the way the data was presented. A sliding window over a real routine constantly falls between two activities, containing the transition from one to the next. None of these situations exist in a cued dataset where every clip starts and ends on a beep, with one clean activity inside. A model trained only on cued clips has never seen the majority of what it will actually be asked to classify, so it collapses. This is why we insist that a lab number, however good, is only the starting point, and this was directly applied in our main experiment.

The main experiment (Section 5.4) then attacked the change of environment itself, with six daily activities recorded with the same double-pair NLOS setup in three different rooms and evaluated under a strict leave-one-room-out protocol. Following the lesson of the first preliminary experiment, the postures were now recorded as transition-movements (`sit`, `sit_up`, `lying`, `lying_up`) rather than as held states, so that every class carries motion. A large grid search over both model families plateaued at 0.579 macro- F_1 , and hundreds of extra configurations did not move it. The limit was in fact structural, not a matter of hyper-parameters. Changing the architecture is what unlocked the problem. Processing each link separately with a shared (siamese) trunk instead of stacking the two links into one 104-channel input, adding a targeted augmentation recipe (subcarrier and time masking, same-class mixup), keeping AdaBN always on, widening the window to 3.5 s and resampling every window onto a uniform wall-clock timestamp grid brought the final model to 0.766 macro- F_1 on six classes and 0.918 on the four-class grouping, in rooms the model had never seen. Going from the high 0.50s to 0.766 F_1 is the central positive result of this thesis. Cross-environment recognition in NLOS is possible with two cheap ESP32 pairs, for this set of classes, and with a performance that is good enough to be useful.

There is one exception, and it is a clean one. Almost all the remaining errors in the final best pipeline sit in two pairs of classes: `sit`↔`lying` and `sit_up`↔`lying_up`. The model reliably detects that a transition happened and in which direction it went, but not whether it ended on a chair or on the floor. Merging these pairs into “going down” and “getting up” removes the problem and gives 0.918 F_1 . In fact, for an elderly-care monitor this coarser grouping still captures the clinically interesting transfer events, so it is a reasonable compromise rather than a failure, but it is a real limitation of an amplitude-only pipeline and is considered as future work in the next subsection.

The last question was how much the second pair is actually worth. Averaging the two links’ probabilities gives 0.766 / 0.918 F_1 , against 0.724 / 0.888 for the best single-link deployment for the 6-class and 4-class grouping, respectively. In detail, the second pair buys about 4 points of F_1 on six classes and 3 points on four classes on average. This is a real gain, but honestly it is not enough to justify doubling the hardware, and the number of power sockets. The clearly better argument for the second pair is a different one. It doubles the amount of data collected per session. Every recording session produces two labelled views of the same activity instead of one, which matters enormously when the whole dataset is a few hundred windows per room collected by a single person. And

the benefit is transferable, because a link trained inside the pooled shared trunk beats the same link trained on its own by 3.8 points of F_1 (link B) and 7.0 points (link A). The two geometries act as free domain augmentation for one another. So the practical recipe we end up recommending is to collect with two pairs and deploy with one. In fact, the late-fusion architecture supports this without retraining, since the same shared trunk runs unchanged on one link or two. A related observation is that where a link is placed matters as much as how many links there are: link B beat link A by 5 to 8 points of macro- F_1 on the six-class task in most configurations we tested (and by 2 to 4 on the four-class grouping), which is a gap as large as the one the entire second pair buys. For NLOS HAR, choosing the right geometry is therefore worth at least as much as adding a second pair while at the same time costing nothing.

Finally, the model was successfully extended to recognise idle situations. Then, it was deployed live in rooms that were not part of the three training rooms, running on a laptop reading the two receivers in real time. Two things came out of this. First, calibration to a new environment is cheap. AdaBN only re-estimates the batch-normalisation statistics from unlabelled windows, so calibrating means asking the subject to perform one cued window per class without the need for any annotation or retraining. Then, the live system just using a single pair, reached 0.779 macro- F_1 / 0.783 accuracy on the extended seven classes grouping and 0.899 / 0.883 on the five class grouping. This took place in a fourth room, unseen during training. A later cued session using the double pair reached 0.801 macro- F_1 / 0.8 accuracy on seven classes, and 0.948 / 0.943 on the five-class grouping, also in a previously unseen room. The two live sessions differ in room, day and calibration, so they measure that the pipeline works in each setting, not the value of the second pair. Second, and encouragingly given the kitchen experience, the model did not collapse on continuous recordings. Running it on long, uninterrupted sequences, `walk`, `prepare_meal`, `no_action` and most posture transitions were still recognised correctly. The errors concentrated in the moments between activities. In other words, the instants where the person is repositioning, which have no corresponding class in the training set and are therefore forced into the nearest activity. So the gap between cued and continuous operation is much smaller here than in the kitchen study.

6.2 Limitations

The results above come with a number of limitations that should be stated plainly.

A single subject. All in-house recordings were performed by one person, who was also the experimenter. Our cross-environment numbers are cross-room but same-subject. In other words, cross-subject generalization is not guaranteed and needs to be tested and evaluated in the future. This is because height, gait, speed, and the way a person sits and lies down all modify the CSI.

Three rooms and small data. The leave-one-room-out protocol uses three rooms recorded over several days with a few hundred labelled windows per room. This is enough to show that transfer is possible but is a very small number of environments and samples to train on. As a result, we believe part of the residual error very likely comes from this

rather than from physics of the signal. The improved performance during live cued sessions (achieved by training the model on all three rooms) strongly supports this conclusion.

Configurations chosen on the held-out folds. With roughly 420 windows per room, an inner validation split was too small to rank configurations reliably, so we ranked every pipeline choice of the cross-environment experiment on the mean score over the three held-out rooms. This is not leakage, since no test label ever reaches a gradient update and AdaBN uses none, but it does make our numbers optimistic. In other words, 0.766 and 0.918 are the best of a large search. The closest thing we have to an unselected check is the live sessions of Section 5.4.8, which ran with the recipe already fixed (but more training data) and landed slightly above the offline estimate.

Cued clips. Even in the cross-environment experiment, training data consists of clips that start and end on a beep with exactly one activity inside. There are no genuine between-activity transitions, and no overlapping activities. This is exactly the reason the live continuous sessions still make mistakes at the activity boundaries.

The chair-versus-floor confusion. As discussed above, endpoint height is not recoverable from amplitude alone based on our experiments, and every descriptor we tried saturated at the same confusion. The six-class and seven-class granularity performances are therefore capped by this limitation, which is surpassed in the coarser four- and five-class grouping.

6.3 Perspectives

The limitations above suggest concrete next steps, and we list them in the order in which we would tackle them.

Separating sitting from lying. This is the single most valuable improvement, because it is the only thing standing between the six- and seven-class numbers and the coarser four- and five-class ones. Two directions are open to us on this hardware, and a third would require changing it. The first is to stop throwing away the low-frequency content. Instead of one band-passed input, feed the network a second, low-pass or unfiltered branch that keeps the near-DC level, and let the network decide how much to trust it. The second, and cheapest, is a geometric one: a RX placed lower, or a link crossing the room at a height between chair and floor, would see the two endpoints differently by construction. It is worth noting that our better link (B) was already the lower one, at 55 cm height. The third is closed to us. The most direct cue, Doppler, lives in the phase, which our single-antenna, unsynchronised boards cannot deliver stably across packets. Settling whether endpoint height is unrecoverable in general, or only from amplitude, would therefore require a different hardware configuration.

Collecting the data that is missing from cued clips. The errors that survive in continuous operation are almost all boundary errors, and the reason is simply that nothing resembling a boundary exists in the training set. The fix is to collect transition data from

each room and retrain the model without changing the architecture. In detail, adding a genuine `transition` class to the classifier itself, exactly as `no_action` was added, could improve the continuous recordings performance. A cheaper complement, which was already mentioned, is to require a minimum number of consistent consecutive predictions over a threshold so that a single bad window of a transition does not become a reported activity. Equally important is that continuous recordings become part of the final evaluation of the pipeline.

More rooms, and more than one subject. These are the two axes on which our evaluation is thinnest, and they are also the cheapest to extend, since collecting in a new room or with a new person needs no new hardware or software, just extra time. More rooms would give the model a wider basis for learning what is room-independent, and would allow different approaches and ablation studies to be judged fairly, reducing any potential seed noise. More subjects would answer two questions at once. Those are whether the model generalises to a new body by using a leave-one-subject-out split, and a subject-and-room-disjoint split variant, and whether AdaBN adapts to a new person as well as it adapts to a new room, since nothing in it is specific to the environment.

Exploring TX-RX placements systematically. One of the clearest findings of the thesis is that link B was consistently better than link A, by 5 to 8 points of macro- F_1 on the six-class task in most configurations. In fact, this was not something we designed, it is something we discovered afterwards. The two links differ in receiver height (55 versus 95 cm) and in TX-RX distance (3.5 versus 3.8 m), so we know a difference exists but not which factor causes it. A systematic sweep of the placement parameters of RX height and TX-RX distance would tell us what makes a good link, and would let us design the optimal placement for our target class set.

Towards a real deployment. Beyond recognition itself, an install-and-forget monitor needs the model to run on the edge rather than on a laptop. Our models were kept small with this in mind, but implementing and deploying them on the device itself was outside the scope of this thesis. Moreover, it needs the unlabelled AdaBN calibration to become an automatic procedure a non-technical person can carry out. Longer term, the useful output for a clinician is not a stream of activity labels but statistics over days, for instance how many transfers, how long they take, and how much movement the subject performs. The encouraging part is that the reliability required for those aggregates is lower than for per-window accuracy.

6.4 Working in a team

This thesis was carried out inside the ZoeCare startup that belongs to the CentraleSupélec Hub. The startup consists of a hardware team and a data-science team, which I was part of. In total, there are around ten people, with the majority being interns or apprentices. The hardware group configured and maintained the ESP32-S3 devices, flashed the right firmware onto them, and produced the data acquisition scripts, including the double-pair TX-RX data collection script that allowed me to collect my own data for this thesis.

The data-science group answered many of my questions on preprocessing, hand-crafted features from the CSI and on the format of the CSI data, while several of my own findings and ideas were passed to them to experiment on their applications. In detail, most of the team works on fall detection, while some people work on localisation and gait prediction. Lastly, the startup collaborates with laboratories, combining the wifi sensing technology with other domains such as behaviour analysis, while also receiving strong interest from clinical environments since the wifi sensing technology is non-intrusive: nothing is worn, and no camera is involved.

APPENDIX A

Supplementary Material

A.1 Work Timeline

This appendix summarises the chronological progression of the work, week by week. Entries are grouped into the six phases that structure the thesis; each row corresponds to the week beginning on the date shown.

Table A.1: Week-by-week timeline of the thesis work (March-August 2026).

Week of	Activities and outcomes
Phase 1 - State of the art and familiarisation	
9 & 16 March	Literature review on Wi-Fi CSI-based HAR. Benchmarking: identification of the most promising papers and public repositories, and reproduction of the pipelines they suggest.
23 & 30 March	First hands-on acquisitions: familiarisation with the ESP32 capture chain, collection of a first personal dataset, characterisation of the recurring data errors and noise sources, and definition of an initial acquisition protocol.
Phase 2 - Acquisition protocol and double-pair setup	
7 & 13 April	Documentation of the data-collection procedure for the collaboration with LURPA, and installation of the corresponding capture setup on their side.
20 April	Double-pair (two TX-RX links) acquisition setup and first experiments with it.
Phase 3 - Cross-day generalisation	
4 May	Cross-day capture campaign to quantify day-to-day variability, grid search over classical machine-learning pipelines, comparison of same-day and cross-day generalisation.
11 May	Additional cross-day captures, first deep-learning approaches and model combinations targeting the cross-day generalisation gap.
17 May	Laptop hardware failure and replacement from Fondation CentraleSupélec. Continuation of the three-class multiclass classification experiments, adding results report and extended literature review.
24 May	Literature review, results report on double-pair 6-class, 4-class and 3-class HAR, with the associated insights, coordination of the data collection with ENS.

continued on next page

Table A.1 (continued)

Week of	Activities and outcomes
31 May	Feature selection for single-pair cross-day 4-class HAR to identify the most informative descriptors. Recording of one further unseen day and evaluation of the existing model on it. Data collection for ENS, pipeline experiments and delivery of the output logs.
Phase 4 - ENS kitchen dataset, segmentation and deep learning	
8 June	Improved log outputs for ENS. New problem of activity-window segmentation: using the stand class as a reference together with Markov transition probabilities was unsuccessful, whereas a CNN classifying each window as transition or activity proved viable. Training of deep architectures for the ENS activity recognition task, the AGC correction was introduced and improved performance substantially.
Phase 5 - Cross-environment generalisation	
15 June	Updated AGC correction and addition of a low-pass Butterworth filter for signal smoothing. Cross-environment data collection and further grid searches.
22 June	Documentation of the ESP-Fi benchmark and of the kitchen-activity experiment in the thesis, preparation of the mid-presentation, extended data collection (part of it corrupted by heat). Exploration of a promising self-supervised approach on ESP-Fi, and identification of the limitations of the ESP-Fi HAR paper: LOS-only activities, short TX-RX distance and 10 s windows.
29 June	Continued cross-environment data collection, thesis documentation (fine-grained kitchen section), grid search for the cross-environment setting.
7 July	Analysis of the grid-search results and improvement of the best configurations, documentation of the cross-environment study in the thesis, performance gains from a Siamese TCN backbone combined with adaptive batch normalisation and a three-seed ensemble.
Phase 6 - Real-time deployment and writing	
13 July	First real-time tests. The single-pair setup performs well, the dominant error being lying predicted as sitting depending on where the activity is performed. Double-pair test, acquisition-script improvements and thesis documentation.

continued on next page

Table A.1 (continued)

Week of	Activities and outcomes
20 July	Systematic exploration of the real-time behaviour (what works and when it fails), identification and correction of an AdaBN calibration issue using 3.5 s unlabelled calibration windows, experiments including an empty-room class and thesis documentation.
26 July	Thesis documentation: addition of the elderly-care sections.
3 August	Performance improvements through per-packet normalisation experiment, restructuring of the report, first continuous real-time recordings.
10 August	Report writing (continuous and real-time sections), and demo application to team.
17 August	Working on the thesis report and coordinated data collection for fall detection task
24 August	Documentation of work and passing it to the rest of data science team and working on the thesis report.

A.2 Reproduction of published ESP32 benchmarks: protocol and results

This appendix documents the reproduction of Section 5.1 in full: the preprocessing recipe taken from the ESP-Fi HAR reference implementation, the benchmarked models and their training schedule, the same-environment leave-one-subject-out results against the published figures, and the subject-disjoint control for the cross-environment experiment. Throughout, performance is measured with classification accuracy and macro-averaged F_1 .

A.2.1 Preprocessing, models and protocols

Preprocessing. The reproduction follows the preprocessing implemented in the ESP-Fi HAR reference code,¹ which is not fully described in the original article. Concretely:

1. **Amplitude extraction.** The CSI amplitude is read directly from its file; phase information is discarded.
2. **Shape alignment.** If the matrix is stored in transposed form (52×950), it is transposed to the canonical layout (950×52).
3. **Per-sample z -score normalisation.** The amplitude matrix is standardised using its own mean and standard deviation:

$$\tilde{x} = \frac{x - \mu_x}{\sigma_x + \varepsilon}, \quad \varepsilon = 10^{-8} \quad (\text{A.1})$$

¹<https://github.com/AutoSmartGroup/ESP-Fi-HAR>

where μ_x and σ_x are computed over all elements of the sample.

4. **Reshape.** A channel dimension is added to the beginning of the tensor, yielding the final shape $1 \times 950 \times 52$.

No additional denoising or windowing steps (Chapter 4) are applied at this stage, so that the measured accuracies reflect the published baseline rather than this thesis’s own pipeline.

Models and training. The benchmark suite comprises seven deep-learning models (CNN, ResNet18, MobileNetV3, EfficientNet-Lite, LSTM, GRU and a Transformer) and three classical machine-learning baselines (SVM, Random Forest (RF), Logistic Regression (LR)). Deep models are trained with the AdamW optimiser, a cosine-annealing learning-rate schedule and cross-entropy loss, on amplitude-only inputs. Following the original benchmark, no validation split is used: the checkpoint is selected on training accuracy and the held-out fold is reserved for final scoring only.

Evaluation protocols. The same-environment results below adopt the leave-one-subject-out (LOSO) protocol of the original ESP-Fi benchmark. For each environment independently, eight folds are run. In every fold one of the eight subjects is held out for testing while the model is trained on the remaining seven. We report the mean $\pm$ standard deviation of accuracy and the mean macro- F_1 over the eight folds, aggregated per environment. Enforcing subject-independence avoids the identity-leakage pitfall discussed in Section 2.4.3. The cross-environment leave-one-environment-out (LOEO) protocol of Section 5.1 instead trains on three environments and tests on the fourth, with all eight subjects present in both. On the other hand, Section A.2.3 adds the variant that removes the subject overlap.

A.2.2 ESP-Fi HAR: cross-subject and same-environment recognition

Table A.2 reports the same-environment LOSO results and Table A.3 the per-environment difference against the published figures. Across all four rooms, ResNet18 is the strongest model, reproducing the central finding of the original benchmark that ResNet18 is the best backbone under same-environment LOSO evaluation. Our ResNet18 reaches 58.9/72.9/75.5/51.3% (corridor/office/meeting/laboratory) accuracy against the paper’s 63.6/70.1/65.5/58.3%; the per-environment gaps range from -7.1 to $+10.0$ points and average $+0.3$, i.e. roughly equivalent. The qualitative hierarchy is also preserved: on average the convolutional family (ResNet18, then EfficientNet-Lite, MobileNetV3 and CNN) outperforms the recurrent models (LSTM, GRU), which in turn sit close to the classical baselines. Every deep model exceeds the 14.3% seven-class chance level, whereas SVM, RF and LR remain weak (17–31%), confirming the paper’s conclusion that classical classifiers on flattened amplitude are insufficient.

Two systematic deviations are worth noting. Our classical baselines are consistently a few points above the paper’s (on average $+6.4$ for RF, $+5.3$ for SVM, $+1.9$ for LR), while most non-ResNet deep models reproduce below it, the gap being largest for the Transformer (-19 on average) and the EfficientNet-Lite, CNN and GRU group (roughly

−9 to −12). These are precisely the architectures most sensitive to the training schedule: with no validation split, a single seed, and the Transformer in particular trained at a very small batch size, the higher-capacity or more finicky models are the most likely to settle in poorer optima. ResNet18, which is the best-performing backbone, reproduces faithfully.

Table A.2: LOSO (Leave-One-Subject-Out) benchmark results – same environment on the ESP-Fi HAR dataset (7 activities, 8 subjects, 4 environments). Format: Acc% $\pm$ std / F1%. Paper values in parentheses.

Type	Model	Corridor		Office		Meeting Room		Laboratory	
		Acc $\pm$ Std	F1	Acc $\pm$ Std	F1	Acc $\pm$ Std	F1	Acc $\pm$ Std	F1
DL	ResNet18	58.93 $\pm$ 10.93 / (63.59 $\pm$ 14.91)	56.47 / (60.81)	72.86 $\pm$ 10.27 / (70.11 $\pm$ 19.60)	70.94 / (64.00)	75.54 $\pm$ 5.74 / (65.53 $\pm$ 18.22)	73.73 / (61.26)	51.25 $\pm$ 12.90 / (58.33 $\pm$ 22.73)	48.01 / (49.14)
	EfficientNetLite	42.68 $\pm$ 10.57 / (61.60 $\pm$ 10.60)	38.65 / (57.13)	47.68 $\pm$ 12.48 / (63.74 $\pm$ 8.74)	46.35 / (58.80)	61.96 $\pm$ 7.97 / (61.98 $\pm$ 5.14)	60.10 / (57.93)	37.86 $\pm$ 8.81 / (30.35 $\pm$ 6.96)	32.53 / (46.27)
	MobileNetV3	44.11 $\pm$ 13.56 / (53.64 $\pm$ 13.24)	39.63 / (47.28)	55.18 $\pm$ 9.22 / (51.82 $\pm$ 11.85)	53.32 / (47.00)	50.18 $\pm$ 7.95 / (53.28 $\pm$ 15.18)	47.77 / (46.80)	34.29 $\pm$ 10.77 / (43.31 $\pm$ 16.25)	30.59 / (38.43)
	CNN	42.86 $\pm$ 10.30 / (57.55 $\pm$ 11.43)	38.98 / (53.20)	45.89 $\pm$ 13.32 / (48.31 $\pm$ 8.02)	42.13 / (43.80)	46.61 $\pm$ 8.67 / (48.26 $\pm$ 11.14)	45.37 / (42.70)	31.43 $\pm$ 7.82 / (52.61 $\pm$ 12.19)	26.80 / (46.89)
	Transformer	30.00 $\pm$ 7.28 / (54.43 $\pm$ 12.30)	24.99 / (49.85)	30.71 $\pm$ 8.97 / (41.93 $\pm$ 9.47)	28.37 / (38.20)	28.57 $\pm$ 12.73 / (47.51 $\pm$ 10.71)	25.01 / (42.43)	28.04 $\pm$ 8.94 / (50.33 $\pm$ 15.11)	23.21 / (46.35)
	LSTM	27.14 $\pm$ 10.22 / (32.15 $\pm$ 13.40)	21.90 / (29.82)	28.93 $\pm$ 7.23 / (38.02 $\pm$ 14.21)	26.29 / (33.80)	35.71 $\pm$ 8.77 / (36.20 $\pm$ 12.04)	32.32 / (33.84)	25.36 $\pm$ 10.24 / (32.73 $\pm$ 13.25)	20.44 / (30.21)
	GRU	23.21 $\pm$ 10.27 / (30.93 $\pm$ 3.78)	18.66 / (28.74)	23.39 $\pm$ 7.78 / (38.02 $\pm$ 8.63)	21.14 / (33.00)	26.79 $\pm$ 7.35 / (34.55 $\pm$ 4.21)	23.19 / (31.17)	22.50 $\pm$ 6.90 / (28.38 $\pm$ 8.98)	17.81 / (24.56)
ML	RF	30.89 $\pm$ 13.79 / (23.57 $\pm$ 7.14)	26.62 / (19.01)	30.36 $\pm$ 4.63 / (21.25 $\pm$ 6.16)	29.27 / (17.19)	30.89 $\pm$ 8.40 / (24.64 $\pm$ 12.37)	27.43 / (21.82)	20.54 $\pm$ 5.76 / (17.68 $\pm$ 7.59)	16.89 / (13.47)
	SVM	30.36 $\pm$ 10.82 / (24.64 $\pm$ 8.01)	26.64 / (19.00)	31.43 $\pm$ 8.05 / (24.11 $\pm$ 7.09)	29.42 / (19.08)	27.50 $\pm$ 6.00 / (20.00 $\pm$ 9.20)	22.01 / (15.83)	20.36 $\pm$ 6.99 / (19.82 $\pm$ 10.61)	16.76 / (14.31)
	LR	19.64 $\pm$ 7.31 / (19.29 $\pm$ 7.63)	17.13 / (16.87)	25.54 $\pm$ 3.37 / (22.86 $\pm$ 3.98)	23.69 / (19.51)	17.14 $\pm$ 4.71 / (16.43 $\pm$ 5.05)	13.86 / (12.66)	18.75 $\pm$ 5.58 / (14.82 $\pm$ 2.36)	14.52 / (11.14)

Paper values in parentheses. Bold = best per scene. DL = Deep Learning, ML = Machine Learning.

Table A.3: Difference in mean accuracy (%) between our LOSO results and the paper (Δ = ours – paper). Positive values indicate improvement over the paper; negative values indicate a gap.

Type	Model	Corridor	Office	Meeting Room	Laboratory
DL	ResNet18	−4.66	+2.75	+10.01	−7.08
	EfficientNetLite	−18.92	−16.06	−0.02	−12.49
	MobileNetV3	−9.53	+3.36	−3.10	−9.02
	CNN	−14.69	−2.42	−1.65	−21.18
	Transformer	−24.43	−11.22	−18.94	−22.29
	LSTM	−5.01	−9.09	−0.49	−7.37
	GRU	−7.72	−14.63	−7.76	−5.88
ML	RF	+7.32	+9.11	+6.25	+2.86
	SVM	+5.72	+7.32	+7.50	+0.54
	LR	+0.35	+2.68	+0.71	+3.93

A.2.3 Subject-disjoint cross-environment control

The plain LOEO protocol of Section 5.1 (like the paper’s) lets every subject appear in both the training environments and the test environment, so part of the apparent generalisation could be subject-identity leakage of the kind flagged in Section 2.4.3. To isolate this effect we add a subject-disjoint LOEO variant (Table A.4): for each target environment, eight folds are run in which one subject is held out as the test set in the target environment and excluded from training entirely. Training uses the remaining seven subjects in the other three environments. The results are essentially unchanged for ResNet18, scoring 74.3/74.3/79.8/75.5% accuracies and a mean of 76.0% against 76.1% for the leaky variant. For these four rooms, then, subject identity is not the dominant cue. In other words, cross-environment performance is driven mainly by the size and diversity of the training set rather than by memorised per-subject signatures.

Table A.4: LOEO with subject-disjoint constraint on the ESP-Fi HAR dataset. For each target environment, 8 folds are run (one held-out subject per fold); training uses the other 3 environments with the held-out subject removed. Format: Acc % $\pm$ std / Macro-F1 %. Bold = best per environment.

Type	Model	Corridor	Office	Meeting Room	Laboratory
DL	ResNet18	74.29 $\pm$ 8.22 / 72.96	74.29 $\pm$ 8.97 / 72.72	79.82 $\pm$ 8.28 / 78.32	75.54 $\pm$ 6.85 / 73.87
	EfficientNetLite	60.36 $\pm$ 13.33 / 57.91	62.32 $\pm$ 11.83 / 60.38	68.39 $\pm$ 6.13 / 65.15	65.54 $\pm$ 12.37 / 62.56
	MobileNetV3	52.50 $\pm$ 15.70 / 48.78	61.61 $\pm$ 9.82 / 59.80	69.11 $\pm$ 7.00 / 66.79	64.46 $\pm$ 9.79 / 62.79
	CNN	50.71 $\pm$ 7.86 / 46.24	50.71 $\pm$ 14.98 / 46.46	56.43 $\pm$ 8.94 / 52.33	53.21 $\pm$ 12.77 / 48.79
	LSTM	20.18 $\pm$ 10.11 / 13.15	25.00 $\pm$ 5.51 / 20.28	30.54 $\pm$ 9.95 / 21.60	32.14 $\pm$ 12.57 / 27.79
	GRU	19.29 $\pm$ 8.05 / 13.58	25.18 $\pm$ 8.43 / 21.60	26.61 $\pm$ 7.08 / 19.21	28.04 $\pm$ 18.34 / 23.88
	Transformer	20.00 $\pm$ 5.91 / 14.27	23.75 $\pm$ 8.90 / 21.41	19.64 $\pm$ 8.90 / 14.59	28.04 $\pm$ 9.16 / 25.86
ML	RF	23.21 $\pm$ 6.60 / 18.13	23.39 $\pm$ 7.32 / 20.13	25.89 $\pm$ 6.63 / 19.76	28.75 $\pm$ 9.55 / 24.77
	SVM	22.68 $\pm$ 7.46 / 18.05	24.29 $\pm$ 7.71 / 19.22	24.82 $\pm$ 5.76 / 18.15	20.18 $\pm$ 4.86 / 15.82
	LR	18.57 $\pm$ 4.77 / 16.98	18.04 $\pm$ 5.23 / 16.74	16.96 $\pm$ 6.50 / 14.53	15.18 $\pm$ 3.74 / 11.26

A.3 Preliminary cross-day double-pair experiment: applied workflow

This appendix documents the applied workflow behind the preliminary double-pair experiment of Section 5.2: the data-collection setup and cross-day protocol, the classical-feature grid search, the deep-learning sweep, the per-class evidence that the four static states (`sit`, `stand`, `lying`, `empty`) are mutually indistinguishable, and the merged three-class confirmation. All parts share the same dataset and the same leave-one-day-out (LODO) cross-validation, with mean macro- F_1 and accuracy over the three held-out days as the ranking metric.

A.3.1 Data collection setup

Two ESP32-S3 TX-RX pairs (identified by their receiver identifiers, AC84 (RX 1) and CB00 (RX 2)) were deployed orthogonally along two walls of a single rectangular room, the first in-house realisation of the double TX-RX architecture of Section 4.2. Unlike the later kitchen experiment, no activity was performed on either direct TX-RX path: all six classes are NLOS to both links. Each receiver reports CSI amplitude over 64 subcarriers at ≈ 100 Hz; the 12 structurally-zero subcarriers are removed, leaving $K=52$ channels per pair, and phase is not captured as throughout this thesis.

The activity set comprises two continuous motion classes (`walk`, `prepare_meal`) and four held static states: `sit`, `stand` and `lying`, each maintained motionless or with minimal motion for the whole segment, plus `empty` (the unoccupied room). There are deliberately no transition movements (the subject is already in the posture when the segment starts) so the static classes can only be separated by what a stationary body does to the channel, not by the movement into the posture.

Data were collected with the audio-cued solo protocol of Section 4.3 in the room MF207 on four days (29 April, 4 May, 5 May and 6 May) as continuous sessions of labelled 45s segments (buffer and changing-zone material discarded). The first day comprises three activity sessions and two dedicated empty-room sessions. Each later day comprises four sessions covering all six classes. Table A.5 gives the per-day segment counts. Both

Table A.5: Labelled 45 s segments per day and class (per receiver; both receivers capture every segment). 29 April contributes $\approx 40\%$ more segments.

Day	walk	prep.	meal	sit	stand	lying	empty	Total
29 Apr	5		5	6	5	5	8	34
4 May	4		4	4	4	4	4	24
5 May	4		4	4	4	4	4	24
6 May	4		4	4	4	4	4	24

RX record every segment simultaneously but write separate capture files. The two files' segment tables are row-aligned, so every labelled segment exists as a matched pair of receiver recordings.

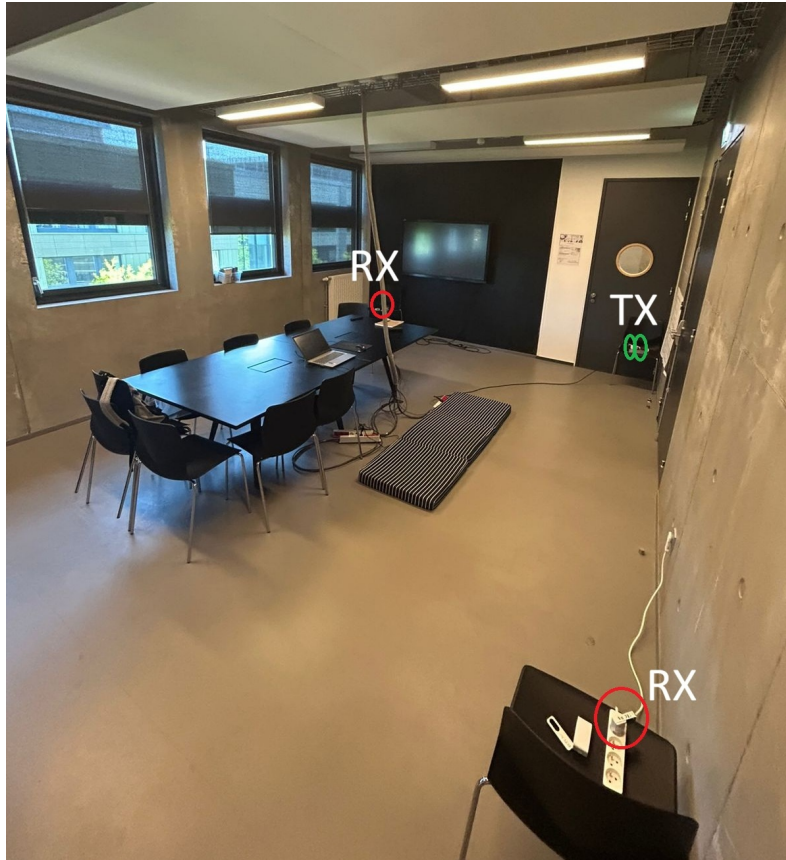


Figure A.1: The room of the preliminary experiment.

A.3.2 Cross-day protocol

The evaluation is a three-fold LODO cross-validation: each fold holds out one of the three May days for testing and trains on the remaining three days (Table A.6). The first day, 29 April, is kept in the training set of every fold and never tested. This is a deliberate choice driven by its sample surplus (Table A.5): testing on it would have made one fold's test set larger and differently composed than the others, whereas using it

Table A.6: The three leave-one-day-out folds. 29 April is training-only in every fold due to its sample surplus.

Fold	Test day	Training days
1	5 May	29 Apr, 4 May, 6 May
2	4 May	29 Apr, 5 May, 6 May
3	6 May	29 Apr, 4 May, 5 May

purely for training exploits the extra data equally in all folds and keeps the three test folds balanced. Because the split is by day, no window from the test day is seen during training, so all scores measure cross-day generalisation. Metrics are accuracy and macro- F_1 and the test folds are class-balanced, so the two metrics track closely.

The two model families consume the double-pair data differently. The classical pipeline fuses the pairs at the feature level: descriptors are computed per RX and concatenated, together with explicit cross-RX features (per-subcarrier Pearson correlation between the two links). The deep pipeline instead pools the two receivers as independent samples (each device contributes its own windows) so neither the early concatenation of the kitchen experiment nor the siamese fusion of the cross-environment experiment is used here. Windowing also differs per family: the classical grid used 4 s windows with a 1 s stride (4,352 windows in total, 985–986 test windows per fold after balancing empty), while the deep sweep used 4 s windows with a 1 s stride (8,661 windows, the two RX entering as independent samples).

A.3.3 Classical feature pipeline and grid search

Each window is condensed into a hand-crafted feature vector of the same design later reused by the kitchen experiment (Appendix A.4.2): 36 per-subcarrier descriptors (amplitude statistics, spectral shape and band powers, autocorrelation profile, Hjorth parameters, permutation entropy, breathing-band measures) for each of the 52 subcarriers of each receiver, 8 receiver-level scalars per receiver, and 53 cross-RX features (4,021 features per dual-pair window in the extended set, with a 22-per-subcarrier “original” variant (2,549) as an alternative).

The cross-day grid search swept 24 configurations (48 model×config result rows) over five active axes: denoising (median vs. Hampel; Section 3.4), per-segment baseline subtraction, window normalisation (none vs. z-score), resampling to a uniform 100 Hz grid, and RX timestamp alignment. The latter is only defined when resampling is on and both pairs are used, which prunes 8 of the $2^5=32$ combinations. Four further axes were pinned to the values that earlier, wider sweeps had already settled: both devices with cross-RX features, the extended feature set, mutual-information feature selection ($k=500$), and $W=4$ s windows with a 1 s stride. The sweep itself scored logistic regression and extra trees. The five best configurations were then re-run with the full suite (adding random forest and XGBoost), and a multilayer perceptron and a residual MLP were added afterwards. Mean macro- F_1 across the grid ranges from 0.305 to 0.558.

The winning configuration is Hampel filtering with per-segment baseline subtraction, no window normalisation, both devices, 100 Hz resampling, the extended feature set,

Table A.7: Classical family, six-class cross-day (LODO) results. Top block: best grid configuration (Hampel filter, baseline subtraction, no window normalisation, both pairs, 100 Hz, extended features, MI $k=500$, $W=4$ s / stride 1 s). Per-fold macro- F_1 (5 May / 4 May / 6 May, i.e. folds 1–3 of Table A.6). Chance level = 0.167.

Model	Mean Acc	Mean F_1	Per-fold F_1
<u>Grid winner, MI $k=500$, both pairs</u>			
Logistic regression	0.558	0.558	0.543/0.552/0.580
XGBoost	0.527	0.513	0.497/0.499/0.543
Random forest	0.513	0.496	0.439/0.472/0.578
Extra trees	0.510	0.485	0.452/0.473/0.531
<u>No feature selection, all features</u>			
Logistic regression, both pairs	0.565	0.555	0.572/0.509/0.585
Logistic regression, RX B only	0.485	0.467	0.455/0.476/0.471

mutual-information selection at $k=500$, $W=4$ s / stride 1 s, classified by logistic regression. The marginal analysis shows baseline subtraction (which is defined as subtracting each subcarrier’s segment-wise temporal mean, so the features describe deviations from the static channel level rather than its absolute amplitude) to be the one decisive axis (0.500 vs. 0.454 mean macro- F_1). Resampling helps slightly (0.481 vs. 0.469), the two denoisers are indistinguishable (0.476 vs. 0.478), and z-scoring wins on average (0.488 vs. 0.465) even though the single best configuration does without it. RX timestamp alignment is not printed in the configuration label, but the two aligned/unaligned variants of the winning label score 0.558 and 0.494, so it matters as much as any swept axis.

Table A.7 reports the six-class results. Logistic regression tops the grid at 0.558 accuracy / 0.558 macro- F_1 , ahead of XGBoost (0.513) and the two forest models (0.485–0.496). Refitted on all 4,021 features without MI selection it reaches 0.565 / 0.555, and this is the variant broken down per class in Appendix A.3.4. Also, restricting the pipeline to the single stronger pair (RX B) costs about nine points of macro- F_1 , which is the clearest evidence that the second pair contributes real information at this granularity.

A.3.4 Where the errors live: the static-state quartet

Table A.8 breaks the classical model down per class, and constitutes the central evidence of this experiment. The two motion classes are essentially solved (**walk** at 0.92 mean F_1 and **prepare_meal** at 0.79) while the three static-body classes and the empty room collapse to 0.31–0.41, with **stand** somewhat higher (0.59); **empty** is not merely weak but erratic, swinging from 0.22 to 0.54 depending on the held-out day. The confusion matrices show the residual mass circulating almost entirely within the quartet: **sit** is predicted as **lying** or **empty**, **lying** as **sit** or **empty**, and **empty** as any of the seated/lying states. A motionless person is not reliably distinguishable from no person at all.

An intermediate merge isolates how much of the failure is the **sit**↔**lying** pair specifically: collapsing only those two into **sit/lying** (a five-class task) lifts the mean macro- F_1 from 0.467 to 0.562 on the same (RX B) feature matrix, but **empty** remains broken (F_1 0.16–0.31 across folds) and **stand** unstable (0.24–0.54). The whole quartet (not any sin-

Table A.8: Per-class F_1 of the highest accuracy classical model (LogReg, six-class, dual pair, all 4,021 features; mean accuracy 0.565) on each held-out day. The motion classes survive; the static quartet collapses to near-chance among themselves.

Class	5 May	4 May	6 May	Mean
walk	0.82	0.97	0.96	0.92
prepare_meal	0.76	0.74	0.88	0.79
stand	0.58	0.64	0.55	0.59
empty	0.54	0.22	0.48	0.41
lying	0.49	0.20	0.26	0.32
sit	0.24	0.29	0.39	0.31

gle pair within it) must be merged before the task becomes solvable, which is what the three-class evaluation of Appendix A.3.6 confirms.

A.3.5 Deep-learning sweep

The deep sweep operates on Hampel-denoised 4 s windows and covers a residual 1-D CNN (CNN1D) and a shallow 1–2-layer variant, a 2-D CNN treating the $T \times C$ window as an image, CNN-LSTM and a small pre-norm Transformer, plus a feature-MLP over the hand-crafted bank (over a smaller 893-feature hand-crafted bank) as a bridge to the classical family. The preprocessing axis follows the per-window modes of Section 4.7.4 (**znorm**, PC1 removal, temporal differencing (**diff**, **pc1_diff**)) with two experiment-specific controls: explicit resampling to 100 Hz, and empty-room background subtraction (training-empty mean, and a closest-empty per-session reference). On-the-fly augmentation, label smoothing, and a domain-adversarial day classifier (DANN, gradient-reversal layer) complete the axes. Note that this preliminary pipeline predates the AGC gain correction and the recording-level band-pass introduced in the later experiments.

Table A.9 reports the six-class sweep. Temporal differencing is the one clearly helpful choice (it cancels the per-subcarrier DC level that drifts between days) and the differenced CNN1D with augmentation tops the family at 0.463/0.451, with the feature-MLP equivalent at 0.459/0.440; DANN, explicitly optimised for day-invariance, does not exceed it. The per-class error structure between the deep learning and classical feature approaches is very similar in both families.

A.3.6 Merged three-class confirmation

To confirm that the static quartet is the sole bottleneck, both families were rerun with **sit**, **stand**, **lying** and **empty** merged into a single **no_action** label, leaving **walk**/**prepare_meal**/**no_action** under the identical LODO protocol. Table A.10 shows the outcome: the classical logistic regression rises from 0.558 to 0.911 macro- F_1 and the deep family from 0.451 to 0.956 (2-D CNN), with several further models (DANN, the 100 Hz-resampled differenced CNN, and the feature-MLP) all in the 0.90-0.91 band. Even a single pair suffices at this granularity (0.853 f1 with the classical pipeline). The jump comes entirely from removing the static-state distinctions; the motion-versus-stillness

Table A.9: Deep-learning family, six-class cross-day (LODO) sweep (4s windows). Mean accuracy and macro- F_1 over the three held-out days. Chance level = 0.167.

Model	Preprocessing	Mean Acc	Mean F_1
CNN1D	raw	0.253	0.186
CNN1D	pc1_znorm	0.243	0.196
CNN1D	pc1_znorm+Aug	0.313	0.264
ShallowCNN1D	pc1_znorm+Aug	0.374	0.309
CNN-LSTM	pc1_znorm	0.283	0.227
Transformer	pc1_znorm	0.279	0.245
CNN2D	pc1_znorm+Aug	0.425	0.378
CNN1D	diff	0.420	0.414
CNN1D	pc1_diff	0.442	0.439
CNN1D	pc1_diff+Aug	0.463	0.451
DANN	pc1_diff+Aug	0.442	0.406
CNN1D	closest-empty bg-subtraction	0.375	0.351
CNN1D	pc1_znorm+100 Hz	0.293	0.247
Feature-MLP	893-feature bank	0.459	0.440

boundary that remains transfers cleanly across days. This is the quantitative confirmation of the finding stated in Section 5.2: an NLOS double-pair amplitude pipeline separates moving from still, but cannot tell which still state it is observing and nor whether anyone is present in the room at all.

A.4 Fine-grained kitchen experiment: data collection and workflow

This appendix documents the fine-grained kitchen experiment of Section 5.3 in full: the in-house acquisition it relies on (sensing geometry, activity set, and the two data regimes), the two classifier-development phases, the complete hand-crafted feature bank, the specification of the deployed pipeline, the per-recording results on the continuous routines, the selection procedure (a custom score and grid search whose use of the deployment recordings makes it unsuitable as the thesis’s headline metric), the downstream purpose, and the segmentation attempts that did not generalise. We fully executed the experiment, while the LURPA lab partner used the inference outputs from the real-world recordings for further behavioural studies.

A.4.1 Data collection

To our best knowledge, no existing dataset captures fine-grained kitchen activity with two co-deployed transmitter–receiver pairs. The experiment of Section 5.3 therefore uses an in-house acquisition built around the double-pair architecture introduced in Section 4.2, following the generic audio-cued solo-collection procedure of Section 4.3. This consists of a same-room, same-subject, same-day, LOS fine-grained HAR experiment.

Table A.10: Merged three-class cross-day (LODO) results (walk/prepare_meal/no_action), both families. Mean over the three held-out days. Chance level = 0.333.

Family	Model	Configuration	Mean Acc	Mean F_1
Classical	LogReg	best grid config, both pairs	0.911	0.911
Classical	LogReg	single pair	0.854	0.853
Deep	CNN2D	pc1_znorm+Aug	0.956	0.956
Deep	DANN	pc1_diff+Aug	0.913	0.913
Deep	CNN1D	pc1_diff+Aug+100 Hz	0.908	0.907
Deep	Feature-MLP	hand-crafted bank	0.904	0.904
Deep	CNN1D	pc1_diff+Aug	0.900	0.898
Deep	CNN1D	pc1_znorm+Aug+LS	0.789	0.786
Deep	Transformer	pc1_znorm	0.692	0.680
Deep	BiLSTM	pc1_znorm	0.346	0.322

Sensing setup and zones. The deployment uses two Seeed Studio XIAO ESP32-S3 TX-RX pairs, each acting as the receiving endpoint of a bistatic link and identified throughout by its device identifier, AC84 and CB00. Both pairs operate concurrently inside a single room which is a classroom space repurposed as a mock kitchen, with two shelving units serving as the counter (a chest-level shelf and a higher top shelf). The two TX-RX links use separate transmitters that are co-located in one corner of the room (powered from the same outlet), while the two receivers are positioned so that the two TX-RX direct LOS paths are mutually orthogonal. In this way, the room is divided into two activity zones: a coffee zone and a tea zone. The two pairs are placed so that each views one zone in LOS, giving the complementary spatial perspectives motivated by the Fresnel-zone blind-spot and orientation-dependence analysis of Sections 3.2.4 and 3.2.5.

Placing each pair so that its target zone (and the subject working in it) lies on the direct TX-RX path is further supported by the device-free kitchen HAR PhD study of [8]. Comparing three sensor placements for fine-grained knife activities, that work found that the configuration in which both the user and the performing area are in the LOS of the sensors was the only one to produce statistically separable activity classes. Placements leaving the user, or both the user and the performing area, in NLOS did not. The PhD author concludes that sensor placement has a profound, measurable impact on device-free kitchen HAR, and identifies as optimal the placement in which the subject and the performing area both lie on the direct TX-RX path. Each bistatic pair in our deployment therefore views the activity it is responsible for sensing in LOS, while the orthogonal geometry simultaneously leaves the other zone’s activity off-path. The layout can be seen in the Figure A.2

Each receiver reports the CSI amplitude over 64 subcarriers at a sampling rate of roughly 100 Hz. Phase is discarded, consistent with the amplitude-only choice justified in Section 3.2.2. Of the 64 subcarriers, 12 are structurally zero and are removed, leaving $K = 52$ usable subcarriers per pair. The two pairs are concatenated along the subcarrier axis, producing an input of $2K = 104$ channels per packet (the dual-pair counterpart of the single-antenna 1×52 layout used in the single pair configuration).

Table A.11: The two TX-RX pairs and the zone each observes in line-of-sight.

Pair	Zone	Nominal geometry	Placement	Figure
CB00	Coffee zone	Line-of-sight in coffee scenarios	RX next to the window	A.4
AC84	Tea zone	Line-of-sight in tea scenarios	RX next to the door	A.3

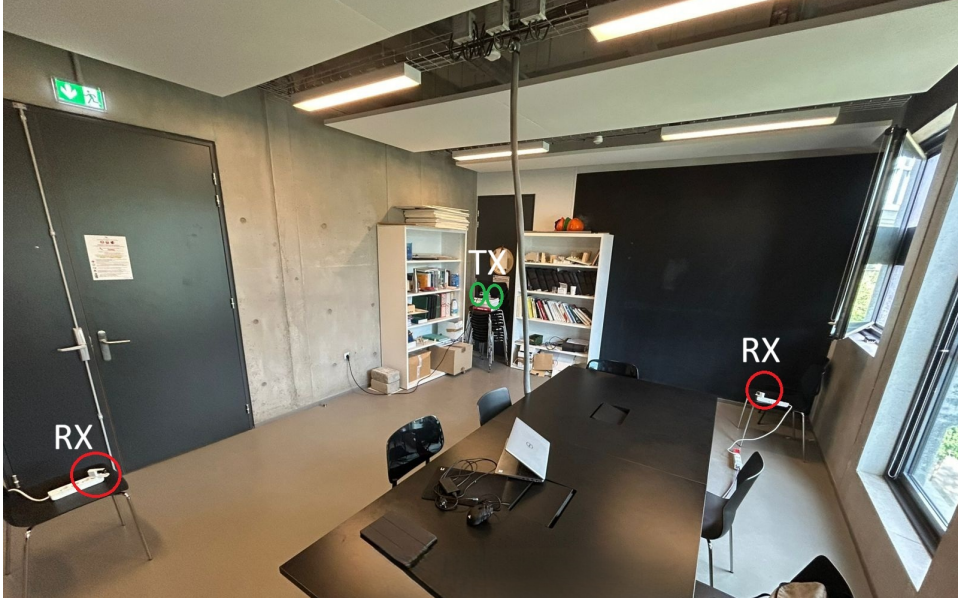


Figure A.2: The mock-kitchen room. Two shelving units act as the counter (chest-level and top shelves), with the two ESP32-S3 pairs (AC84, CB00) covering the tea (left shelving unit) and coffee zones (right shelving unit) respectively. The red circles contain the RXs and the green ones the TXs.

Activity set. The six classes were not chosen arbitrarily: they follow directly from the objective of the collaboration with the LURPA partner, whose downstream pipeline consumes our kitchen-activity pipeline outputs. The set was specified jointly so that a Wi-Fi HAR model predicts short, atomic kitchen movements, which are then combined with temporal information into symbolic virtual events and fed to an automaton-based (PFA) recognition and deviation-detection layer that infers the high-level routine (making coffee or making tea).² In other words, the HAR model is not asked to recognise “make coffee” end-to-end. It provides the alphabet of atomic actions over which the automaton reasons. This framing fixes two properties of the class set: the classes must be broad enough to be detectable from Wi-Fi amplitude yet specific enough to separate the gestures that compose a coffee or tea routine, and they must be short, repeatable units rather than full activities, so that a continuous routine can be expressed as a temporal sequence of them.

Accordingly, the six classes are two generic states (`walk`, `stand`) and four fine-grained manipulations performed at the counter (`grab_above`, `grab_counter`, `pour`,

²Activity classes and recording structure were defined jointly with the LURPA partner as part of the collaboration (internal, unpublished protocol, 2026).



Figure A.3: The tea zone with its shelving unit. This shelving unit acts as the kitchen counter for preparing tea. The black circle shows the object used for the action of **grab_above**, the blue ones the objects used for the action of **pour** and the yellow the object of **grab_counter**. Because the experiment was conducted in a shared university room, the other objects and furniture could not be removed.

prepare_cafe). Each activity was physically scripted as summarised in Table A.12. The fine-grained group is the crux of the design: because several distinct interactions—taking a cup, taking a pod, adding sugar—project onto the same few arm movements, the automaton depends on the recogniser to tell **grab_above**, **grab_counter**, **prepare_cafe** and **pour** apart. Even though the present double-pair geometry captures two coarse zones (coffee/tea), the burden of separating the look-alike counter gestures falls only on the kitchen-activity pipeline, leaving richer zone-level disambiguation to future work.

The pipeline is positioned as the front end of a larger recognition stack: it is trained on isolated repetitions of each atomic class and evaluated, in its intended role, by streaming its predictions over the continuous routines, so that the resulting (**t_start**, **t_end**, **movement**, **confidence**) logs can be handed to the final automaton layer.



Figure A.4: The coffee zone with its shelving unit. This shelving unit acts as the kitchen counter for preparing coffee. The black circle shows the object used for the action of `grab_above`, the blue the object of `pour`, the yellow the object of `grab_counter` and the orange one the object of `prepare_cafe`.

Table A.12: The six kitchen activity classes and how each was physically modelled during collection.

Class	Physical realisation
<code>walk</code>	Walking into position or around the room
<code>stand</code>	Standing still in a zone
<code>pour</code>	Pouring from a metal bottle (for water) or from a glass (for sugar)
<code>prepare_cafe</code>	Inserting a capsule into a small box and pressing a button
<code>grab_counter</code>	Grabbing a capsule / tea bag at chest level from the counter
<code>grab_above</code>	Grabbing a paper cup from the top shelf

Controlled training clips. The training set consists of short, controlled, single-activity clips, each of which had to be collected by hand. The Python script of Section 4.3, which uses distinct sounds to signal the exact moment an action should start and when it should end, was used. This audio-cued protocol made solo collection feasible and slightly faster, but the process remains repetitive and time-consuming (every clip must still be performed, recorded and checked across all classes) which directly bounds how much data

can realistically be gathered. Each class is represented by 33 segments of $\approx 3 - 5$ s (the analysis window is fixed at 2.9 s just slightly under the shortest measured clip) for a total of $6 \times 33 = 198$ labelled segments. All clips were recorded on the same day, by the same subject (myself), in the same room, so the training pool contains no natural variation in person, environment, day, or hardware state. To prevent the classifier from binding an activity to a single zone (and therefore to a single pair), half of each class’s clips were performed in the coffee zone and half in the tea zone. The task is thus posed under same-subject, same-day, same-environment, LOS conditions.

Continuous deployment recordings. The trained models are evaluated not only on held-out controlled clips but also on continuous recordings of real routines. Those routines consist of coffee- and tea-making sequences in which the six kitchen activities are chained together without breaks and interleaved with walking into position, standing, and transitioning between zones. This is a substantially harder, out-of-distribution setting than the controlled clips. Forty sessions were recorded in total (20 coffee and 20 tea). Each routine was likewise performed and captured by hand, with a video camera filming in parallel to provide the ground truth and to allow further analysis. Turning this video into labels is itself a manual, repetitive effort: each recording must be rewatched and transcribed, second by second, into an ordered list of (**start**, **end**, **class**) intervals. Because this annotation is so time-consuming, only six of the recorded routines (three coffee: **cafeA_1**, **cafeB_3**, **cafeC_5** and three tea: **teaB_1**, **teaC_3**, **teaD_5**) were labelled at the segment level, each spanning ≈ 21 -42 s and yielding 39 ground-truth segments in total. These six sequences, together with the leave-one-clip-out evaluation on the controlled clips, are the evaluation targets of the experiment. The subset of those six real routines is an estimate of real-world behaviour, not a census of all 40 routines.

A.4.2 Phase 1-classical pipeline and feature bank

After Hampel denoising and zero-subcarrier removal (no gain correction in this approach), a bank of $77K + 17 = 4021$ statistical and spectral features (with $K = 52$ subcarriers per pair) was computed per clip over both pairs. The layout is, in order: per-subcarrier features for each receiver, receiver-level summaries, a cross-receiver block, and raw-energy features. A lot of the features were suggested by the rest of the data science team:

$$\underbrace{(36K + 8) \times 2}_{\text{per pair: 36 per-subc. + 8 RX-level}} + \underbrace{(K + 1)}_{\text{cross-RX}} + \underbrace{2 \cdot 2K}_{\text{raw energy}} = 4021.$$

The 36 per-subcarrier features (computed for every subcarrier of every receiver) are listed in Table A.13. Autocorrelation lags are converted from seconds to packets at each clip’s native rate.

The remaining $16 + 53 + 208 = 277$ features summarise each pair globally, relate the two pairs, and record raw energy. The eight rx-level features (computed once per pair) are listed in Table A.14; the cross-receiver and raw-energy blocks in Table A.15.

Feature selection used an ℓ_1 -penalised one-vs-rest logistic regression (mean absolute coefficient across classes as the ranking), with a grid over the number of retained features k , recomputed inside every fold to avoid leakage. A broad classifier comparison (Logistic

Table A.13: The 36 per-subcarrier features (each computed per subcarrier, per receiver).

Group	Feature(s)
Statistics	mean, median, std, MAD, range, IQR, skewness, kurtosis
Spectral shape	spectral entropy, high-freq ratio (> 5 Hz), dominant frequency, total absolute power, spectral peakedness
Band powers	0–1, 1–3, 3–8, 8 ⁺ Hz (normalised)
Original-band powers	0.1–0.5, 0.5–2, 2–5, 5–10, 10–25 Hz
Autocorrelation	lags 0.25 s, 0.5 s, 0.75 s, 3 s, 4 s; profile mean and max over lags {0.2, 0.4, 0.6, 0.8, 1.0, 1.5, 2.0} s
Temporal/complexity	zero-crossing rate, Hjorth mobility, Hjorth complexity, sub-window energy std (8 blocks), permutation entropy (order 3)
Breathing band	breathing peak-to-noise ratio, breathing dominant frequency (0.1–0.5 Hz)

Table A.14: The 8 receiver-level features (each computed once per pair).

Feature	Definition
Variance entropy	Shannon entropy of the normalised per-subcarrier std vector
Gini coefficient	Gini index of the per-subcarrier std distribution
Motion-event rate	Number of detected active spans (frames whose $ \Delta $ exceeds the per-subcarrier std) per unit time
Mean event duration	Mean length of those active spans
CV of inter-event int.	Coefficient of variation of the gaps between active spans
Subc. std kurtosis	Kurtosis of the per-subcarrier std distribution
Top-5 energy frac.	Fraction of total variance carried by the 5 highest-variance subcarriers
Breathing ratio	Std of the 0.1–0.5 Hz band-passed mean signal, relative to the raw mean signal

Regression, Random Forest, Extra Trees, Gradient Boosting, XGBoost, SVC-RBF, MLP) was run; the best per-classifier LOO results are in Table A.16. Performance peaked at ≈ 92 – 94% but did not transfer to the real recordings.

A.4.3 Phase 2-deep-learning architecture sweep

The second phase replaced hand-crafted features with end-to-end models on the $(T, 104)$ windows, with on-the-fly augmentation, under the same LOO protocol. Sequential-only models (BiLSTM, Transformer) underperformed in the previous experiment and were omitted. The shallow z -normalised CNN was strongest (Table A.17), again reaching $\approx 94\%$ LOO yet failing on deployment (a second independent route to the same conclusion, and the observation that redirected the investigation from the classifier to the signal).

The models differ in a per-window normalisation mode applied after subcarrier selection and pair concatenation, defined in Table A.18. **PC1 removal** centres each channel over time—removing the static multipath background—and then projects out the first principal component of the resulting $(T, 104)$ window, i.e. the dominant fluctuation shared across all channels, leaving the channel-specific motion residual; **PC1-diff** additionally differences consecutive frames to cancel any residual static offset. The **ResNet1D-stats**

Table A.15: Cross-receiver and raw-energy features.

Block	Count	Definition
Cross-RX Pearson	$K = 52$	Pearson correlation between the two pairs' amplitude, per subcarrier
Cross-RX breathing	1	Magnitude-squared coherence between the two pairs' mean signals in the 0.1–0.5 Hz band
Raw variance	$2K = 104$	Per-subcarrier temporal variance of the un-normalised amplitude, for each pair
Raw mean-square	$2K = 104$	Per-subcarrier mean of the squared amplitude, for each pair

Table A.16: Best classical configuration per classifier, leave-one-out cross-validation on the 198 controlled clips (both pairs, ℓ_1 /native ranking).

Classifier	Ranking	LOO accuracy	LOO macro- F_1
SVC (RBF)	ℓ_1 -LR ($k=2100$)	0.924	0.925
L1 LogReg	ℓ_1 coef ($k=2800$)	0.919	0.919
Extra Trees	native ($k=3700$)	0.909	0.910
Random Forest	native ($k=3500$)	0.884	0.884
MLP	ℓ_1 -LR ($k=1200$)	0.843	0.844

variant is the same residual network but with global mean+std pooling (concatenating the temporal mean and standard deviation) instead of average pooling, so that steady classes such as **stand** are not flattened away.

Table A.18: Per-window normalisation modes used in the deep-learning sweep. A channel is considered one subcarrier of one pair.

Mode	Definition (per window, applied per channel over time)
z-norm	Subtract the per-channel temporal mean and divide by the per-channel temporal std (amplitude structure preserved).
PC1 + z-norm	Subtract the per-channel mean; compute the first principal component $\mathbf{v}_1$ of the window by SVD and remove its projection, $X \leftarrow X - (X\mathbf{v}_1)\mathbf{v}_1^\top$; then apply z-norm to the residual.
PC1-diff	Apply the PC1 removal above, then take the first temporal difference Δ_t (frame-to-frame), cancelling residual static offset; no subsequent z-score (window ends up one frame shorter).

A.4.4 The AGC correction and the deployed pipeline

A signal-level investigation identified a problem related to the hardware itself. The CB00 pair's amplitude barely fluctuated, even for activities performed in its direct LOS, so the models were effectively relying on a single pair and on whatever session-specific structure remained (precisely the single-link blind-spot failure mode that the double-pair architecture is meant to fix). The remedy, after scrutinising the metadata of the ESP32 data

Table A.17: Representative deep-learning models, leave-one-out cross-validation on the controlled clips.

Model	Preprocessing	LOO accuracy	LOO macro- F_1
Shallow 1D CNN	z -norm	0.939	0.940
CNN1D + PC1 removal	PC1 + z -norm	0.904	0.903
ResNet-1D	PC1-diff	0.737	0.737

collection, was to fold the receiver’s automatic-gain-control (AGC) compensation gain back into the raw amplitude,

$$\text{amp}_{\text{corr}} = \text{amp} \cdot \text{compensate_gain}, \quad (\text{A.2})$$

after which the CB00 signal’s activity-dependent structure emerged (Figure A.5) and the two pairs were placed on a comparable footing. This gain correction is what makes the experiment a genuine double-pair study rather than a single-pair one in disguise. Since two pairs are used in the data collection, it is also important to utilize them to the fullest.

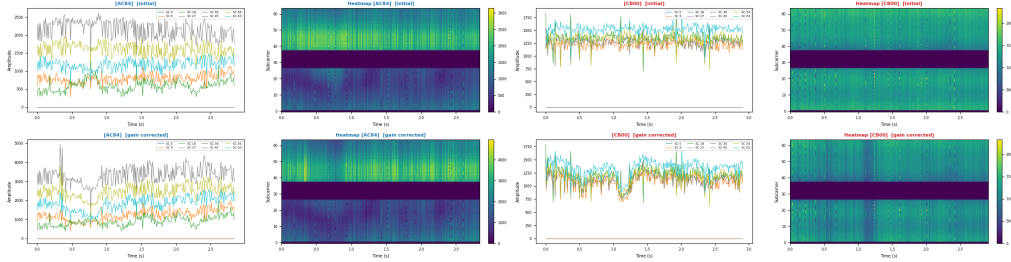


Figure A.5: Effect of the AGC transform on the *pouring* activity in the direct line of sight of the CB00 pair. The near-flat raw amplitude (top right) recovers clear activity-dependent structure after correction.

Figure A.6 shows the same pouring activity on the AC84 link: even where the raw amplitude already carries visible structure, the AGC correction of Equation (A.2) sharpens it considerably.

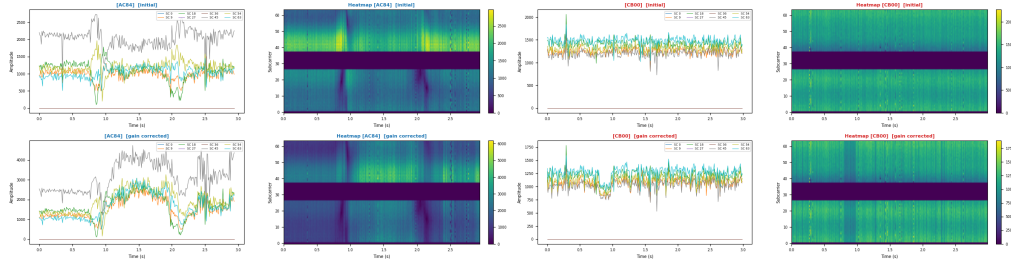


Figure A.6: Effect of the AGC transform on the *pouring* activity in the direct line of sight of the AC84 pair.

Guided by the controlled-data validation of Sections A.4.2 and A.4.3 and by the grid search of Section A.4.6, a single pipeline was selected and fixed for all subsequent analysis:

```
gain_denoise_bandpass | ResNet1D_znorm | device_robust.
```

The signal is gain-corrected (Eq. (A.2)), Hampel-denoised and band-passed (0.5–10 Hz, order 2, zero-phase), then z -normalised per window and classified by a compact one-dimensional residual CNN; every stage is specified in Table A.19. Training uses a “device-robust” augmentation preset whose device dropout (zeroing an entire RX half) explicitly prevents the model from leaning on a single pair, a direct safeguard against a possible silent-pair failure or frequency sampling drop.

Under the leave-one-clip-out (LOO) protocol the pipeline reaches 90.9% accuracy and 0.909 macro- F_1 on the 198 controlled clips (Figure A.7). This is an in-distribution number (same day, same subject, same room) so it measures only that the controlled clips are separable, not that the model generalises. All six classes, including the fine-grained counter actions, are cleanly separated under this protocol. How little of this survives on real routines is quantified in Section A.4.5.

Table A.19: Full specification of the deployed kitchen pipeline (`gain_denoise_bandpass` | `ResNet1D_znorm` | `device_robust`), the configuration for which all real-routine inferences were produced.

Component	Setting
<i>Input and channel layout</i>	
Subcarriers	64 captured; 12 zero ones removed $\Rightarrow$ 52 kept per pair
Pairs	2 (<code>AC84</code> + <code>CB00</code>) concatenated $\Rightarrow 2 \times 52 = 104$ channels
Signal	CSI amplitude only, ≈ 100 Hz
<i>Preprocessing</i> (applied per full segment)	
Gain correction	amplitude $\times$ per-packet <code>compensate_gain</code> (linear; Eq. (A.2))
Hampel filter	window = 9 packets, threshold = 3σ (MAD, scale 1.4826)
Band-pass	zero-phase Butterworth band-pass (order 2, 0.5–10 Hz, $f_s = 100$ Hz)
<i>Windowing and normalisation</i>	
Window	2.9 s, timestamp-based; zero-pad / truncate to $T_{\max} = 289$ packets (95th pct.)
Normalisation	per-window, per-channel z -score (over time)
<i>Architecture: ResNet1D</i> (143,174 parameters)	
Stem	<code>Conv1d(104$\rightarrow$64, $k=9$)</code> $\rightarrow$ Batch Normalization (BN) $\rightarrow$ ReLU $\rightarrow$ <code>MaxPool(2)</code>
Body	<code>ResBlock(64)</code> $\rightarrow$ <code>MaxPool(2)</code> $\rightarrow$ <code>ResBlock(64)</code> $\rightarrow$ <code>AdaptiveAvgPool1d(1)</code>
ResBlock	<code>[Conv1d(64$\rightarrow$64, $k=5$)$\rightarrow$BN$\rightarrow$ReLU$\rightarrow$Conv1d(64$\rightarrow$64, $k=5$)$\rightarrow$BN]</code> + skip, ReLU
Head	<code>Flatten</code> $\rightarrow$ <code>Dropout(0.4)</code> $\rightarrow$ <code>Linear(64$\rightarrow$6)</code>
<i>Augmentation: “device_robust”</i> (on-the-fly, $n_{\text{aug}} = 5$ replicas/epoch)	
Per-channel noise	additive Gaussian, per-channel std $\in [0.01, 0.04] \sigma_w$ (σ_w : window std)
Amplitude scaling	$\times [0.80, 1.20]$
Temporal shift	$\pm 8\%$ of the window
Time masking	zero a contiguous 10–20% time span
Subcarrier masking	zero 5–15 contiguous subcarriers per RX half
Device dropout	probability 0.25: zero one entire RX half
Mixup / label smoothing	$\alpha = 0.2$ / 0.05
EMA weights	decay 0.998
<i>Training</i>	
Optimiser	Adam, lr = 10^{-3} , weight decay = 10^{-3}
LR schedule	cosine annealing
Loss	cross-entropy (label smoothing 0.05) with mixup
Budget	40 epochs, batch size 32
Deployment fit	trained on all 198 clips; validated by leave-one-clip-out

A.4.5 Per-recording results on the continuous routines

Table A.20 gives the per-class breakdown behind the headline collapse reported in Section 5.3, and Table A.21 the per-recording scores. The two generic activities are recovered (`stand` (recall 0.92) and `walk` (recall 0.62)) but the four fine-grained counter actions

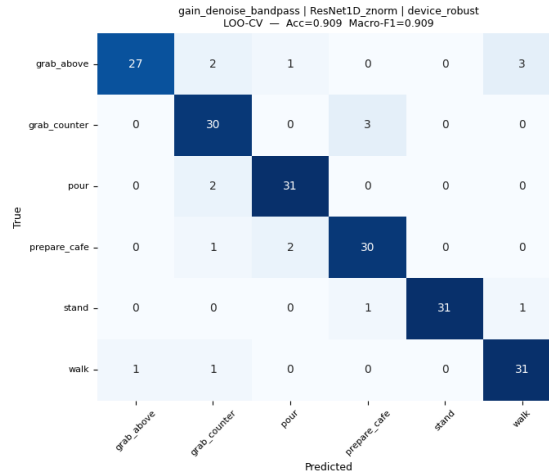


Figure A.7: Leave-one-clip-out confusion matrix of the deployed pipeline on the 198 controlled clips (33 per class; accuracy = 0.909, macro- F_1 = 0.909). In-distribution, every class is well separated.

collapse: **pour** reaches only 0.46 recall and is most often mistaken for **grab_counter**; **grab_counter** drops to 0.13; and **grab_above** is effectively never recognised (recall 0.00), its windows scattering into **walk**, **prepare_cafe** and **pour**. **prepare_cafe** attains a high recall (0.91) but a low precision (0.29): rather than being reliably recognised, it acts as a sink that absorbs windows from the other counter actions. The contrast with Figure A.7 is stark, **grab_above** scores 0.82 recall in-distribution but 0.00 on real routines.

Table A.20: Per-class performance of the deployed pipeline on the six manually labelled real routines (350 windows). Precision/recall/ F_1 are window-level. The two generic activities survive; the four fine-grained counter actions collapse.

Class	Precision	Recall	F_1	Support
stand	0.92	0.92	0.92	110
walk	0.69	0.62	0.65	81
pour	0.62	0.46	0.53	65
prepare_cafe	0.29	0.91	0.44	23
grab_counter	0.12	0.13	0.13	30
grab_above	0.00	0.00	0.00	41
Overall	acc = 0.589		macro- F_1 = 0.445	

Performance varies widely across the six recordings (Table A.21), from 0.226 on **teaB_1** to 0.513 on **cafeB_3**, but it does not separate by drink type: the coffee and tea routines interleave, and their mean macro- F_1 is almost identical (0.384 against 0.365). The higher accuracy of the coffee routines (0.650 against 0.472) is a matter of composition rather than difficulty, for the reason given below. Figure A.8 illustrates the typical failure, on **cafeA_1**: the **stand** core and the **walk** boundaries are recovered, while the short **grab_above**/**grab_counter**/**pour** block is swallowed by **prepare_cafe** and **walk**. The

same pattern holds on `cafeC_5` (Figure A.10) and, in its most fragmented form, on `teaB_1` (Figure A.11), the weakest recording of the six. Elsewhere the pipeline does recover `pour` as a coherent block, as in Figures A.9, A.12 and A.13.

Table A.21: Per-recording window-level scores of the deployed pipeline on the six labelled real routines. Macro- F_1 is averaged over the classes that occur in each recording: the tea routines contain neither `stand` nor `prepare_cafe`.

Recording	Windows	Accuracy	Macro- F_1
<code>cafeA_1</code>	79	0.633	0.307
<code>cafeB_3</code>	75	0.747	0.513
<code>cafeC_5</code>	79	0.570	0.332
<code>teaB_1</code>	41	0.317	0.226
<code>teaC_3</code>	35	0.514	0.382
<code>teaD_5</code>	41	0.585	0.487
All	350	0.589	0.445

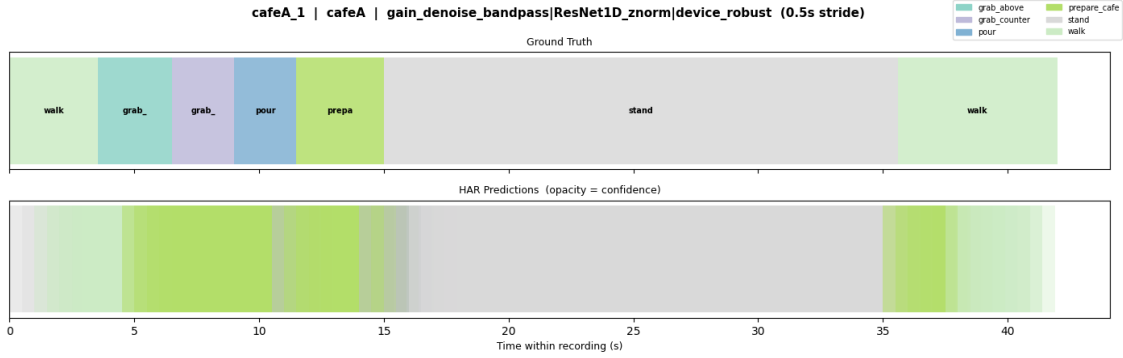
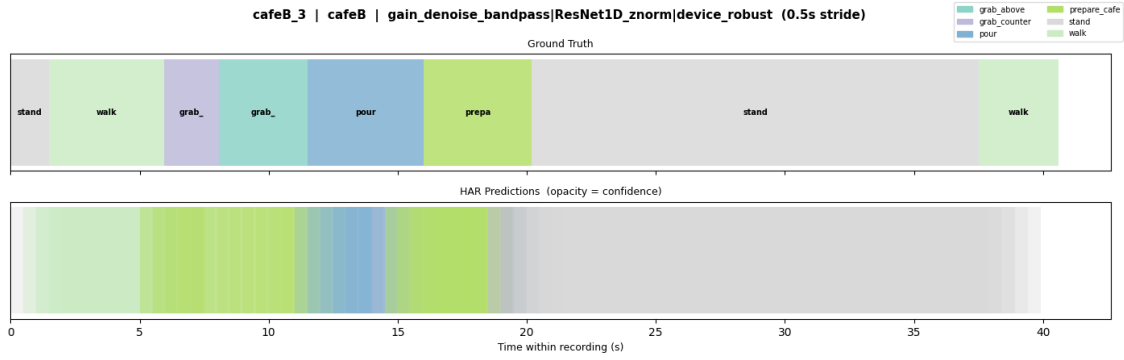
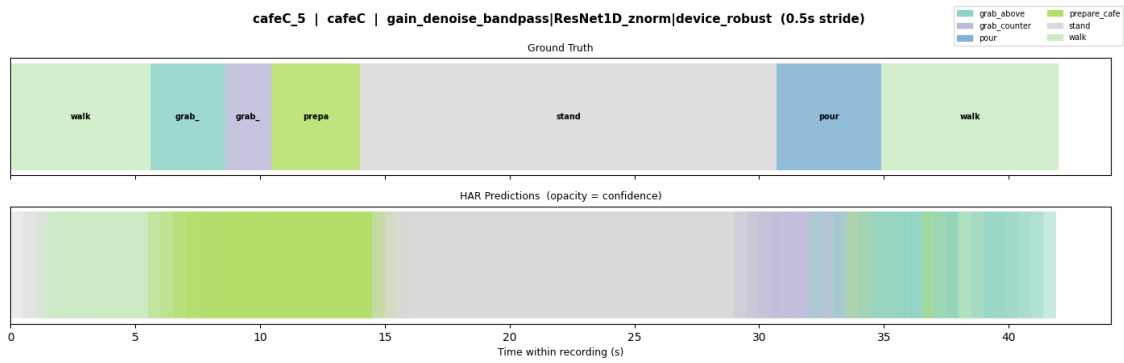


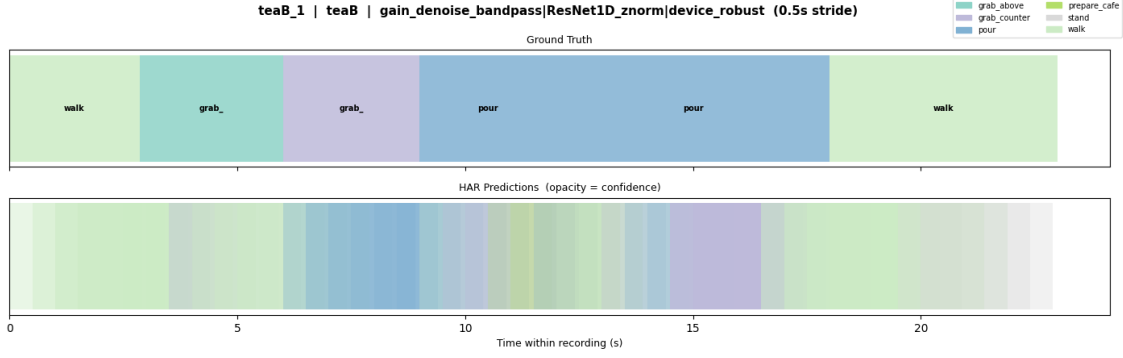
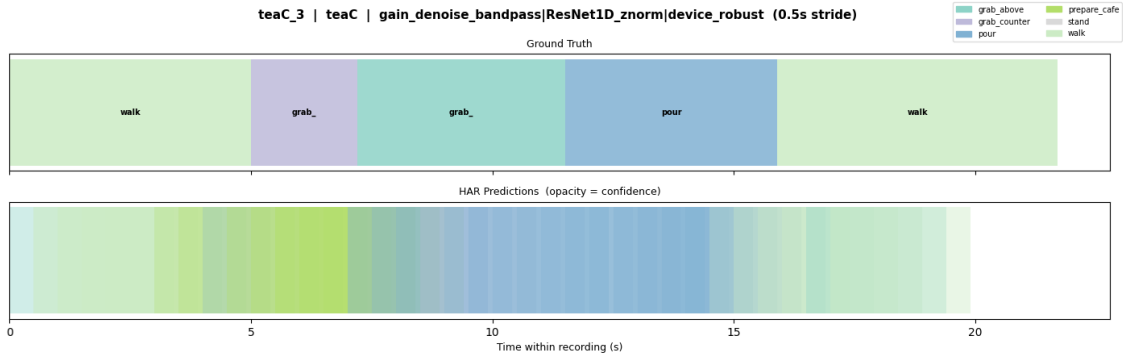
Figure A.8: Ground truth versus deployed-pipeline predictions (where opacity is the prediction confidence) for `cafeA_1`.

What the per-recording timelines reveal. The timelines show that the collapse is structured, not random. The fine-grained counter block is absorbed by whichever broad class dominates that drink’s routine: `prepare_cafe` in coffee and `pour` in tea. Tellingly, `prepare_cafe` is still predicted on 10 tea windows even though a coffee machine is never operated in a tea routine: a pure gesture-similarity false positive. Conversely, `pour` is recognised far better in tea (recall 0.56) than in coffee (0.27), because the tea routine contains a long kettle pour that dominates several consecutive windows whereas the coffee pour is brief, being a pour of sugar. The long static `stand` periods, which occur only in the coffee routines, account for 110 of the 350 scored windows and are recovered almost perfectly (recall 0.92); tea routines contain no `stand`, so their score rests entirely on the harder kitchen counter actions and `walk`. Finally, confidence is visibly lower on the tea recordings, with the weakest recording also producing the most fragmented prediction

Figure A.9: Ground truth versus deployed-pipeline predictions for `cafeB_3`.Figure A.10: Ground truth versus deployed-pipeline predictions for `cafeC_5`.

stream. The long kettle `pour` of the tea routines, by contrast, is captured as a coherent block, consistent with `pour`'s higher recall in tea than in coffee.

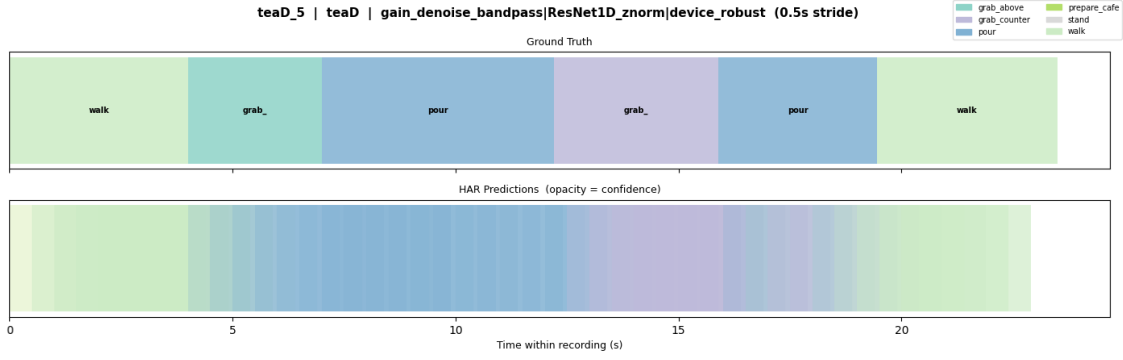
Next steps. Two directions follow directly. First, closing the controlled-to-real gap calls for substantially more training data per action. The model was trained on only 33 short clips per class, each executed in a fixed, scripted way, which cannot capture the natural variation that the same action takes on in a real routine—for example `walk` performed along different paths and across different parts of the room, or `grab` and `pour` carried out from slightly different positions and at different speeds. The aim here is not cross-subject or cross-room generalisation but simply richer coverage of each action itself: collecting many more samples that span this within-action variability is the most direct way to make the fine-grained classes separable in deployment rather than only on the controlled clips. Second, the current model classifies each 2.9s window on its own, ignoring the order in which actions occur. A natural improvement is to add a sequence model (a Markov chain or hidden Markov model) on top of the classifier, so that each window's decision combines the model's per-class probabilities with how likely each action is to follow the ones already seen, using prior knowledge of how coffee and tea routines typically unfold (for example, a `grab` action is far more likely to be followed by `pour` or `prepare_cafe` than by another `grab`). Encoding such behaviour-science priors to up- or down-weight the candidate classes is the most promising route to recovering the fine-grained actions that

Figure A.11: Ground truth versus deployed-pipeline predictions for `teaB_1`.Figure A.12: Ground truth versus deployed-pipeline predictions for `teaC_3`.

a purely per-window classifier cannot separate from amplitude alone, and is the natural continuation of this experiment.

A.4.6 Pipeline selection: Composite Validation Score and grid search

Because LOO on controlled clips overestimated deployment performance, candidates were instead ranked against the six labelled recordings via dense sliding-window inference (2.9s window, 0.25s stride). To grade overlapping windows (including boundary windows) we designed and used a Composite Validation Score (CVS). The window is assigned the class of the segment containing its centre; writing d_b for the distance from that centre to the nearest boundary between two labelled segments, a window near a transition is down-weighted by $\text{pos}_w = 0.5 + 0.5 \min(1, d_b/1.5s)$, and each window carries its softmax confidence. The two weighted accuracies this yields are the boundary-weighted accuracy (BWA), which averages correctness under the positional weights so that windows in a segment core count up to twice as much as those at a transition, and the confidence-weighted accuracy (CWA), which averages correctness under the confidences so that confidently asserted mistakes are penalised more heavily than hesitant ones. The intent is to grade a candidate the way deployment would, since neither a confident error nor a window straddling a transition (which no fixed-stride model can be expected to resolve) should count

Figure A.13: Ground truth versus deployed-pipeline predictions for `teaD_5`.

the same as a clean prediction inside a label segment. With $\mathbb{1}_i$ the correctness indicator,

$$\text{BWA} = \frac{\sum_i \mathbb{1}_i \text{pos_w}_i}{\sum_i \text{pos_w}_i}, \quad \text{CWA} = \frac{\sum_i \mathbb{1}_i \text{conf}_i}{\sum_i \text{conf}_i}, \quad \text{CVS} = 0.4 \text{macro-}F_1 + 0.3 \text{CWA} + 0.3 \text{BWA}.$$

The grid crossed preprocessing (gain + Hampel, with/without band-pass), architecture (CNN1D and ResNet1D variants with z -norm or PC1 removal, plus a mean+std-pooling ResNet1D) and six augmentation presets (standard, mixup-only, strong, label-smooth, device-robust, focal), each $\pm$ a binary action/transition soft gate (giving 60 configurations, each scored with and without the gate (120 rows)). The band-pass configurations swept the top (Table A.23), the soft gate never helped, and the deployed `ResNet1D_znorm | device_robust` row (CVS 0.541) was preferred over the marginally higher CNN1D row (CVS 0.545) because its errors were dominated by small temporal shifts rather than gross mislabels.

Every preset draws from the same pool of operations. The first seven are augmentations applied on the fly and independently to each window on every epoch access, while the rest act at the batch, loss or optimiser level:

- **Gaussian noise** (σ): zero-mean Gaussian noise added to every sample of the window. Its level is tied to the window's own scale: σ_w is the standard deviation of the entire window (one scalar over all 104 channels and all time steps) so σ sets a relative noise level and windows of different amplitude are perturbed proportionally. A global preset applies the same standard deviation $\sigma \sigma_w$ to every channel. A per-channel preset ($\dagger$) instead draws one standard deviation per channel, independently and uniformly from $[0.01, \sigma] \sigma_w$ and re-drawn on every epoch access, so within a single window some subcarriers are corrupted several times more heavily than others—mimicking the per-subcarrier noise-floor differences between the two receivers.
- **Amplitude scaling**: multiply the whole window by a random factor in the stated range.
- **Temporal shift**: shift the window in time by $\pm(\text{fraction}) \times T$, zero-filling the vacated end.
- **Temporal reversal**: flip the window along time with probability 0.30.
- **Time masking**: zero a contiguous span of 10–20% of the window length.

- **Subcarrier masking**: for each RX half independently, zero 5–15 contiguous subcarriers.
- **Device dropout** (p): with probability p , zero one entire (randomly chosen) RX half.
- **Mixup** (α): train on convex combinations of window/label pairs, $\lambda \sim \text{Beta}(\alpha, \alpha)$ - one λ per batch.
- **Label smoothing** (ε): soften the cross-entropy targets by ε .
- **Focal loss** (γ): weight the loss by $(1 - p_t)^\gamma$, down-weighting easy windows.
- **EMA**: keep an exponential moving average of the weights (given decay) as the final model.
- n_{aug} : number of augmented replicas of each training sample per epoch.

The six presets combine these as in Table A.22.

Table A.22: The six augmentation presets ($\dagger$ is per-channel gaussian noise)

Preset	Noise σ	Amp. scale	Shift	Masking / dropout	Mixup α	Label sm. ε	Focal γ	EMA	n_{aug}
standard	0.03	[0.85, 1.15]	$\pm 6\%$	-	-	-	-	-	4
mixup-only	0.03	[0.85, 1.15]	$\pm 6\%$	-	0.4	-	-	-	4
strong	0.05 $\dagger$	[0.70, 1.30]	$\pm 10\%$	reversal, time, subcarrier	0.4	-	-	-	5
label-smooth	0.03	[0.85, 1.15]	$\pm 6\%$	-	-	0.05	-	-	4
device-robust	0.04 $\dagger$	[0.80, 1.20]	$\pm 8\%$	time, subcarrier, device	0.25	0.05	-	0.998	5
focal	0.03	[0.85, 1.15]	$\pm 6\%$	-	-	-	1.5	-	4

Table A.23: Grid-search top rows (of 120), ranked by CVS. All rows use `gain_denoise_bandpass` preprocessing; only model | augmentation is shown. “binseg” = binary action/transition soft gate.

#	Configuration (model aug)	binseg	CVS	F_1	CWA	BWA
1	CNN1D_znorm strong	–	0.545	0.351	0.686	0.661
2	ResNet1D_znorm device_robust	–	0.541	0.352	0.690	0.643
3	CNN1D_znorm strong	yes	0.538	0.351	0.665	0.661
4	ResNet1Dstats_znorm strong	–	0.538	0.352	0.672	0.650
5	ResNet1Dstats_znorm label_smooth	–	0.536	0.375	0.679	0.608

This selection is the reason the per-recording numbers of Section A.4.5 are in-sample: the recordings were used to choose the pipeline. The applied goal was not classification accuracy per se but to feed the per-class softmax probability streams (exported alongside the predictions) into a downstream automata model of kitchen routines, which is why the inference pass logs full class probabilities rather than hard labels.

A.4.7 Exploratory and failed segmentation

Two attempts to handle transition windows did not yet generalise. A binary CNN gate (action vs transition/pause), following the ML-SEGM idea of Pham et al. [64], failed because the only available negatives were buffer / change-zone clips rather than genuine action-to-action transitions (hence the soft gate degrades rather than improves CVS in Table A.23). A second set of strategies was explored as alternatives to fixed-stride windowing: stand-energy skipping, a stand-anchor plus PELT change-point segmentation, an HMM Viterbi

post-processing pass with a behaviourally-informed transition matrix, and k -NN template matching with novelty rejection. The Markov/HMM direction is the most promising but requires real behaviour-analysis priors on activity-transition probabilities, which were not available here. However, the exported per-class probabilities of each window are intended to enable it downstream by the LURPA lab partner.

A.5 Cross-environment experiment: applied workflow

This appendix documents the applied workflow behind the cross-environment experiment of Section 5.4. This section contains material too detailed for the main text: the exact search space and component glossary of the stage-1 grid search (Sections A.5.1-A.5.4), the self-supervised pretraining setup (Section A.5.5), the complete specification of the final training recipe (Section A.5.6), and the precise definitions of the temporal-sampling variants (Section A.5.7). All parts share the same harness: the geometry-aligned three-room dataset of Section 4.8, leave-one-room-out (LORO) cross-validation with the room as the fold, and mean macro- F_1 over the three held-out rooms as the ranking metric..

A.5.1 Stage-1 grid search: shared harness and preprocessing glossary

Because the full cross-product of all grid axes is infeasible on the available CPU-only hardware, the stage-1 search (Section 5.4.1) was not executed as one monolithic grid. Each family is implemented as a single re-configurable grid harness (one search loop whose axis values are declared in a configuration block), and that harness was executed twice with different axis presets: first a screening execution at a fixed $W = 3.0$ s window covering the wide, unresolved axes, and then (after inspecting its results) an evidence-guided stage-2 execution that fixed the axes the screening had resolved and restored the window sweep. The axis tables below report the two presets side by side. The stage-2 preset is the configuration that survives in the accompanying repository, while the screening preset and its results are preserved in the timestamped result csv files.

Both grids draw their signal-conditioning options from the same cumulative pipeline (Section 4.4). A configuration name encodes which stages are switched on. Table A.24 defines every preprocessing identifier used in this appendix and in the results of Section 5.4.1.

Both grids also share a sampling-rate axis with three defined options: **orig** (windows cut on wall-clock time, packet stream left as recorded), **resamp100** (each window linearly resampled onto a fixed 100 Hz grid, yielding fixed-length, time-aligned tensors), and **drop_1t90** (discard windows whose effective packet rate falls below 90 Hz). The third option was discarded in preliminary runs (it is equivalent to **orig** minus $\approx 5\%$ of the windows) so the executed grids sweep **orig** vs. **resamp100** (features) or fix **resamp100** (deep stage-2, where fixed-length inputs are required anyway and the screening showed parity).

A.5.2 Stage-1 grid search: classical feature grid

The classical grid reuses the hand-crafted feature bank of Appendix A.4.2 unchanged (per-subcarrier statistical, spectral, autocorrelation and complexity descriptors, receiver-level summaries, cross-receiver correlations and raw-energy features, $(36K+8)\times 2 + (K+1) +$

Table A.24: Glossary of the preprocessing stages.

Identifier	Definition
raw	CSI amplitude as recorded: no gain correction, no denoising, no band-pass (true baseline).
gain	AGC gain correction only (amplitude $\times$ per-packet <code>compensate_gain</code>).
gain_denoise	Gain correction + Hampel despiking (window 9 packets, 3σ MAD).
gain_denoise_bandpass	As above + zero-phase Butterworth band-pass, 0.5–10 Hz (order 2), applied per full recording.
gd_bp_0.3_8	Same, but band limits 0.3–8 Hz.
gd_bp_0.5_15	Same, but band limits 0.5–15 Hz (only in the DL grid notebook).

Table A.25: Axes of the classical feature grid. Values in grey were eliminated by the screening pass and do not appear in stage-2.

Axis	Screening values ($W=3.0$ s)	Stage-2 values
Preprocessing	gain , gain_denoise , gain_denoise_bandpass	raw , gain , gain_denoise_bandpass , gd_bp_0.3_8
Calibration	off , mean_sub , std_norm , rms_norm , empty_rownorm	off
Window W	3.0 s	2.5, 3.0, 3.5 s
Rate	orig , resamp100	orig , resamp100
Post-processing	all , l1_top300 , perenv_all , perenv_l1_top300	same four
Classifier	logreg , rf , svm_rbf	same three
Configurations	360	288

$2 \cdot 2K = 4021$ features per dual-pair window ($K=52$) and sweeps six axes around it (Table A.25). The screening pass crossed $3 \times 5 \times 1 \times 2 \times 4 \times 3 = 360$ configurations (1,080 model fits); stage-2, with the calibration axis resolved and the window sweep restored, crossed $4 \times 1 \times 3 \times 2 \times 4 \times 3 = 288$.

Calibration axis. Every recording folder begins with an **empty** segment (the static room, no person present; Section 4.8). The calibration recipes use its per-subcarrier statistics to remove the static channel before feature extraction: **mean_sub** subtracts the empty-room mean per subcarrier; **std_norm** additionally divides by the empty-room standard deviation (movement expressed in units of the room’s baseline noise); **rms_norm** divides each subcarrier by its empty-room RMS energy (down-weighting statically strong subcarriers); **empty_rownorm** subtracts the empty-room mean and then z-scores each time frame across its subcarriers. All four underperformed **off** by a wide margin (screening means ≈ 0.39 – 0.44 vs. 0.467 macro- F_1): the empty reference captures one instant of a channel that drifts between sessions, so calibrating against it injects session noise rather than

removing room identity. Stage-2 therefore fixes `cal=off` and leaves domain alignment to the post-processing axis.

Post-processing axis. This axis controls what happens between the feature matrix and the classifier; its identifiers compose two independent choices.

- **Feature selection** (`all` vs. `l1_top300`). `all` passes the standardised 4021-dimensional vector unchanged. `l1_top300` inserts the ℓ_1 ranking selector of Section 4.7.1: a one-vs-rest logistic regression with ℓ_1 penalty ($C=0.05$, liblinear, class-balanced) is fitted on the standardised training-fold features, features are ranked by the mean absolute coefficient across the six one-vs-rest problems, and the top $k=300$ are kept. The selector sits after the `StandardScaler` inside the scikit-learn pipeline and is therefore re-fitted inside every LORO split on the training rooms only—the held-out room never influences which features survive, avoiding selection leakage.
- **Per-environment standardisation** (`perenv_*` prefix). Before the pipeline is applied, each room’s feature matrix is z-scored with its own mean and standard deviation—including the held-out room, which is standardised with its own statistics at test time. This is unsupervised domain alignment: it uses no labels and no cross-room statistics, so it introduces no leakage (any deployed system can compute its own room’s feature statistics from unlabelled observation, exactly as AdaBN does for the deep models in Section 4.7.6). Removing the room-specific offset and scale of every feature is what neutralises the static-multipath fingerprint that otherwise dominates hand-crafted descriptors.

Thus `perenv_l1_top300`—the setting appearing in the winning configurations of Section 5.4.1—reads: z-score each room’s features with its own statistics, then keep the 300 features top-ranked by the fold-internal ℓ_1 logistic regression, then classify.

Classifiers. Three standard learners close the pipeline, each preceded by a `StandardScaler`: `logreg` = logistic regression ($C=1.0$, class-balanced, 2000 iterations); `rf` = random forest (300 trees, class-balanced); `svm_rbf` = SVM with RBF kernel ($C=10$, γ =scale, class-balanced).

Table A.26 lists the five best stage-2 configurations. The pattern is unambiguous: all top rows use per-environment standardisation with the ℓ_1 top-300 selection and the 3.5s window, and the light `gain`-only preprocessing ties or beats the full band-pass—for hand-crafted features, whose spectral descriptors implement their own frequency selectivity, the per-environment standardisation does the domain-alignment work that the band-pass does for the deep models.

A.5.3 Stage-1 grid search: deep-learning grid

The deep grid feeds the stacked $T \times 104$ dual-pair window directly into the one-dimensional convolutional models of Section 4.7.2 and sweeps six axes (Table A.27). The architecture axis instantiates that family as ResNet1D, its mean-and-standard-deviation-pooling variant ResNet1DStats (Section 4.7.2), and TCN1D; the shallow CNN1D baseline was not carried into this grid. The screening pass crossed $3 \times 8 \times 1 \times 1 \times 3 \times 1 = 72$ configurations,

Table A.26: Classical feature grid, stage-2: top-5 of 288 configurations (mean macro- F_1 over LORO folds; all use `cal=off`).

Preproc.	W	Rate	Post-proc. clf.	Mean F_1	Worst room
gain	3.5	orig	perenv_l1_top300 rf	0.558	0.532 (sb207)
gain	3.5	orig	perenv_l1_top300 svm_rbf	0.554	0.545 (sb207)
gain_denoise_bandpass	3.5	resamp100	perenv_l1_top300 svm_rbf	0.553	0.484 (sb207)
gain_denoise_bandpass	3.5	orig	perenv_l1_top300 svm_rbf	0.549	0.499 (sb207)
gain_denoise_bandpass	3.5	resamp100	perenv_l1_top300 logreg	0.541	0.467 (sb207)

Table A.27: Axes of the deep-learning grid. Values in grey were eliminated by the screening pass.

Axis	Screening values ($W=3.0$ s)	Stage-2 values
Preprocessing	gain, gain_denoise_bandpass	raw, gain_denoise, gain_denoise_bandpass, gd_bp_0.3_8, gd_bp_0.5_15
Window transform	znorm, pc1_znorm, pc2_znorm, diff_znorm, pc1_diff_znorm, empty_std, empty_rms, empty_rownorm	znorm, diff_znorm, pc1_znorm
Window W	3.0 s	2.5, 3.0, 3.5 s
Rate	resamp100	resamp100
Architecture	TCN1D, ResNet1D, ResNet1DStats	TCN1D, ResNet1DStats
Augmentation	none	none, standard
Configurations	72	180

its main job being to resolve the eight-way per-window transform axis and the architecture; stage-2, with both resolved, crossed $5 \times 3 \times 3 \times 1 \times 2 \times 2 = 180$ configurations centred on the band-pass limits, the window duration and the augmentation.

Per-window transform axis. Applied to each $T \times 104$ window after windowing and before the network, this axis extends the normalisation modes of Section 4.7.4 with empty-room-calibrated variants; Table A.28 defines all eight. The screening verdict was clear-cut: the plain statistical detrends (`diff_znorm` 0.529, `znorm` 0.525, `pc1_znorm` 0.503 best-config macro- F_1) dominated, while every `empty_*` calibration collapsed to mean macro- F_1 0.24–0.28 (best configuration ≤ 0.38)—the same session-drift failure mode as the feature-side calibration axis, and the reason both stage-2 grids abandon the empty-room reference entirely.

Augmentation axis. `none` trains on the windows as-is; `standard` draws, for every epoch access, additive Gaussian noise (std $0.03 \sigma_w$, global), random amplitude scaling in $[0.85, 1.15]$ and a temporal shift of $\pm 6\%$ of the window, with two augmented replicas per training sample—the jitter-and-scale preset later shown by the ablation of Section 5.4.4 to be a no-op at best.

Table A.28: Glossary of the per-window transforms of the deep grid (applied per window; “z-score” is always per channel over time). The `empty_*` modes use per-folder empty-room statistics ($\mu_e, \sigma_e, \text{rms}_e$ per subcarrier).

Identifier	Definition
<code>znorm</code>	Per-channel z-score (baseline; Section 4.7.4).
<code>pc1_znorm</code>	Project out the window’s first principal component in channel space (the shared static-multipath background), then z-score. Being a cross-channel rank projection, it is not cancelled by the subsequent z-score.
<code>pc2_znorm</code>	Same with the top-2 principal components (stronger background removal).
<code>diff_znorm</code>	Frame-to-frame first difference (high-pass detrend), then z-score.
<code>pc1_diff_znorm</code>	PC1 removal, then first difference, then z-score.
<code>empty_std</code>	$(w - \mu_e)/\sigma_e$: movement in units of the empty-room baseline noise; no trailing z-score (it would cancel the calibration).
<code>empty_rms</code>	w/rms_e : normalise each channel by its empty-room energy.
<code>empty_rownorm</code>	Subtract μ_e , then z-score every time frame across its subcarriers.

Table A.29: Deep-learning grid, stage-2: top-5 of 180 configurations (mean macro- F_1 over LORO folds; all use `resamp100`).

Preproc.	Transform	W	Model	Aug.	Mean F_1	Acc.
<code>gd_bp_0.3_8</code>	<code>diff_znorm</code>	3.5	TCN1D	none	0.579	0.592
<code>gd_bp_0.3_8</code>	<code>diff_znorm</code>	3.5	TCN1D	standard	0.569	0.582
<code>gain_denoise_bandpass</code>	<code>diff_znorm</code>	3.5	TCN1D	none	0.567	0.583
<code>gd_bp_0.5_15</code>	<code>pc1_znorm</code>	3.5	ResNet1DStats	none	0.560	0.571
<code>gd_bp_0.5_15</code>	<code>znorm</code>	3.5	TCN1D	standard	0.559	0.576

Table A.29 lists the five best stage-2 configurations. Every top row uses the 3.5 s window and a band-passed input; TCN1D with `diff_znorm` on the 0.3–8 Hz band leads at 0.579, and the `standard` augmentation never improves on `none`. A small follow-up sweep of 66 further runs was carried out around this winner: 25 band-limit configurations (all 5×5 combinations of low cut-offs 0.2–1.5 Hz and high cut-offs 7–15 Hz, at $W=3.5$ s with `diff_znorm` and TCN1D), followed by 41 runs that fixed the band at the resulting best limits and re-checked the window duration, the sampling rate, the per-window transform, the architecture and the augmentation against it. Together they nudged the best scratch-TCN score to 0.588 (0.2–10 Hz, `diff_znorm`, $W=3.5$ s); this verdict, however, did not transfer to the final recipe of Section 5.4.4—on the z-norm/augmentation-suite model the 0.2 Hz low cut hurt—so the 0.5–10 Hz band of Section 4.4 was retained.

A.5.4 Stage-1 grid search: single-pair companion grids

Both grids were repeated per link, feeding only one receiver’s $T \times 52$ window through the identical search space. Table A.30 reports the best configuration per link and family. Two

Table A.30: Single-pair companion grids: best configuration per link and family (mean macro- F_1 over LORO folds, $W=3.5$ s screening budget).

Link	Family	Best configuration	Mean F_1
CB00	Deep	gain_denoise_bandpass znorm TCN1D	0.541
CB00	Features	gain perenv_l1_top300 RF	0.539
AC84	Deep	gain_denoise znorm TCN1D	0.430
AC84	Features	gain perenv_l1_top300 RF	0.423

observations carry into the main text: the winning configuration types are similar to the ones in the dual grids (perenv_l1_top300 features, band-passed znorm/TCN1D deep), and the two links are far from interchangeable—link CB00 loses only ≈ 4 points of f_1 to the dual input, while link AC84 collapses by ≈ 14 , the geometry gap analysed further in Section 5.4.6.

A.5.5 Self-supervised pretraining details

The SSL experiments of Section 5.4.2 use the ESP-Fi HAR dataset of Section A.2.1 as the pretraining dataset: its samples are single-link, 52-subcarrier, ≈ 100 Hz CSI amplitude, i.e. format-identical to one thesis link after removal of the 12 zero subcarriers, so a single-link 52-channel encoder can be pretrained on ESP-Fi and reused unchanged on either receiver downstream. Every window is z-scored per subcarrier and resampled to a fixed length before entering the encoder, which makes the pretraining agnostic to the downstream window duration (pretrain once, sweep W only downstream). Also, the SSL experiments use unlabelled double-pair recordings from the cross-day experiments.

First iteration (transformer encoder). Three SSL methods sharing one transformer backbone were compared: a masked autoencoder (MAE; masked temporal-patch reconstruction), SimCLR (NT-Xent contrastive loss), and BYOL (EMA target network). Fine-tuned under LORO, MAE was the best of the three and lifted the transformer by ≈ 18 points over the same architecture trained from scratch, but the pretrained transformer still trailed a TCN trained from scratch (Table 5.5).

Second iteration (masked reconstruction of the TCN trunk). The pretext was therefore moved onto the TCN trunk itself, in the style of masked convolutional pretraining: random contiguous time spans of 8–24 steps are zeroed until 50% of the window is masked, a small transposed-convolution decoder reconstructs the input, and the L_2 loss is computed on the masked steps only (40 epochs; dataset capped at 14,000 windows; masked-reconstruction MSE decreasing from 0.90 to 0.80). The pretrained trunk is then fine-tuned with the exact scratch recipe on the siamese pair model of Section 4.7.3. Diagnostics: a frozen-trunk linear probe scores 0.499 vs. 0.394 for a randomly initialised trunk (the representation is genuine), a discriminative learning-rate variant (lower trunk LR) hurt (0.522), and after full fine-tuning the SSL advantage shrinks to $\approx +1$ point—the

basis for the ablation verdict of Section 5.4.4 that pretraining is redundant under the final recipe.

A.5.6 Final training recipe: full specification

Table A.31 specifies every stage of the final cross-environment pipeline—the configuration behind the headline results of Sections 5.4.5 and 5.4.6. Two deliberate contrasts with the kitchen recipe are worth flagging: the AGC gain correction is omitted (Section 5.4.6), and the two links are processed by a weight-shared siamese trunk with late fusion instead of early channel concatenation.

A.5.7 Timestamp-grid resampling variants

The sampling study of Section 5.4.5 holds the recipe of Table A.31 fixed and varies only the step that converts a timestamp-cut window into a fixed-length tensor; Table A.32 defines every variant. Because the TCN trunk is fully convolutional with global mean+std pooling, all variants use the identical architecture, and the training time-mask span is scaled with L so the masked fraction of the window stays constant. Two technical notes. First, the FFT resampler implicitly treats each window as one period of a repeating signal; because the window’s start and end values generally differ, this creates small oscillation artefacts near the window edges, and their effect is included in what the `L160_fft` variant measures. Second, in the `noresamp_*` variants the windows are zero-padded to a common length after per-channel z-normalisation, so that the appended zeros do not distort each window’s mean and standard deviation; this follows the convention of the original grid-search pipeline of Section 5.4.1.

A.5.8 The `no_action` class: construction and full results

This section completes the label-set extension of Section 5.4.7 with the source-data breakdown, the per-fold results and the per-class scores behind the summary table of the main text.

Source material and window construction. The class required no additional acquisition: each recording session of the cross-environment campaign (Section 4.8) already contains one short capture of the room with nobody performing an activity, stored in the segment archives under the action label `empty` and discarded by every earlier stage of the experiment. Table A.35 gives the per-room breakdown and the resulting window pools. Apart from the reduced stride motivated in the main text, the only implementation detail worth recording is the subsampling rule, which differs between the two granularities: at five classes the pool is reduced to the balance target by an evenly-spaced selection over the time-ordered windows, which avoids retaining near-duplicate neighbours, whereas at seven classes it is a seeded random subsample of the pool. Training and held-out test sets are therefore exactly class-balanced, with $7 \times 70 = 490$ test windows per fold at seven classes and $5 \times 140 = 700$ at five. Models, seeds, LORO folds, AdaBN and the five deployment configurations are unchanged from Section 5.4.6.

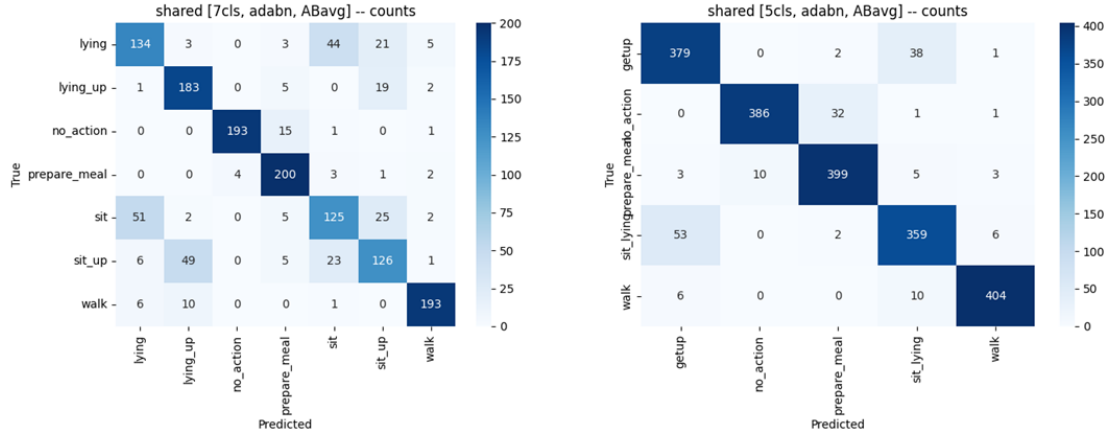


Figure A.14: Pooled LORO confusion matrices of ABavg late fusion for both granularities.

Per-fold results. Table A.36 gives the per-room macro- F_1 behind the means of Table 5.9. The fold pattern of the six-class experiment is reproduced: at seven classes **mc208** is the hardest room for the late-fusion model and **mf207** the easiest, while at five classes **sb207** takes over as the hardest. This is a reminder that the LORO means average over markedly unequal environments.

Per-class behaviour. Table A.37 reports the per-class F_1 of all five configurations, pooled over the three held-out rooms—computed on the concatenated LORO predictions rather than averaged over folds, which is why the macro row differs slightly from the fold means above. **no_action** is the best-recognised class of the seven-class set in four of the five configurations and level with **walk** in the fifth while at five classes the merging of the posture pairs lifts **walk** slightly above it. In the winning configuration it reaches $F_1 = 0.948$ (precision 0.980, recall 0.919) at seven classes and $F_1 = 0.946$ (precision 0.975, recall 0.919) at five.

Confusion structure. Figure A.14 shows the pooled confusion matrices of the best configuration for both granularity problems. In the winning configuration, 193 of the 210 **no_action** windows are recovered and 15 of the remaining 17 go to **prepare_meal**, while only 4 windows in the entire test set are wrongly declared idle. On the other hand, Figure A.15 shows the same view for the best single-pair deployment—the shared trunk tested on link B alone—which halves the deployed hardware by dropping one TX-RX pair. Removing one pair costs 3.5–4.5 points of overall accuracy and leaves the idle state itself with an f_1 above 0.90.

A.5.9 Real-time deployment: rooms, adaptation and confusion matrices

This section carries the material behind Section 5.4.8: the two deployment rooms, what the install-time adaptation is worth in each of them, and the live confusion matrices.

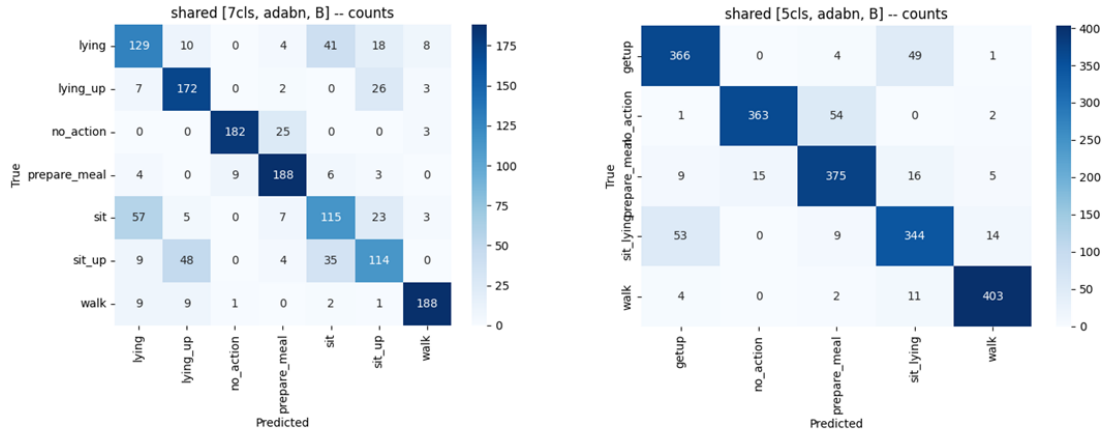


Figure A.15: Pooled LORO confusion matrices of the shared trunk tested on link B alone, for both granularities.

The live program. The system is a single Python script, `realtime_har.py`, which reuses the preprocessing functions of the offline pipeline verbatim rather than re-implementing them. For the live program to work, all RXs need to be connected to the laptop using a USB cable. Then, the program identifies each board by the last four hex digits of its MAC (`..AC84` = link A, `..CB00` = link B). Also, it maps both hardware clocks onto the PC monotonic clock, which is what lets the timestamp-grid resampling of Section 5.4.5 operate on two independent boards. In addition, it loads the three cached seed networks, trained on all three recorded rooms and both links pooled. Then, it runs the cued calibration sweep and applies AdaBN per link and per seed, and then it runs the inference loop. One cycle is a 1.5s ready banner, a beep, the 3.5s classified window, a double beep, a short settle, the prediction, and the operator's true-label button. The Hampel filter and the band-pass are applied not to the window itself but to the surrounding $[t_{\text{end}} - 60\text{s}, t_{\text{end}} + 2\text{s}]$ of stream, the past context standing in for the recording-level filtering of Section 4.4 and the 2s tail pushing the `filtfilt` edge transient out of the classified window. The conditioned window is then z-normalised and interpolated onto the 160-point grid exactly as in training. Both reported sessions used one calibration round, i.e. seven cued windows per link, and a 2s tail.

The two live rooms. Figure A.16 shows the two environments, neither of which appears in the training data. Room `e212` is a university room of CentraleSupélec. The living room is a different proposition entirely, containing various furniture. Each room imposes its own static multipath on top of the motion the model is meant to read, which is why performance might move slightly from room to room.

What the install-time AdaBN is worth. Because the program dumps the exact tensors that entered the network, a session can be re-scored offline under a different batch-norm regime with the inputs held fixed. Table A.38 does this for both sessions: the same windows, the same weights, the same fusion, and the only difference is whether the batch-norm statistics come from the calibration sweep performed in the room or from the pooled



Figure A.16: The two live deployment rooms. Left: room e212, 10 August, double pair. Right: the living room, 23 July, single pair. Green circles showcase the TX and red ones the RX.

training rooms. Keeping the training statistics costs 8.6 accuracy points and 11.4 macro- F_1 points on the double-pair session and 3.3 / 6.9 on the single-pair one. In both sessions the loss is larger in macro- F_1 than in accuracy, i.e. the unadapted model does not degrade evenly but collapses on particular classes, which is the behaviour a domain shift can produce.

Live confusion matrices. Figures A.17 and A.18 gives both matrices. The double-pair session was run as a balanced sweep of five windows per class; the single-pair session was deliberately unbalanced, being a free session in a home in which **walk** occurs far more often than a lying transition. Six of the seven double-pair errors sit inside the posture quartet, **lying** being the worst class with three of its five windows predicted as **sit**—the chair-versus-floor confusion of Section 5.4.3. The single-pair session repeats that pattern and adds a walk-specific failure (3 **walk** windows predicted **lying_up**, one **lying**): when the operator begins walking a beat after the cue, the first second of the window is stationary, and the training dataset contains almost no **walk** window with a stationary onset, because those windows were cut from longer continuous recordings at a 2.5 s stride. This is an artefact of the cued protocol rather than a model defect, but it is exactly the boundary effect the continuous evaluation of Section 5.4.9 is designed to investigate. Two extra insights are found: **no_action** was correct on every window of both sessions, including windows in which the subject stood still inside the room rather than leaving it, so what the class detects is the absence of motion; and **lying** and **lying_up** were recognised in the living room when performed on a sofa, a piece of furniture that appears nowhere in the training data.

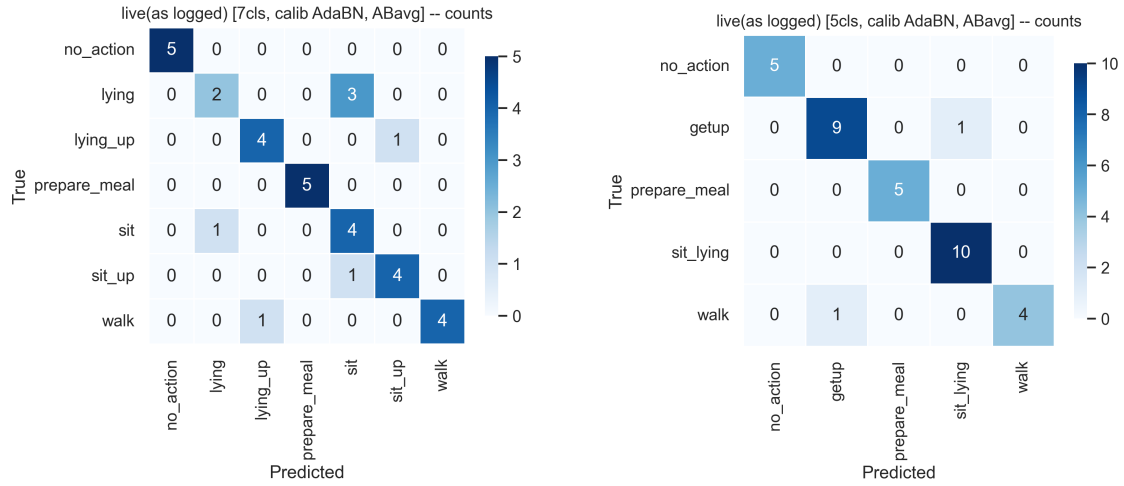


Figure A.17: Live confusion matrices of 10 August session in room e212 using the double pair. Seven-class granularity on the left and five-class on the right.

A.5.10 Continuous recordings: mode specification and remaining timelines

This section provides the supporting details for Section 5.4.9. It explains the differences between the real-time continuous mode and the cued mode (Appendix A.5.9), details the actions in each recording, and presents seven additional prediction timelines along with their limitations. This test was also performed in room e212, seen in Figure A.16.

The continuous mode. The continuous mode reuses the acquisition, the calibration sweep and the inference stack of Appendix A.5.9 unchanged, and replaces only the inference cycle. Instead of a beep-delimited 3.5s window, each cycle records a fixed 20s of freely performed activity between an opening and a closing beep, and classifies it afterwards with a sliding window. Per link, the recording is taken as a single snapshot spanning $[t_{\text{start}} - 60\text{s}, t_{\text{end}} + 2\text{s}]$ and passed through one preprocessing call—Hampel filter and band-pass—so that the filter context matches the recording-level filtering of the training path (Section 4.4) and the `filtfilt` edge transients fall outside the classified span. The windows are then cut from that conditioned signal at every start in $[0, 20 - W]$ spaced 0.25s apart, giving 67 windows per 20-sec recording. Each window is z-normalised and interpolated onto the 160-point timestamp grid as in training, scored by the three seed networks of the link, and the per-link probabilities are averaged over the links. Both panels of every figure below are that output—the timeline of predicted labels with confidence as opacity, above the per-class probabilities plotted at each window centre—which is why the leading and trailing $W/2$ of each recording is shaded: no window centre falls there.

Two properties of this design limit what Section 5.4.9 can claim, and a third makes up for them. The mode saves nothing, since it was built as a live test. As a result, these recordings are not re-scored offline. The operator performs the chain freely, so only the order of the activities is known, not the moment at which each one starts and ends.

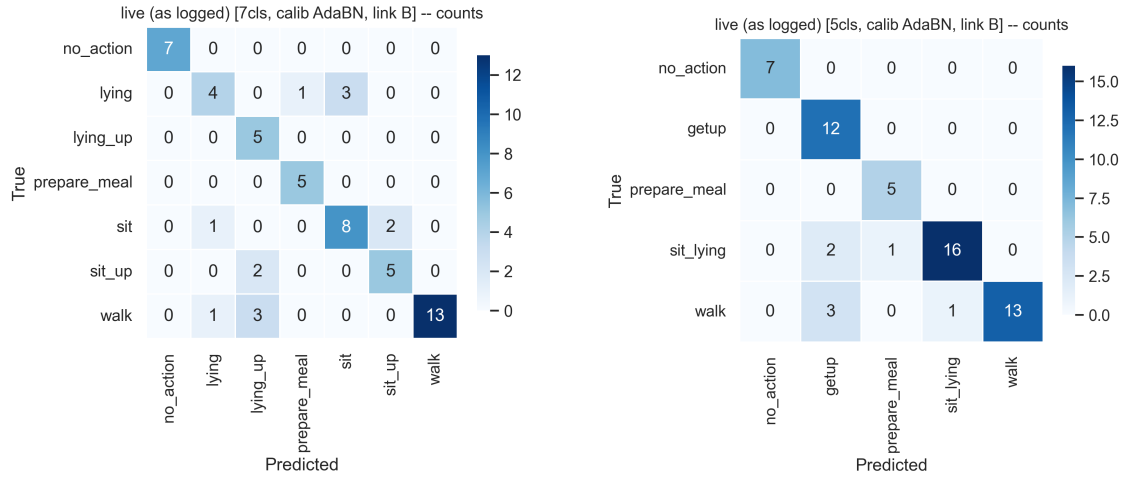


Figure A.18: Live confusion matrices of the 23 July session in the living room using the single pair. Seven-class granularity on the left and five-class on the right.

There is therefore no per-window ground truth, and no score is computed anywhere in this section. What can be done instead is to read each timeline against the sequence that was performed, and that turns out to be enough, because the model is good enough to make the boundaries visible on its own: the predictions do not run together into one undifferentiated block but break into blocks of their own, and the edges of those blocks line up with the points at which the subject changed activity. Every recording can therefore be read off directly—which activities the model caught, in which order, and which it confused or missed.

Suppressing the boundary runs. The false predictions described in Section 5.4.9 are brief and have usually low confidence scores, whereas the correct predictions are longer and highly confident. Therefore, the probability curves alone provide enough information to distinguish between them: a deployed system does not need to act on individual windows. Requiring a minimum number of consecutive agreeing windows above a confidence threshold before an activity event is emitted would discard most of these runs while leaving the plateaus untouched, at the cost of a detection latency an activity-logging application can absorb. This is a direction rather than a result: no such rule is implemented or evaluated here, and it is left as future work.

Scope. Three limits bound the reading. The evaluation covers nine 20s recordings performed by one subject in one room, so it demonstrates that the recogniser survives continuous operation rather than providing another estimate of its accuracy. Without boundary annotations the observations are read off the timelines rather than computed, so they identify failure modes without quantifying their frequency. Obtaining that quantification means annotating continuous recordings second by second against video, as was done for the six kitchen routines of Section A.4.1, and is the natural next step. Finally, the recordings were deliberately activity-dense, chaining four to six activities into 20s with only brief standstills, whereas a real deployment is idle for the overwhelming majority of

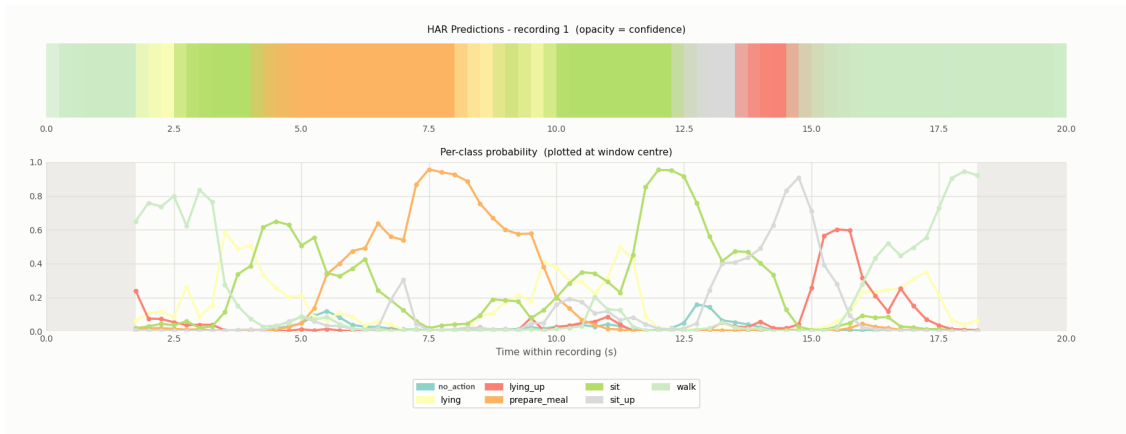


Figure A.19: Continuous recording 1 sequence: **walk** → **prepare_meal** → **sit** → **sit_up** → **walk**. All actions are identified, but some false actions are predicted between transitions; e.g., before **prepare_meal**, a lying and sit action is falsely predicted.

its operating time. Therefore, the boundary errors are encountered here at a far higher rate than a monitor in a home would encounter.

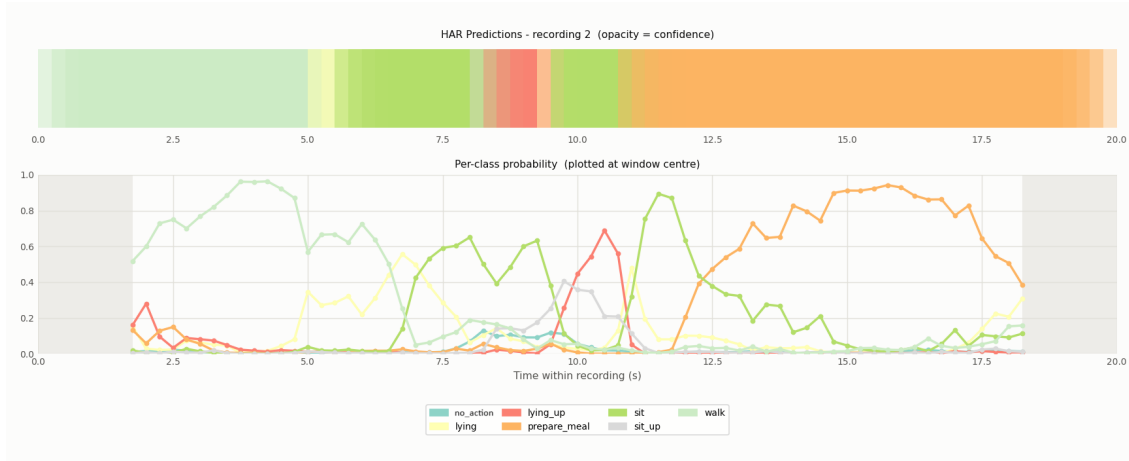


Figure A.20: Continuous recording 2: walk $\rightarrow$ lying $\rightarrow$ lying_up $\rightarrow$ prepare_meal. Again, all actual actions are correctly identified. However, there is a false prediction of sitting before prepare_meal.

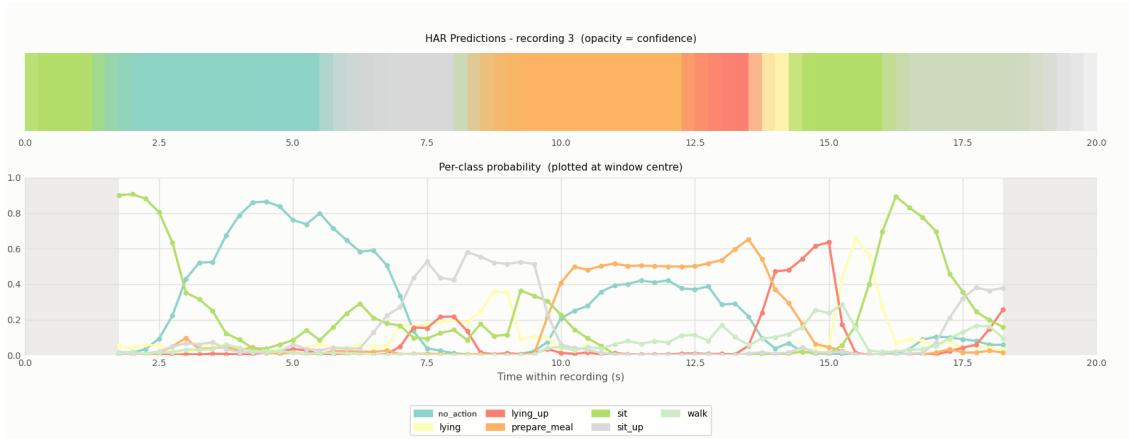


Figure A.21: Continuous recording 3: sit $\rightarrow$ no_action $\rightarrow$ sit_up $\rightarrow$ prepare_meal $\rightarrow$ lying $\rightarrow$ lying_up. The first half of the sequence is perfectly predicted. However, right after prepare_meal there is a false lying_up prediction.

Table A.31: Full specification of the final cross-environment pipeline (siamese shared-trunk TCN, late fusion), used for the results 5.4.6.

Component	Setting
<u>Input and channel layout</u>	
Subcarriers	64 reported; 12 zero subcarriers removed $\Rightarrow$ 52 kept per link
Links	2 (AC84, CB00), each a separate $T \times 52$ stream into the shared trunk
Signal	CSI amplitude only, nominal 100 Hz (true rate wobbles ≈ 90 –110 Hz)
<u>Preprocessing (per full recording)</u>	
Gain correction	none (negative on this pipeline; Section 5.4.6)
Hampel filter	window = 9 packets, threshold = 3σ (MAD, scale 1.4826)
Band-pass	zero-phase Butterworth (order 2), 0.5–10 Hz, recording-level
<u>Windowing and normalisation</u>	
Window	$W = 3.5$ s, cut on packet timestamps; transitions one window each; walk/prepare_meal stride 2.5 s (six-class), 0.5 s (four-class and balancing samples per room)
Normalisation	per-window, per-channel z-score (over time)
Resampling	timestamp-grid interpolation to $L = 160$ points (uniform wall-clock grid; time-aligns the two links; Section A.5.7)
<u>Architecture: siamese TCN trunk (weights shared across links)</u>	
Stem	Conv1d (52 $\rightarrow$ 64, $k=7$) $\rightarrow$ BN $\rightarrow$ ReLU $\rightarrow$ MaxPool (2)
Body	4 residual blocks of dilated convolutions, dilations 1, 2, 4, 8; each block: Conv1d (64 $\rightarrow$ 64, $k=3$, dil. d) $\rightarrow$ BN $\rightarrow$ ReLU $\rightarrow$ Conv1d (64 $\rightarrow$ 64, $k=3$, dil. d) $\rightarrow$ BN, then + skip and ReLU
Pooling	concatenated temporal mean + std $\Rightarrow$ 128-d descriptor per link
Head	Dropout (0.4) $\rightarrow$ Linear (128 $\rightarrow n_{\text{cls}}$) shared across links, applied to each link's descriptor separately
Fusion	late: average of the two links' softmax probabilities (single-link deployment uses one link's probabilities unchanged)
<u>Augmentation (on-the-fly, during training)</u>	
Subcarrier masking	with $p = 0.7$: zero 3–8 randomly selected subcarriers
Time masking	with $p = 0.7$: zero a contiguous span of 8–20 steps (fixed at $L=160$)
Same-class mixup	per batch with $p = 0.5$: blend with a same-class partner, per-sample $\lambda \in [0.6, 0.95]$; partner may come from the other link (Section 5.4.4)
Base augmentation	none (jitter/scale dropped by the ablation)
<u>Training</u>	
Optimiser	AdamW, lr = 10^{-3} , weight decay = 10^{-3}
Schedule / loss	cosine annealing; cross-entropy with label smoothing 0.05
Budget	30 epochs, batch size 32; no SSL pretraining (random init)
Seed ensemble	3 seeds {0, 1, 2}, softmax probabilities averaged
<u>Test-time adaptation (label-free)</u>	
AdaBN	reset all BN running statistics; two forward passes over the unlabelled test-room windows (cumulative averaging), separately per test link; weights untouched

Table A.32: Definitions of the temporal-sampling variants of Table 5.7. All operate on Hampel-cleaned windows; “recording-level band-pass” is the 0.5–10 Hz filter of Table A.31.

Variant	Definition
L{96,128,160,320}_lin	Recording-level band-pass; linear interpolation onto L uniformly spaced <u>index</u> positions (effective rate $\approx 27\text{--}91$ Hz; $L=320 \approx$ no downsampling).
L{96,128,160}_tstamp	Recording-level band-pass; linear interpolation onto a uniform <u>wall-clock</u> grid of {96,128,160,320} points built from the true packet timestamps—removes packet-rate jitter and exactly time-aligns the two links’ windows.
L160_fft	Recording-level band-pass; Fourier-domain (band-limited) resampling to 160 points instead of linear interpolation.
L160_bp_after	Window cut from the Hampel-only stream; resampled to 160 points; band-pass applied <u>on the uniform grid</u> (where $f_s = 45.7$ Hz is exact by construction); then z-norm. Tests whether the recording-level filter’s $f_s=100$ Hz assumption matters.
noresamp_pad	No resampling: as-recorded packets of the timestamp-cut window, zero-padded/truncated to 352 steps (≈ 3.5 s at ≈ 100 Hz); recording-level band-pass as usual.
noresamp_bp_win	As above, but the band-pass is applied <u>per window</u> after cutting instead of on the whole recording—isolates filter edge effects.

Table A.33: Classification report for the best 6-class activity recognition model (AdaBN ensemble, links=ABavg, pooled). The same model achieves a fold-mean macro-F1 of 0.766.

Activity	Precision	Recall	F1-Score	Support
Lying	0.68	0.67	0.67	210
Lying Up	0.72	0.90	0.80	210
Prepare Meal	0.95	0.98	0.96	210
Sit	0.67	0.60	0.63	210
Sit Up	0.69	0.60	0.64	210
Walk	0.95	0.92	0.93	210

Table A.34: Classification report for the best 4-class activity recognition model (AdaBN ensemble, links=ABavg, pooled). The same model achieves a fold-mean macro-F1 of 0.918.

Activity	Precision	Recall	F1-Score	Support
Get Up	0.86	0.89	0.87	420
Prepare Meal	0.98	0.97	0.97	420
Sit/Lying	0.87	0.85	0.86	420
Walk	0.97	0.96	0.97	420

Table A.35: `no_action` source material per room, and the resulting window pool. The pool is the number of 3.5s windows obtainable at the stated stride; the balance target is the per-class window count the seven-/five-class datasets are balanced to, and is met with a small surplus in every room, so `no_action` is always subsampled down to match the other classes.

Room	Source segments			7-class (0.5 s stride)		5-class (0.25 s stride)	
	4 s	10 s	Total dur.	Pool	Target	Pool	Target
mc208	10	5	90 s	76	70	151	140
sb207	8	5	82 s	73	70	146	140
mf207	6	5	74 s	71	70	142	140
Total	24	15	246 s	220	210	439	420

Table A.36: `no_action` experiment, per-fold results: AdaBN + 3-seed ensemble macro- F_1 on each held-out room. Columns are the five deployment configurations of Section 5.4.6; the means are the figures reported in Table 5.9.

		Single-link training		Shared trunk (A+B)		
Task	Held-out room	A	B	A	B	ABavg
<u>7-class</u>						
	mc208	0.634	0.690	0.651	0.672	0.680
	sb207	0.613	0.728	0.615	0.782	0.801
	mf207	0.634	0.716	0.811	0.747	0.850
	<u>mean</u>	0.627	0.711	0.692	0.734	0.777
<u>5-class</u>						
	mc208	0.870	0.906	0.899	0.901	0.930
	sb207	0.783	0.851	0.738	0.879	0.898
	mf207	0.837	0.829	0.906	0.864	0.925
	<u>mean</u>	0.830	0.862	0.848	0.881	0.917



Figure A.22: Continuous recording 5: `no_action` → `prepare_meal` → `walk` → `sit`. Right after the idle state, there is confusion regarding the transition to `prepare_meal`.

Table A.37: Per-class F_1 with the `no_action` class, pooled over the three LORO folds (AdaBN + 3-seed ensemble). Each class contributes 210 test windows at seven classes and 420 at five.

Class	Single-link training		Shared trunk (A+B)		
	A	B	A	B	ABavg
<u>7-class</u>					
<code>no_action</code>	0.891	0.920	0.869	0.905	0.948
<code>walk</code>	0.795	0.889	0.834	0.906	0.928
<code>prepare_meal</code>	0.766	0.842	0.789	0.855	0.903
<code>lying_up</code>	0.648	0.706	0.722	0.758	0.801
<code>lying</code>	0.405	0.599	0.530	0.607	0.657
<code>sit</code>	0.429	0.549	0.543	0.562	0.614
<code>sit_up</code>	0.465	0.480	0.576	0.577	0.627
macro (7 classes)	0.628	0.712	0.695	0.739	0.783
<u>5-class</u>					
<code>walk</code>	0.901	0.938	0.879	0.954	0.968
<code>no_action</code>	0.901	0.923	0.873	0.910	0.946
<code>prepare_meal</code>	0.826	0.870	0.860	0.868	0.933
<code>get_up</code>	0.805	0.816	0.829	0.858	0.880
<code>sit_lying</code>	0.719	0.762	0.799	0.819	0.862
macro (5 classes)	0.830	0.862	0.848	0.882	0.918

Table A.38: The install-time adaptation of AdaBN, scored over the same operator-labelled windows of each live session. The two halves differ only in the batch-norm statistics. In detail, those batch-norm statistics are estimated in the room against those carried over from the pooled training rooms, meaning the input tensors, the weights and the fusion are identical.

Session	Deployment	n	No AdaBN		With AdaBN		Δ	
			Acc	macro- F_1	Acc	macro- F_1	Acc	macro- F_1
10 August, room e212	dual, prob. average	35	0.714	0.687	0.800	0.801	+0.086	+0.114
23 July, living room	single, link B	60	0.750	0.710	0.783	0.779	+0.033	+0.069

Table A.39: The nine continuous recordings, each 20 s long, with the sequence of activities performed in each. `no_action` marks a complete standstill mid-recording. Recordings 4 and 6 are the two shown in the main text. A bug occurred at recording 7 and thus it was discarded.

Rec.	Sequence performed	Timeline
1	walk → prepare_meal → sit → sit_up → walk	Fig. A.19
2	walk → lying → lying_up → prepare_meal	Fig. A.20
3	sit → no_action → sit_up → prepare_meal → lying → lying_up	Fig. A.21
4	lying → no_action → lying_up → walk	Fig. 5.4
5	no_action → prepare_meal → walk → sit	Fig. A.22
6	lying → no_action → lying_up → sit → no_action → sit_up	Fig. 5.5
8	prepare_meal → walk → lying → no_action → lying_up	Fig. A.23
9	no_action → walk → sit → sit_up → walk	Fig. A.24
10	walk → sit → no_action → sit_up → prepare_meal	Fig. A.25

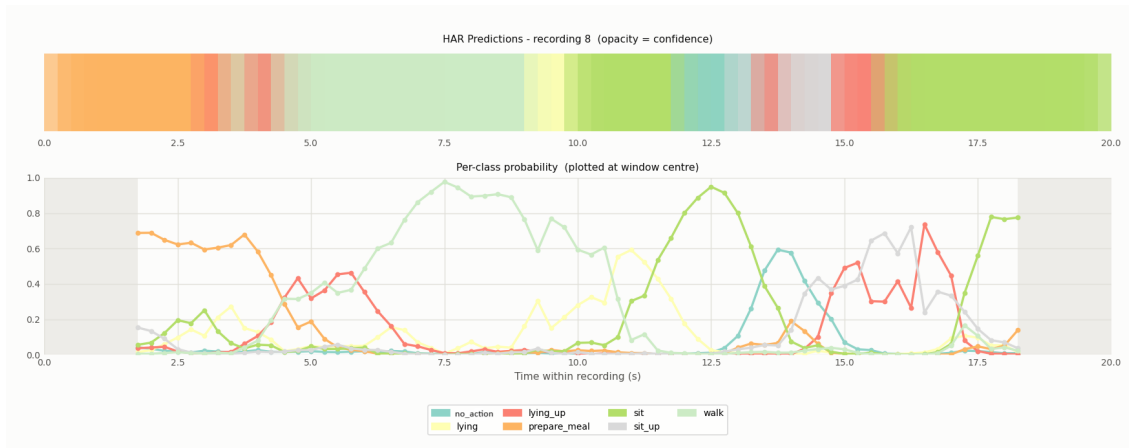


Figure A.23: Continuous recording 8: `prepare_meal` → `walk` → `lying` → `no_action` → `lying_up`. The model seems to identify all the actions. As already explained, `sit` and `lying` actions are difficult for the model to distinguish. The same case is for `sit_up` and `lying_up`. However, this limitation is fixed in the 5-class merged granularity.

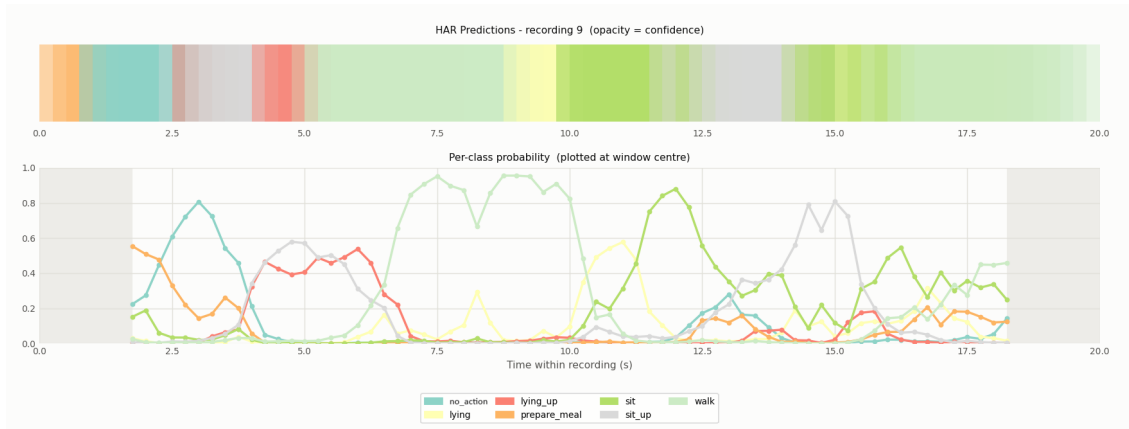


Figure A.24: Continuous recording 9: `no_action` → `walk` → `sit` → `sit_up` → `walk`. All sequence actions are identified. There are some false recognitions, like `sit_up` and `lying_up` after `no_action`, and the `sit` actions after `sit_up` towards the end.

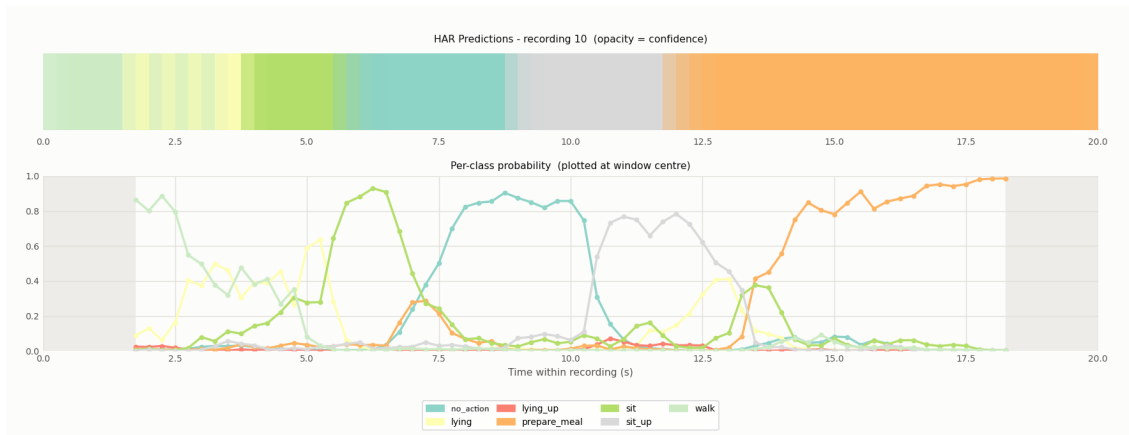


Figure A.25: Continuous recording 10: `walk` → `sit` → `no_action` → `sit_up` → `prepare_meal`. This sequence is almost perfect with the exception of the false-lying predictions before `sit`.

Bibliography

- [1] U. Desa, “World population prospects 2019: Highlights,” New York (US): United Nations Department for Economic and Social Affairs, vol. 11, no. 1, p. 125, 2019. (Cited on pages 1 and 11.)
- [2] S. Pleschberger, E. Reitingner, B. Trukeschitz, and P. Wosko, “Older people living alone (opla)–non-kin-carers’ support towards the end of life: qualitative longitudinal study protocol,” BMC geriatrics, vol. 19, no. 1, p. 219, 2019. (Cited on pages 1 and 11.)
- [3] X. Niu, S. Li, Y. Zhang, Z. Liu, D. Wu, R. C. Shah, C. Tanriover, H. Lu, and D. Zhang, “Wimonitor: Continuous long-term human vitality monitoring using commodity wi-fi devices,” Sensors, vol. 21, no. 3, p. 751, 2021. (Cited on pages 1, 3, 11, 12, 13 and 22.)
- [4] J. Chung, I. Pretzer-Aboff, P. Parsons, K. Falls, and E. Bulut, “Using a device-free wi-fi sensing system to assess daily activities and mobility in low-income older adults: protocol for a feasibility study,” JMIR Research Protocols, vol. 13, no. 1, p. e53447, 2024. (Cited on pages 1, 3, 11, 12, 13, 21, 22 and 47.)
- [5] J. E. Gaugler, M. Hovater, D. L. Roth, J. A. Johnston, R. L. Kane, and K. Sarsour, “Analysis of cognitive, functional, health service use, and cost trajectories prior to and following memory loss,” Journals of Gerontology Series B: Psychological Sciences and Social Sciences, vol. 68, no. 4, pp. 562–567, 2013. (Cited on page 1.)
- [6] F. Li, M. A. A. Al-Qaness, Y. Zhang, B. Zhao, and X. Luan, “A robust and device-free system for the recognition and classification of elderly activities,” Sensors, vol. 16, no. 12, p. 2043, 2016. (Cited on pages 1, 11, 12 and 22.)
- [7] F. Miao, Y. Huang, Z. Lu, T. Ohtsuki, G. Gui, and H. Sari, “Wi-fi sensing techniques for human activity recognition: Brief survey, potential challenges, and research directions,” ACM Computing Surveys, vol. 57, no. 5, pp. 1–30, 2025. (Cited on pages 1, 2, 7, 8, 13, 14, 15 and 19.)
- [8] M. G. Moghaddam, Device-free Human Activity Recognition for the Kitchen. PhD thesis, University of Ottawa, 2026. (Cited on pages 1, 7, 8, 11, 17, 19 and 89.)
- [9] Z. Yang, Z. Zhou, and Y. Liu, “From rssi to csi: Indoor localization via channel response,” ACM Computing Surveys (CSUR), vol. 46, no. 2, pp. 1–32, 2013. (Cited on pages 1 and 9.)
- [10] M. A. Al-Qaness, M. Abd Elaziz, S. Kim, A. A. Ewees, A. A. Abbasi, Y. A. Alhaj, and A. Hawbani, “Channel state information from pure communication to sense and track human motion: A survey,” Sensors, vol. 19, no. 15, p. 3329, 2019. (Cited on pages 1 and 9.)

- [11] W. Wang, A. X. Liu, M. Shahzad, K. Ling, and S. Lu, "Understanding and modeling of wifi signal based human activity recognition," in Proceedings of the 21st annual international conference on mobile computing and networking, pp. 65–76, 2015. (Cited on pages 1, 9 and 23.)
- [12] D. Halperin, W. Hu, A. Sheth, and D. Wetherall, "Tool release: Gathering 802.11 n traces with channel state information," ACM SIGCOMM computer communication review, vol. 41, no. 1, pp. 53–53, 2011. (Cited on pages 2, 10 and 23.)
- [13] M. Atif, S. Muralidharan, H. Ko, and B. Yoo, "Wi-esp-a tool for csi-based device-free wi-fi sensing (dfws)," Journal of Computational Design and Engineering, vol. 7, no. 5, pp. 644–656, 2020. (Cited on pages 2, 10 and 23.)
- [14] Z. Wen, Y. Ruan, X. Wang, J. Zhou, H. Gao, and T. Li, "Esp-fi har: A low-power wifi csi dataset for ad-hoc iot human activity recognition," Ad Hoc Networks, p. 104192, 2026. (Cited on pages 2, 10, 19 and 51.)
- [15] R. R. Kumar, A. S. Kumar, N. S. Harsha, P. A. Sarma, and R. S. Varsha, "Real-time human activity detection using wi-fi csi and lstm on edge devices," in Conference on Social and Sustainable Innovation in Technology & Engineering (SASI-ITE 2025), pp. 433–450, Atlantis Press, 2025. (Cited on pages 2, 8, 11, 19 and 24.)
- [16] K. Liu, "Star: A privacy-preserving, energy-efficient edge ai framework for human activity recognition via wi-fi csi in mobile and pervasive computing environments," arXiv preprint arXiv:2510.26148, 2025. (Cited on pages 2, 11, 19, 24, 28 and 30.)
- [17] J. Strohmayer and M. Kampel, "Wifi csi-based long-range through-wall human activity recognition with the esp32," in International Conference on Computer Vision Systems, pp. 41–50, Springer, 2023. (Cited on pages 2, 10 and 24.)
- [18] Q. Bu, G. Yang, X. Ming, T. Zhang, J. Feng, and J. Zhang, "Deep transfer learning for gesture recognition with wifi signals," Personal and Ubiquitous Computing, vol. 26, no. 3, pp. 543–554, 2022. (Cited on pages 2, 18 and 19.)
- [19] F. Wang, T. Zhang, W. Xi, H. Ding, G. Wang, D. Zhang, Y. Cui, F. Liu, J. Han, J. Xu, et al., "A survey on wi-fi sensing generalizability: Taxonomy, techniques, datasets, and future research prospects," IEEE Communications Surveys & Tutorials, 2026. (Cited on pages 2, 13, 14, 15 and 19.)
- [20] K. Ye, S. Wu, Y. Cai, L. Zhou, L. Xiao, X. Zhang, Z. Zheng, and J. Lin, "Transfer-learning-based human activity recognition using antenna array," Remote Sensing, vol. 16, no. 5, p. 845, 2024. (Cited on pages 2, 17, 18 and 19.)
- [21] K. Xu, J. Wang, H. Zhu, and D. Zheng, "Evaluating self-supervised learning for wifi csi-based human activity recognition," ACM Transactions on Sensor Networks, vol. 21, no. 2, pp. 1–38, 2025. (Cited on pages 2, 17 and 19.)
- [22] D. Varga, "Mitigating data leakage in a wifi csi benchmark for human action recognition," Sensors, vol. 24, no. 24, p. 8201, 2024. (Cited on pages 2 and 14.)

- [23] S. M. Hernandez and E. Bulut, "Wifi sensing on the edge: Signal processing techniques and challenges for real-world systems," IEEE Communications Surveys & Tutorials, vol. 25, no. 1, pp. 46–76, 2022. (Cited on pages 3, 13, 15 and 16.)
- [24] W. E. Wong, A. H. Wong, W. Q. Peh, and C. K. Tan, "A high-dimensional, multi-transceiver channel state information dataset for enhanced human activity recognition," Data in Brief, vol. 55, p. 110673, 2024. (Cited on pages 3, 16 and 19.)
- [25] S. Sirisilla, B. Pardhasaradhi, J. Zhou, A. Jha, and L. R. Cenkeramaddi, "Lightweight 2d cnn fusion for human activity recognition using multi-transceiver csi data on edge devices," in 2025 IEEE Applied Sensing Conference (APSCON), pp. 299–302, IEEE, 2025. (Cited on pages 3, 16 and 19.)
- [26] K. Khamaisi and B. Rodrigues, "Less is more: The dilution effect in multi-link wireless sensing," arXiv preprint arXiv:2602.10823, 2026. (Cited on pages 3, 16, 24, 25 and 35.)
- [27] J. Chen, K. Yang, X. Zheng, S. Dong, L. Liu, and H. Ma, "Wimix: A lightweight multimodal human activity recognition system based on wifi and vision," in 2023 IEEE 20th International Conference on Mobile Ad Hoc and Smart Systems (MASS), pp. 406–414, IEEE, 2023. (Cited on pages 7 and 8.)
- [28] S. K. Yadav, K. Tiwari, H. M. Pandey, and S. A. Akbar, "A review of multimodal human activity recognition with special emphasis on classification, applications, challenges and future directions," Knowledge-Based Systems, vol. 223, p. 106970, 2021. (Cited on page 7.)
- [29] A. Y. Radwan, M. Yildirim, N. Hasanzadeh, H. Tabassum, and S. Valaee, "A tutorial-cum-survey on self-supervised learning for wi-fi sensing: Trends, challenges, and outlook," IEEE Communications Surveys & Tutorials, 2025. (Cited on pages 7 and 17.)
- [30] P. Kumari, L. Mathew, and P. Syal, "Increasing trend of wearables and multimodal interface for human activity monitoring: A review," Biosensors and Bioelectronics, vol. 90, pp. 298–307, 2017. (Cited on page 7.)
- [31] G. Geraci, F. Meneghello, F. Wilhelmi, D. Lopez-Perez, I. Val, L. G. Giordano, C. Cordeiro, M. Ghosh, E. Knightly, and B. Bellalta, "Wi-fi: Twenty-five years and counting," arXiv preprint arXiv:2507.09613, 2025. (Cited on pages 7, 8, 22, 26 and 36.)
- [32] S. Patil and A. Kobsa, "Privacy considerations in awareness systems: designing with privacy in mind," in Awareness systems: advances in theory, methodology and design, pp. 187–206, Springer, 2009. (Cited on page 8.)
- [33] A. Lentzas and D. Vrakas, "Non-intrusive human activity recognition and abnormal behavior detection on elderly people: A review," Artificial Intelligence Review, vol. 53, no. 3, pp. 1975–2021, 2020. (Cited on page 8.)
- [34] T. Baltrušaitis, C. Ahuja, and L.-P. Morency, "Multimodal machine learning: A survey and taxonomy," IEEE transactions on pattern analysis and machine intelligence, vol. 41, no. 2, pp. 423–443, 2018. (Cited on page 8.)

- [35] L. Guo, L. Wang, C. Lin, J. Liu, B. Lu, J. Fang, Z. Liu, Z. Shan, J. Yang, and S. Guo, “Wiar: A public dataset for wifi-based activity recognition,” IEEE Access, vol. 7, pp. 154935–154945, 2019. (Cited on pages 8 and 9.)
- [36] M. Youssef, M. Mah, and A. Agrawala, “Challenges: device-free passive localization for wireless environments,” in Proceedings of the 13th annual ACM international conference on Mobile computing and networking, pp. 222–229, 2007. (Cited on page 8.)
- [37] G. Deak, K. Curran, J. Condell, and D. Deak, “Detection of multi-occupancy using device-free passive localisation,” IET Wireless Sensor Systems, vol. 4, no. 3, pp. 130–137, 2014. (Cited on page 9.)
- [38] X. Liu, J. Cao, S. Tang, and J. Wen, “Wi-sleep: Contactless sleep monitoring via wifi signals,” in 2014 IEEE Real-Time Systems Symposium, pp. 346–355, IEEE, 2014. (Cited on page 9.)
- [39] W. Wang, A. X. Liu, M. Shahzad, K. Ling, and S. Lu, “Device-free human activity recognition using commercial wifi devices,” IEEE Journal on Selected Areas in Communications, vol. 35, no. 5, pp. 1118–1131, 2017. (Cited on page 9.)
- [40] Y. Xie, Z. Li, and M. Li, “Precise power delay profiling with commodity wifi,” in Proceedings of the 21st Annual international conference on Mobile Computing and Networking, pp. 53–64, 2015. (Cited on page 10.)
- [41] J. A. Armenta-Garcia, F. F. Gonzalez-Navarro, J. Caro-Gutierrez, and C. I. Garcia-Reyes, “Tools and methods for achieving wi-fi sensing in embedded devices,” Sensors, vol. 25, no. 19, p. 6220, 2025. (Cited on page 10.)
- [42] Z.-Y. Lim, L.-Y. Ong, and M.-C. Leow, “Radio frequency-based human activity dataset collected using esp32 microcontroller in line-of-sight and non-line-of-sight indoor experiment setups,” Data in Brief, vol. 57, p. 111101, 2024. (Cited on pages 10 and 19.)
- [43] C. Yin, Z. Huang, H. Jiang, J. Chen, Z. WANG, C. Wang, Z. Chen, Z. Chen, and H. Yan, “A spatio-temporal feature distribution network for device-free power inspection activity using wifi csi,” IEEE Transactions on Industrial Informatics, 2025. (Cited on pages 11 and 19.)
- [44] Y. Wang, J. Liu, Y. Chen, M. Gruteser, J. Yang, and H. Liu, “E-eyes: Device-free location-oriented activity identification using fine-grained wifi signatures,” in Proceedings of the 20th annual international conference on Mobile computing and networking, pp. 617–628, 2014. (Cited on pages 12, 13 and 22.)
- [45] X. Chen, C. Li, C. Jiang, W. Meng, and W. Xiao, “Wiphase: A human activity recognition approach by fusing of reconstructed wifi csi phase features,” IEEE Transactions on Mobile Computing, vol. 24, no. 1, pp. 394–406, 2024. (Cited on page 14.)

- [46] H. Huang and S. Lin, “Widet: Wi-fi based device-free passive person detection with deep convolutional neural networks,” in Proceedings of the 21st ACM international conference on modeling, analysis and simulation of wireless and mobile systems, pp. 53–60, 2018. (Cited on page 14.)
- [47] P. F. Moshiri, R. Shahbazian, M. Nabati, and S. A. Ghorashi, “A csi-based human activity recognition using deep learning,” Sensors, vol. 21, no. 21, p. 7225, 2021. (Cited on page 14.)
- [48] S. M. Hernandez and E. Bulut, “Online stream sampling for low-memory on-device edge training for wifi sensing,” in Proceedings of the 2022 ACM Workshop on Wireless Security and Machine Learning, pp. 9–14, 2022. (Cited on page 15.)
- [49] S. Gao, D. Yan, F. Han, Y. Zhang, and Y. Yan, “Speedfi: Fine-grained speed estimation for bodyweight exercise with wifi,” ACM Transactions on Sensor Networks, 2026. (Cited on pages 16, 23, 24, 26, 30 and 35.)
- [50] M. R. M. Bastwesy, Achieving Robust and Cost-effective Head Nodding and Body Gesture Recognition in Real-world Scenarios with Wi-Fi CSI Analysis. PhD thesis, Kyushu University, 2024. (Cited on page 16.)
- [51] M. Touhiduzzaman, J. Chung, I. Pretzer-Aboff, and E. Bulut, “Wireless sensing-based daily activity tracking system deployment in low-income senior housing environments,” in Proceedings of the 30th Annual International Conference on Mobile Computing and Networking, pp. 2260–2267, 2024. (Cited on page 16.)
- [52] S. Chaudhari, K. Pise, D. Fukate, and S. Gawande, “Device-free indoor localization with esp32 wi-fi csi fingerprints: Grid-based regression, feature modeling, and multi-link experiments,” 2026. (Cited on page 16.)
- [53] S. Arshad, C. Feng, R. Yu, and Y. Liu, “Leveraging transfer learning in multiple human activity recognition using wifi signal,” in 2019 IEEE 20th International Symposium on "A World of Wireless, Mobile and Multimedia Networks" (WoWMoM), pp. 1–10, IEEE, 2019. (Cited on pages 17 and 18.)
- [54] H. Li, X. Chen, H. Du, X. He, J. Qian, P.-J. Wan, and P. Yang, “Wi-motion: A robust human activity recognition using wifi signals,” arXiv preprint arXiv:1810.11705, 2018. (Cited on pages 23 and 24.)
- [55] W. Zhuang, Y. Lu, Y. Shen, and J. Su, “Human daily breathing monitoring via analysis of csi ratio trajectories for wifi link pairs on the i/q plane,” Sensors, vol. 24, no. 22, p. 7352, 2024. (Cited on page 23.)
- [56] D. Zhang, Y. Zeng, F. Zhang, and J. Xiong, “Wifi csi-based vital signs monitoring,” in Contactless Vital Signs Monitoring, pp. 231–255, Elsevier, 2022. (Cited on page 25.)
- [57] Sseed Studio, “Getting started with Sseed Studio XIAO ESP32-S3.” https://wiki.seeedstudio.com/xiao_esp32s3_getting_started/, 2024. [Accessed: 13-May-2026]. (Cited on pages 27 and 28.)

- [58] F. Abuhoureyah, Y. C. Wong, A. S. B. M. Isira, and M. N. Al-Andoli, “Csi-based location independent human activity recognition using deep learning,” Human-Centric Intelligent Systems, vol. 3, no. 4, pp. 537–557, 2023. (Cited on pages 28 and 29.)
- [59] A. Zhuravchak, O. Kapshii, and E. Pournaras, “Human activity recognition based on wi-fi csi data-a deep neural network approach,” Procedia Computer Science, vol. 198, pp. 59–66, 2022. (Cited on pages 28 and 30.)
- [60] R. K. Pearson, Y. Neuvo, J. Astola, and M. Gabbouj, “Generalized hampel filters,” EURASIP Journal on Advances in Signal Processing, vol. 2016, no. 1, p. 87, 2016. (Cited on pages 28 and 29.)
- [61] D. Avola, A. Bernardini, F. Danese, M. Lezoche, M. Mancini, D. Pannone, and A. Ranaldi, “Transformer-based person identification via wi-fi csi amplitude and phase perturbations,” arXiv preprint arXiv:2507.12854, 2025. (Cited on pages 28, 29 and 30.)
- [62] T. Z. Chowdhury, Using Wi-Fi channel state information (CSI) for human activity recognition and fall detection. PhD thesis, University of British Columbia, 2018. (Cited on page 31.)
- [63] Y. Li, N. Wang, J. Shi, J. Liu, and X. Hou, “Revisiting batch normalization for practical domain adaptation,” arXiv preprint arXiv:1603.04779, 2016. (Cited on page 44.)
- [64] M. T. Pham, L. T. Hoang, H. D. T. Hong, T. D. Minh, and T. P. Nguyen, “An wifi csi signal enhancement framework for activity recognition using machine learning automatic segmentation,” SN Computer Science, vol. 5, no. 5, p. 524, 2024. (Cited on page 104.)

Abstract: Device-free human activity recognition (HAR) from Wi-Fi Channel State Information (CSI) offers a way to monitor the elderly at home without intruding on them. No wearables and no cameras are needed in the room. In practice, though, most reported systems are evaluated under conditions that make them look better than they really are. In detail, the activity may be performed on the direct line of sight between transmitter and receiver, the train/test split may share rooms, and the system may only be evaluated on pre-segmented clips and no continuous real routines. As a result, performance tends to fall once the system is deployed live.

This thesis studies CSI-based HAR under conditions closer to a real elderly-care deployment, built entirely on low-cost ESP32-S3 devices rather than specialised chipsets. We compare a conventional single transmitter-receiver link against a double-pair configuration and, to our knowledge, for the first time, evaluate the double-pair setup with the activity performed outside the direct line of sight of either link. The central question is cross-room generalisation. In other words, we train in two environments and test in another. A Siamese trunk that processes each link with shared weights, combined with targeted augmentation and label-free test-time adaptation, reaches 0.766 macro-F1 on six activities and 0.918 macro-F1 once the chair and floor endpoints are merged, in rooms never seen during training. We then move beyond the offline, pre-segmented clip evaluation common in the literature and assess the model in two more demanding regimes. First, we implement a real-time setting, where predictions are produced live as the person moves, and secondly a continuous-recording setting, where the model must run over uninterrupted streams that include the transitions between actions and idle periods. The model held up well under both regimes: real-time performance matched the offline evaluation, and continuous recordings were recognised reliably apart from the transition moments between activities. Evaluations reveal the second link’s value depends on the stage of the workflow. During data collection, both links record every repetition simultaneously from distinct geometries, doubling the training set at no extra cost to the participant. During deployment, however, a single well-placed link trained inside the shared trunk comes within 3 to 4 points of macro-F₁ of the two-link fusion, which does not justify the hardware and synchronisation overhead of a second live pair. The recipe we recommend is therefore to collect with two pairs and deploy with one.

Keywords: Human Activity Recognition, Device-Free Wi-Fi Sensing, Cross-Environment Generalization, Elderly Care Monitoring, Edge Hardware, Channel State Information